Senior/Lead Java Full-Stack Engineer: Deep Technical Reference and Interview Guide

Technical reference and 142-question interview bank grounded in a working financial-technology project

Synthetic demonstration project

Independently designed for engineering-interview preparation. It is not a real client system and uses no real customer data, credentials, proprietary APIs or confidential processes.

Document: PDF 2 of 2 - Deep Technical Reference and Interview Guide

Covers: Java, Spring Boot, Hibernate, system design, PostgreSQL/Oracle/MongoDB, Angular, DevOps, cloud, OAuth2/JWT/OWASP, AI-assisted development, technical leadership

Synthetic data only. Educational implementation - not a production financial system.

Companion: SSNC_FinTech_Platform_Architecture_and_Workflows.pdf

Status wording used throughout: executed = run and passing in the build environment; written, not executed = present but not run here; conceptual = design discussion only.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Table of contents

How to use this handbook, project setup and module tour **5**

How to use this guide 5
Project setup and execution 5
Module-by-module tour 6
A 5-minute architecture story (rehearse this) 6

Java, Spring Boot and Hibernate reference **7**

JVM and garbage collection 7
Concurrency 7
Collections and data structures 8
Dependency injection and Spring fundamentals 8
Spring transactions 8
Hibernate / JPA 9
REST APIs and exception handling 9
Security (Spring Security 6) 10
Testing 10
Performance 10

System design and distributed systems reference **11**

HLD and LLD - how to structure an answer 11
Microservices and service boundaries 11
Idempotency 11
Consistency 11
Concurrency control (at system level) 12
Resilience patterns 12
API versioning and backward compatibility 12
Scalability 12
Failure handling checklist 13
Distributed tracing, correlation, observability in design 13

Databases reference **14**

SQL essentials (with the project's tables) 14
PostgreSQL specifics 14
Oracle compatibility (conceptual) 14
MongoDB use cases 15
Indexes and query plans 15
Isolation levels, locking, deadlocks 15
Connection pools 16
Schema migrations 16

Angular reference **17**

Components, templates and change detection 17

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Services and dependency injection . . . 17
RxJS . . . 17
Signals . . . 17
Reactive forms . . . 17
Routing . . . 18
HTTP interceptors . . . 18
Performance . . . 18
Testing . . . 18
Secure frontend practices . . . 18
DevOps, cloud and security reference . . . 19
Docker . . . 19
Kubernetes . . . 19
CI/CD . . . 19
Cloud architecture (AWS flavour) . . . 19
OAuth 2.0 and OpenID Connect . . . 20
JWT validation . . . 20
OWASP . . . 20
Secrets management . . . 20
Monitoring, logging, tracing . . . 20
Performance troubleshooting . . . 20
AI-assisted development reference . . . 21
GitHub Copilot workflows . . . 21
Specification-driven development . . . 21
Context engineering . . . 21
Reusable instructions and handoff . . . 21
Reviewing AI-generated code . . . 21
Test generation . . . 22
Refactoring with assistants . . . 22
Safe dependency upgrades . . . 22
Guardrails (organisation level) . . . 22
Technical leadership and agile delivery . . . 23
Product requirements (example PRD) and feature specifications . . . 23
High-level and low-level design . . . 24
Sprint task breakdown, dependencies and risk register . . . 25
Pull-request checklist and code-review checklist . . . 27
Code-review scenarios . . . 28
Troubleshooting exercises . . . 30
Mentoring junior developers . . . 31

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Release planning, release-readiness checklist and rollback plan . . . 33
Post-incident review (blameless) . . . 34
Job-description traceability matrix . . . 36
Interview question bank . . . 42
Java and Spring Boot . . . 42
Databases . . . 49
Microservices, distributed systems and financial transaction integrity . . . 53
Angular and REST APIs . . . 59
Security . . . 64
Testing, CI/CD and cloud . . . 67
Performance and reliability . . . 70
AI-assisted development and technical leadership . . . 73
React frontend edition . . . 77

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

How to use this handbook, project setup and module tour

How to use this guide

This handbook is the **second PDF**: a technical reference plus a bank of interview questions with model answers, all grounded in a working project (`ssnc-fintech-platform`). Read it in three passes:

1. **Orientation (1 evening):** this chapter and the project tour; run the application (below) so every later sentence has something concrete behind it.
2. **Reference chapters (a week):** Java/Spring, system design, databases, Angular, DevOps/security, AI-assisted development. Each concept is explained with an *analogy*, a *minimal example*, the *pitfall*, and *where it lives in the project*.
3. **Question bank (ongoing):** 130+ questions, 40+ of them scenario based. Answer aloud first, then compare with the model answer; practise the "likely follow-up" questions - they are what separates a memorised answer from understanding.

How model answers are structured: **Short answer** (what you would say in 30 seconds) -> **Reasoning and implementation** -> **Alternatives and trade-offs** -> **Risks and pitfalls** -> **Likely follow-ups** -> **In this project** (a pointer you can show). Interview tip: *lead with the principle, then the mechanism, then the evidence* ("I would take a row lock, because the check-then-act is a race; here is how it looks; we proved it with a 20-thread test").

> Honesty: the repository states what was executed and what was not (see the validation report). In an interview, do the same - "I ran X and measured Y; I did not run Z" is a strength, not a weakness.

Project setup and execution

```
# prerequisites: JDK 21, Maven 3.9, Node 22+ (any recent LTS), a Chromium-based browser for UI tests.
Docker is optional.
# 1. build everything and run all tests
mvn -f backend/pom.xml verify                          # unit + integration (incl. embedded
PostgreSQL); jars in backend/*/target
( cd frontend && npm ci && npx ng lint && npx ng build )
# 2. start the backend (in-memory H2, synthetic seed data, local token issuer, simulator for the
clearing network)
scripts/run-local.sh                                   # API http://localhost:18480, simulator :18490,
Swagger UI /swagger-ui.html
# 3. serve the UI (static + /api proxy)  ->  http://localhost:4280
node scripts/serve-frontend.mjs
# 4. sign in: alice.customer / bob.customer / sam.support / omar.ops / olivia.ops / adam.admin
(password = your LOCAL_DEMO_PASSWORD, default "demo-password")
# 5. exercise it
node tests/e2e/e2e-api.mjs --with-chaos                # live API scenarios incl. timeouts, circuit
breaker, reconciliation
node tests/e2e/ui-e2e.mjs                              # browser journeys
node performance/local-baseline.mjs                    # local load baseline (dev-machine numbers only)
# 6. optional: full local infrastructure with PostgreSQL, Grafana, Prometheus, Jaeger
cp .env.example .env   # edit CHANGE_ME values, then:
docker compose -f infrastructure/docker-compose.yml --env-file .env up --build
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Things to try during practice: submit a withdrawal whose cents are `.13` (declined), `.99` (two transient 503s then success), `.66` (timeout but executed -> reconciliation); switch the simulator to `FAIL_5XX` and watch the circuit breaker open on the dashboard's service-health card; submit 60,000 as an operator and approve it as the *other* operator; open the same account as the wrong customer and find the `ACCESS_DENIED` audit row.

Module-by-module tour

Module (package)	What it does	The one thing to be able to explain
`common`	`MoneyPolicy`, `ApiException`, error envelope, `GlobalExceptionHandler`, correlation-id and size-limit filters, paging whitelist, masking	why money is a string on the wire and why excess precision is rejected
`security`	JWT resource server, `AccessPolicy`, rate limiter, local issuer (profile-gated)	two-layer authorization and why foreign ids return 404
`customer`	profiles, PII masking by role, suspend/ reactivate	masking is a data-exposure control, enforced server-side
`account`	accounts, lifecycle state machine, `debit/ credit` invariants, closure rules	the entity enforces its own invariants; balance changes only via the ledger service
`transaction`	state machine, validator, idempotent submit, outbox, processor, ledger, reconciliation, maintenance	the three-way outcome classification and why `Unknown` never refunds
`clearing`	port + REST adapter (timeouts, retry with jitter, circuit breaker) + in-process substitute	retries are only safe because the operation is idempotent
`audit`	immutable audit events	independent transaction for denials; append-only trigger
`ops`	dashboard read models, health, metrics	observability as a feature, not an afterthought
`clearing-simulator`	deterministic + chaos stand-in for the external network	testing failure modes you cannot trigger on a real provider
`frontend`	Angular 20 SPA	token in memory, idempotency key policy, state views, role-aware navigation

A 5-minute architecture story (rehearse this)

"It is a modular monolith because the money invariants need one transaction boundary; splitting account from transaction would force a saga for something a single ACID commit does for free. A client submits a transaction with an idempotency key; one database transaction records it, validates it and writes an outbox event. A worker takes the event, locks the account rows in id order and applies the ledger postings; withdrawals reserve funds first, call the external network *outside* any transaction, and classify the answer as authorized, declined, not-sent or unknown. Unknown goes to reconciliation with the funds held. Security is a JWT resource server with role rules plus object-level checks; every change and denial is audited. I proved the properties with concurrency tests on H2 and on a real PostgreSQL, chaos scenarios against a simulator, a ledger-integrity check after every run, and I measured one optimisation before and after."

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Java, Spring Boot and Hibernate reference

JVM and garbage collection

Analogy: the heap is a big workshop floor; the garbage collector is a cleaner who walks around removing tools nobody is holding any more. Young objects die fast (most requests create short-lived garbage), so the floor is divided into a small "young" area that is swept often and cheaply and an "old" area swept rarely and expensively (generational hypothesis).

- **Memory areas:** heap (objects), metaspace (class metadata, native memory), thread stacks, code cache, direct buffers. A container's RSS is *more* than heap - leave headroom (`MaxRAMPercentage=75`).
- **G1 (default):** region-based, mostly concurrent, targets a pause goal (`-XX:MaxGCPauseMillis=200`). Young collections are stop-the-world but short; mixed collections reclaim old regions; **Full GC** is the failure mode (heap too small, humongous allocations, evacuation failure). ZGC/ Shenandoah: sub-millisecond pauses, larger footprint - use when latency SLOs demand it and measurements show pause-bound behaviour.
- **Diagnosing:** `-Xlog:gc*`, `jvm_gc_pause_seconds` by cause, `jcmd <pid> GC.heap_dump`, JFR (`JFR.start`), MAT dominator tree. *Example (beginner):* heap-after-GC low points rise hourly = leak. *Example (advanced):* "to-space exhausted" = allocation outruns evacuation: reduce allocation or raise the heap before touching flags.
- **Common leak sources in services:** unbounded caches/maps, `ThreadLocal`s on pooled threads, listeners never removed, unpaged queries. **In this project:** list endpoints are paged (cap 100), the rate-limiter map is bounded, heap dump on OOM is enabled in the image.
- **Virtual threads (Java 21):** cheap blocking threads (`spring.threads.virtual.enabled=true`). They remove *thread* scarcity, not *connection* scarcity - the Hikari pool still limits DB concurrency; `synchronized` blocks pin carrier threads in 21.

Concurrency

Analogy: a bank branch with one safe: a lock is the key to the safe; a deadlock is two tellers each holding one key and waiting for the other's.

- **Memory model:** `volatile` gives visibility/ordering for a single variable, not atomic compound actions; `synchronized`/locks give mutual exclusion + happens-before; `java.util.concurrent` atomics use CAS.
- **Race condition vs data race:** a data race is unsynchronised conflicting access; a *race condition* is a logic error that depends on timing - a check-then-act on a balance is a race condition even with fully synchronised accessors. (Therefore the database lock, not a Java `synchronized`, is what protects a balance in a multi-instance deployment.)
- **Deadlock:** needs mutual exclusion, hold-and-wait, no preemption, circular wait. Break circular wait with a **global lock order** (the ledger locks accounts in ascending id order). Detect with thread dumps / the database's detector; bound waits with timeouts.
- **Executors:** size pools for the workload (CPU-bound ~ cores, I/O-bound larger or virtual threads), bound queues, name threads, handle rejection; never use unbounded pools for request work.
- **CompletableFuture / structured concurrency** for fan-out; always set timeouts. **Thread-safety of Spring beans:** singletons must be stateless (or carefully synchronised: `TokenBucketRateLimiter` synchronises per bucket).
-

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Testing concurrency: start threads together with a `CountDownLatch`, one thread per task, assert the final state and invariants, add `@Timeout` (a hung latch is a test bug - one occurred while building this project: a pool smaller than the task count).

Collections and data structures

Structure	Use	Complexity / note
`ArrayList`	default list	O(1) index, amortised O(1) append, O(n) middle insert
`LinkedList`	rarely	O(1) at ends, O(n) index; poor cache locality - almost never the right choice
`HashMap`	lookup	O(1) average; needs consistent `equals/hashCode`; resize cost; not thread-safe
`ConcurrentHashMap`	shared map	lock-striped/CAS; `compute*` operations are atomic per key (used for the rate-limiter buckets and the simulator store)
`TreeMap/TreeSet`	sorted	O(log n), range queries
`EnumMap/EnumSet`	enum keys	array-backed, fastest (the state machine uses `EnumSet`)
`ArrayDeque`, `PriorityQueue`	queues	deque for stacks/queues; heap for scheduling
`List.of/Set.of/Map.of`	immutables	no nulls, defensive copying (`copyOf`)

Pitfalls: mutable keys, `equals` without `hashCode`, `BigDecimal.equals` (scale) vs `compareTo`, modifying a collection while iterating, `Arrays.asList` is fixed-size, streams are single-use and `parallelStream` on the common pool is rarely appropriate in a server.

Dependency injection and Spring fundamentals

Analogy: instead of each class building its own tools, a workshop foreman (the container) hands them the right tools when they arrive.

- Constructor injection (immutability, testability, no hidden dependencies); a single constructor needs no `@Autowired`. Scopes: singleton (default), prototype, request.
- **Resolution:** by type; ties broken by `@Primary`, then `@Qualifier`, then parameter/field **name**. *Gotcha from this project:* with two `TransactionTemplate` beans, marking one `@Primary` would override name matching and silently give the read-only injection point the wrong template.
- **Configuration:** `@ConfigurationProperties` records (typed, validated), profiles, `@ConditionalOnProperty` (the local issuer exists only when enabled), externalised configuration order (env > application-{profile}.yml > application.yml).
- **Lifecycle:** `@PostConstruct`, `SmartLifecycle`, graceful shutdown phases; AOP proxies (transactions, security, async) apply only to calls that go **through the proxy** - self-invocation bypasses them.
- **Auto-configuration:** conditional beans; `--debug` prints the conditions report; back-off semantics (a user-defined `JwtDecoder` makes the issuer-based one back off).

Spring transactions

- `@Transactional` creates a proxy that begins/commits/rolls back around the method. **Rollback rules:** unchecked exceptions roll back by default, checked do not (unless `rollbackFor`).
-

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Propagation: `REQUIRED` (join or create), `REQUIRES_NEW` (suspend and start a new one - used for audit rows of denials that must survive a rollback), `MANDATORY` (must already exist - used for ledger/audit/outbox code that must never run unprotected), `NOT_SUPPORTED`, `NEVER`, `NESTED` (savepoints).

- **Pitfalls:** self-invocation, `private` methods, exceptions swallowed inside (transaction marked rollback-only), long transactions holding connections, I/O inside a transaction, lazy loading outside it (`open-in-view: false`), `readOnly=true` is a hint (not a security feature).
- **Programmatic `TransactionTemplate`:** when the boundary must wrap *some* statements and exclude others - the processor's external call. Also avoids proxy surprises.
- **Isolation/locking:** see the database chapter. **Events:** `@TransactionalEventListener(AFTER_COMMIT)` for post-commit work - or, for reliability, the outbox (this project) because an after-commit listener can lose the event if the process dies right after commit.

Hibernate / JPA

- **Entity lifecycle:** transient -> managed (persistence context, dirty checking) -> detached -> removed. `save()` on a managed entity is a no-op for updates; flush happens at commit/query.
- **`isNew` trap:** Spring Data's `save()` calls `persist` when the id is null or the **version attribute is null**, else `merge`. With assigned UUID ids and a *primitive or non-null* version, a brand-new entity is merged, Hibernate sees "version 0 but no row" and throws a stale-state/optimistic failure. *Real bug found here*; fix: wrapper `Long` version.
- **Fetching:** LAZY by default for associations; N+1 arises when iterating lazy associations or running one query per parent - fix with fetch joins, `@EntityGraph`, batch fetching (`default_batch_fetch_size`), or DTO projections. This project avoids associations entirely (ids only) to keep aggregate boundaries explicit.
- **Locking:** `@Version` optimistic; `@Lock(PESSIMISTIC_WRITE)` + `jakarta.persistence.lock.timeout` for `SELECT ... FOR UPDATE` (here with a 5 s bound).
- **Mapping tips:** `@Enumerated(STRING)` (never ordinal), `BigDecimal` with `precision/scale`, `Instant` in UTC, `@Column(updatable=false)` for immutables, `ddl-auto: validate` with Flyway.
- **Specifications/Criteria** for dynamic filters (parameterised - no injection); projections for read models; avoid `findAll()` on growing tables.
- **Batching:** `hibernate.jdbc.batch_size`, `order_updates`; IDENTITY ids disable insert batching (UUID/sequence allow it).

REST APIs and exception handling

Analogy: an API is a restaurant menu: stable names, clear descriptions, and the kitchen can change as long as the dishes do not.

- Resource nouns, HTTP verbs, status codes (the API chapter table), idempotency of `GET/PUT/DELETE`, *not* of `POST` (hence idempotency keys), content negotiation, pagination, filtering, versioning, HATEOAS (rarely worth it), conditional requests (`ETag`/`If-Match` for lost-update protection - an alternative to a `version` field in the body).
- **Validation:** Bean Validation on DTOs (`@Valid`), custom constraints, group-level validation (`@AssertTrue` for cross-field rules), validate at the boundary and again in the domain.
- **Error handling:** one `@RestControllerAdvice`, stable machine-readable `code`, safe `message`, correlation id, logging once. Map technical exceptions (optimistic lock -> 409, lock timeout -> 503 with `Retry-After`, constraint violation -> 409); never leak internals. RFC 7807 `application/problem+json` is the standard envelope many teams adopt - this project uses a simpler custom envelope with the same ideas.
-

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Jackson: strict (`FAIL_ON_UNKNOWN_PROPERTIES`), `BigDecimal` as string (`@JsonFormat(shape=STRING)`), `Instant` ISO-8601, never serialise entities.

Security (Spring Security 6)

- Filter chain: `SecurityFilterChain` bean; `oauth2ResourceServer().jwt()`; `@EnableMethodSecurity` + `@PreAuthorize("hasRole(...)")`; custom `AuthenticationEntryPoint`/`AccessDeniedHandler` for JSON errors; `SessionCreationPolicy.STATELESS`.
- **JWT validation:** signature (JWKS), `exp/nbf`, `iss`, optionally `aud`; map claims to authorities with a converter; never trust claims the IdP did not sign.
- **CSRF:** needed for cookie-based sessions, unnecessary for header-bearer APIs - a conscious, documented decision. **CORS:** allow-list, not `*`, no credentials. **Headers:** CSP, HSTS, `X-Content-Type-Options`, frame options, referrer policy.
- **Object-level authorization** is a *domain* rule (`AccessPolicy`) - annotations alone cannot express "this customer's account". Passwords: BCrypt/Argon2 (the demo issuer uses BCrypt and a decoy hash for constant-ish login time).

Testing

JUnit 5 (`@Test`, `@DisplayName`, parameterised tests, `@Timeout`), AssertJ (fluent assertions), Mockito (mocks/spies/`InOrder`; use sparingly), Spring Boot Test (`@SpringBootTest`, `MockMvc`, slices), Testcontainers/embedded PostgreSQL for real database behaviour, `@DynamicPropertySource` for wiring ports, `spring-security-test` or real tokens. **Test pyramid:** many fast unit tests, fewer integration tests, few end-to-end tests. **Flaky tests** come from time, shared state, ordering and concurrency - fix the cause.

Performance

Measure first (profilers, JFR, async-profiler, metrics); fix algorithmic and I/O problems before micro-optimising; N+1 and unbounded results dominate real services; connection-pool and timeout design matter more than JVM flags; cache after the query is right; use backpressure and bounded queues; keep transactions short. **Latency percentiles** (p95/p99) tell the user's story, averages hide tails; beware *coordinated omission* when load testing (use arrival-rate executors).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

System design and distributed systems reference

HLD and LLD - how to structure an answer

High-level design: clarify requirements (functional, non-functional, scale, consistency needs) -> estimate load -> draw the context and component diagram -> choose storage and consistency model -> walk the critical flows -> address failure, security, observability -> list trade-offs and what you would do next. **Low-level design:** pick one component and go deep: classes and responsibilities, state machine, algorithm, data model and SQL, concurrency control, error handling, tests. The ledger/processor design (`hld-and-lld.md`) is a ready example. *Analogy:* HLD is the city map; LLD is the floor plan of one building.

Microservices and service boundaries

- **Benefits:** independent deployment and scaling, team autonomy, failure isolation, technology choice. **Costs:** distributed transactions, network failure modes, operational overhead (pipelines, tracing, versioning), data duplication, harder testing.
- **Boundaries follow business capabilities and data ownership** (bounded contexts), not technical layers. A good boundary rarely needs a distributed transaction; if two parts constantly change together or must be atomic, they belong in one service. **Rule used here:** accounts + ledger + transactions share an invariant, so they share a transaction (ADR-001).
- **Smells:** chatty services, shared database tables, synchronous call chains of depth > 2-3, "distributed monolith" (must deploy together), a service with no owner.
- **Evolution:** start modular, extract along seams when a measured need appears (independent scaling, different release cadence, team ownership); strangler-fig pattern for extraction.

Idempotency

Analogy: pressing an elevator button twice does not summon two elevators.

- Naturally idempotent: `GET`, `PUT` (full replace), `DELETE`. `POST` creating a resource is not - make it idempotent with a client-supplied **idempotency key** persisted atomically with the effect (ADR-004).
- Mechanics: unique constraint on `(principal, key)`; store a request fingerprint to detect misuse; return the original response on replay; define retention; scope per principal.
- **Consumers** of at-least-once messages must be idempotent too (status check + unique postings). Idempotent producers/consumers + at-least-once delivery = effectively-once *effects*.
- Pitfall: using a time window ("same amount within 5 s") as the only guard - false positives and negatives.

Consistency

- **ACID** within one database; **CAP** (during a partition choose consistency or availability); **BASE/ eventual consistency** across services. Choose per fact: balances need strong consistency; dashboards and search can lag.
- Patterns: **transactional outbox** (atomic state change + event), **saga** (sequence of local transactions with compensations - orchestration vs choreography), **CQRS/read models** (eventually consistent projections), **reconciliation** (detect and repair drift), **two-phase commit** (rarely the right answer: blocking, coordinator failure, unsupported by most external systems).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Read-your-writes:** after a `202` the client polls the resource; the dashboard reads the same database so there is no replica lag in this design; with replicas route post-write reads to the primary or use a version token.

Concurrency control (at system level)

Optimistic (version check, retry) vs pessimistic (lock) vs serialisable isolation vs single-writer partitioning vs atomic conditional updates; choose by contention level and cost of conflict. For a balance: lock the row, check, change (ADR-005). For idempotency: a unique constraint. For queue claiming: lease + optimistic version or `SKIP LOCKED`. *Distributed locks* (Redis/ZooKeeper) are a last resort: they cannot give the safety of a database constraint (clock/GC pauses, lock expiry) - use fencing tokens if you must.

Resilience patterns

Pattern	Purpose	Detail used here
Timeout	bound waiting	connect 500 ms, read 2 s, pool 3 s, lock 5 s
Retry with exponential backoff + jitter	ride out transient errors	only for idempotent `authorize`; full jitter (random 0..cap) avoids synchronised retry storms
Circuit breaker	stop calling a failing dependency	closed -> open (50 % of 10 calls) -> half-open; open = fast `NotSent` + refund
Bulkhead	isolate resources	async processor keeps external latency off request threads; per-pool limits
Rate limiting / load shedding	protect capacity	token bucket per subject
Fallback / graceful degradation	partial service	withdrawals fail safely while deposits and transfers continue
Idempotent retry + dead-letter queue	no duplicate effects, no lost poison messages	outbox `DEAD` state + alert
Health checks, graceful shutdown	safe deploys	readiness, preStop, drain

Never retry on "unknown" blindly; classify outcomes (authorized / declined / not sent / unknown) and reconcile.

API versioning and backward compatibility

URI/header/media-type versioning; additive changes only within a version; tolerant readers; consumer-driven contract tests; deprecation headers and sunset; expand/contract for the data behind it (ADR-007).

Scalability

Stateless instances behind a load balancer; scale reads with replicas/caches; scale writes by partitioning (shard by account/customer), batching, or single-writer-per-key; remove hot spots (a "house account" every transaction touches); asynchronous processing to smooth bursts; capacity-plan from measurements (arrival rate x service time = concurrency, Little's law). **In this project:** the database is the scaling limit; per-account serialisation is inherent; the dispatcher is horizontally safe; next steps are parallel workers, a broker relay, partitioned tables.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Failure handling checklist

For every interaction ask: what is the timeout? is it idempotent? what happens on *unknown*? who is alerted? what does the user see? can it be replayed? what state is left behind if the process dies right here? Chaos testing (the simulator's `FAIL_5XX`/slow modes) validates the answers.

Distributed tracing, correlation, observability in design

Correlation id at the edge, propagated in headers, logged everywhere, stored in audit rows; traces for latency attribution; RED/USE metrics; SLOs with error budgets; runbooks linked from alerts.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Databases reference

SQL essentials (with the project's tables)

```
-- joins and aggregation: balance per currency of non-closed accounts
SELECT currency, SUM(balance) FROM accounts WHERE status <> 'CLOSED' GROUP BY currency;
-- window function: running balance per account from the ledger
SELECT account_id, id, SUM(CASE WHEN entry_type='DEBIT' THEN -amount ELSE amount END) OVER (PARTITION
BY account_id ORDER BY id) AS running
FROM ledger_entries;
-- anti-join: customers without accounts
SELECT c.* FROM customers c LEFT JOIN accounts a ON a.customer_id = c.id WHERE a.id IS NULL;
-- upsert (PostgreSQL)
INSERT INTO idempotency_records (...) VALUES (...) ON CONFLICT (principal, idempotency_key) DO
NOTHING;
-- CTE + keyset pagination
WITH page AS (SELECT * FROM transactions WHERE created_at < :cursor ORDER BY created_at DESC, id DESC
LIMIT 20) SELECT * FROM page;
```

Know: `INNER/LEFT` joins, `GROUP BY/HAVING`, subqueries vs joins vs `EXISTS`, `NULL` semantics (`= NULL` is never true; `NOT IN` with NULLs), `DISTINCT`, window functions, CTEs, set operations, `CASE`.

PostgreSQL specifics

MVCC (each statement sees a snapshot; writers do not block readers; old versions cleaned by `VACUUM`), `TIMESTAMPTZ`, `NUMERIC`, UUID, partial/expression/GIN indexes, `EXPLAIN (ANALYZE, BUFFERS)`, `pg_stat_statements`, `pg_stat_activity`, `pg_blocking_pids`, `SELECT ... FOR UPDATE [NOWAIT|SKIP LOCKED]`, advisory locks, transactional DDL, `CREATE INDEX CONCURRENTLY`, partitioning, `LISTEN/NOTIFY`, `max_connections` and PgBouncer.

Oracle compatibility (conceptual)

Area	PostgreSQL	Oracle
empty string	`''` is a value	`'' IS NULL`
pagination	`LIMIT/OFFSET`	`OFFSET .. FETCH NEXT` (12c+), `ROWNUM` (legacy)
DDL	transactional	auto-commit
partial indexes	yes	function-based index trick
isolation	READ COMMITTED (default), REPEATABLE READ, SERIALIZABLE	READ COMMITTED (default), SERIALIZABLE, read-only; consistent reads via undo; `ORA-08177`
sequences/identity	identity/sequence	identity (12c+)/sequence
booleans	`BOOLEAN`	`NUMBER(1)` (23ai adds `BOOLEAN`)
tooling	`EXPLAIN`	`EXPLAIN PLAN`, AWR/ASH, SQL Monitor
Spring/Hibernate: dialect and driver change, Flyway vendor folder, review native SQL, retest locking.		

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

MongoDB use cases

Document model fits hierarchical, schema-flexible data (KYC documents, notifications, event payloads, product catalogues) and read-optimised projections; scale-out by sharding; TTL indexes; multi-document ACID transactions exist but are costlier. Not a fit for a double-entry ledger or strongly relational constraints. Pattern: keep the relational store as the system of record and feed Mongo projections from events.

Indexes and query plans

Analogy: an index is the index at the back of a book - you jump to the page instead of reading every page, but every new page must also be added to the index.

- B-tree: equality, range, sort; **composite index order** = equality columns first, then the range/sort column (`(source_account_id, created_at DESC)`); a leading column that is not filtered makes the index useless.
- **Selectivity:** index helps when it returns a small fraction; PostgreSQL may prefer a sequential scan on small tables (correctly). **Covering** indexes (`INCLUDE`) allow index-only scans. **Partial** indexes shrink to the interesting rows (`status IN (...)`).
- Reading `EXPLAIN`: node types (Seq Scan, Index Scan, Bitmap Heap Scan, Index Only Scan), `Rows Removed by Filter`, estimated vs actual rows (stale statistics -> `ANALYZE`), sort method (disk spill), nested loop vs hash join, loops.
- Anti-patterns: functions on indexed columns (`lower(email)` without an expression index), leading wildcard `LIKE '%x'`, implicit casts, `OR` across different columns, deep `OFFSET`, `SELECT *` on wide rows, over-indexing (write cost, bloat).
- **Project example:** the account-history query with `source = :id OR target = :id` - plan walks `ix_tx_created` for a busy account, switches to a bitmap OR of the two account indexes for a sparse account (captured plans appendix).

Isolation levels, locking, deadlocks

Level	Dirty read	Non-repeatable	Phantom	Lost update / write skew
READ UNCOMMITTED (PG treats as READ COMMITTED)	no (PG)	yes	yes	yes
READ COMMITTED	no	yes	yes	yes (without locks)
REPEATABLE READ (PG: snapshot)	no	no	no (PG)	write skew possible; update conflicts abort
SERIALIZABLE (PG: SSI)	no	no	no	prevented, at the cost of retries

- **Row locks:** `FOR UPDATE` (exclusive), `FOR SHARE`, `NOWAIT`, `SKIP LOCKED` (job queues); **table locks** from DDL; **gap/next-key locks** in MySQL REPEATABLE READ.
- **Deadlock:** cyclic waits; the DB aborts one victim (`40P01`). Prevent with consistent lock order, short transactions, bounded `lock_timeout`; handle by retrying the victim.
- **Optimistic vs pessimistic** - see ADR-005. **Long transactions** block vacuum and hold locks: keep them short; no I/O inside.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Connection pools

HikariCP: `maximumPoolSize` (bounded by DB capacity, not request threads), `minimumIdle`, `connectionTimeout` (fail fast), `maxLifetime` (below infrastructure idle timeouts), `leakDetectionThreshold`. Formula of thumb: `connections ~= (cores x 2) + effective spindles` on the *database* side; replicas x pool <= `max_connections`. Symptoms of exhaustion: pending threads, acquire time up, timeouts; causes: slow queries, transactions held open, leaks, external calls inside transactions, too many replicas. Mitigations: fix the cause, PgBouncer (transaction pooling), RDS Proxy.

Schema migrations

Flyway/Liquibase; versioned immutable migrations; checksum validation; expand/contract; backward-compatible changes so rolling deployments and rollbacks work; data backfills in batches; `CREATE INDEX CONCURRENTLY`; separate privileged migration role; test migrations against production-sized data; repeatable migrations for views; baseline for existing databases.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Angular reference

Components, templates and change detection

Analogy: a component is a Lego brick: it takes inputs (what you plug into it), emits outputs (what it tells you), and hides its own studs.

- **Standalone components** (no NgModules): `@Component({ imports: [...] })`; `input()`, `output()`, `model()`; `@if/@for/@switch/@defer` control flow; `track` in `@for` (identity for DOM reuse).
- **Change detection:** Default checks the whole tree on every event; **OnPush** checks a component only when an input reference changes, an event originates inside it, a signal it reads changes, or `markForCheck` is called. With signals and OnPush, updates are fine-grained. Zone.js patches async APIs to trigger detection; zoneless is the direction of travel.
- **Lifecycle:** constructor (inject), `ngOnInit`/`effect`, `ngOnChanges`, `ngOnDestroy`; prefer `takeUntilDestroyed()`/`DestroyRef` over manual unsubscribe.

Services and dependency injection

Hierarchical injectors (root, route, component); `providedIn: 'root'` tree-shakable singletons; `inject()` function; injection tokens for configuration; functional interceptors/guards use `inject` too. Testing: `TestBed.configureTestingModule({ providers })`, overriding providers, `HttpTestingController`.

RxJS

Analogy: an Observable is a conveyor belt of values over time; operators are stations that transform, filter or merge the belt.

- Cold (HTTP, creates work per subscription) vs hot (shared); `shareReplay`, `share`. **Flattening:** `switchMap` (cancel previous - search-as-you-type, route params), `mergeMap` (parallel), `concatMap` (ordered queue), `exhaustMap` (ignore while busy - submit buttons). Combination: `combineLatest`, `forkJoin` (all complete; **cancels siblings on first error** - seen in the dashboard tests), `merge`.
- Errors: `catchError`, `retry` (only idempotent), `finalize`. Avoid nested subscribes; avoid subscribing in templates when `async`/signals do it; `toSignal`/`toObservable` bridge worlds.
- HTTP-specific: cancellation by unsubscribe (cancels the XHR), timeouts (`timeout`), polling (`timer`/`interval` + `switchMap`).

Signals

`signal(value)` (writable), `computed(() => ...)` (lazy, memoised, tracks dependencies automatically), `effect(() => ...)` (side effects; use sparingly), `input()`/`model()`. Signals give synchronous, glitch-free derived state and precise change detection; keep RxJS for async streams and convert at the edge (`toSignal`). Mutation: `set`/`update` with new references (immutable updates). **Project use:** view state (`loading/error/rows`), `computed viewState` from three signals, role flags in `AuthService`, form value -> signal via `toSignal(valueChanges)`.

Reactive forms

Typed `FormGroup/FormControl`, `FormBuilder.nonNullable`, sync/async validators, cross-field validators on the group (`transactionShapeValidator`), `statusChanges/valueChanges`, `markAllAsTouched` on submit, disabled-while-pending buttons, error display after touch, accessible errors (`mat-error`, `role="alert"`), `patchValue` vs `setValue`. Server-side errors mapped back to fields (the error envelope's `violations[]`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Routing

Lazy `loadComponent`; functional `CanActivateFn` guards; route `data`/`title`/`resolve`; `withComponentInputBinding()` binds route/query params to component `input()`s (used for `id` and `?tab=`); `returnUrl` handling (validate as an in-app path); preloading strategies; nested routes with a shell.

HTTP interceptors

Functional `HttpInterceptorFn`; **order matters** (the outermost sees the final error); cross-cutting only (auth header, correlation id, error normalisation, retry for idempotent GETs, loading indicators); attach tokens only to your own API origin; clone requests (immutable); handle 401 (sign out/redirect) once.

Performance

OnPush + signals; `track`; lazy routes/`@defer`; bundle budgets; avoid heavy template functions (use `computed`/pure pipes); virtual scrolling for big lists; image optimisation; HTTP caching/`shareReplay`; avoid polling storms (visible-tab only); measure with Angular DevTools and Lighthouse.

Testing

Karma/Jasmine (or Jest/Vitest): `TestBed`, `ComponentFixture`, `fakeAsync/tick`, `HttpTestingController` (`expectOne`, `flush`, `match`), spies, `input` setting via `fixture.componentRef.setInput`; test behaviour users see; page-level journeys with an e2e tool (here puppeteer-core; Playwright/Cypress in teams). **Gotchas seen here:** `forkJoin` cancels siblings on the first error (tests must flush only the failing request); `TestBed` can be configured once per test; periodic timers must be torn down (`fixture.destroy()`).

Secure frontend practices

No `innerHTML`/`bypassSecurityTrust*`; Angular sanitises by default; strict CSP and Trusted Types where possible; token in memory (or HttpOnly cookie via BFF); same-origin API proxy; validate redirect targets; avoid secrets in the bundle; dependency hygiene (`npm audit`); don't trust client-side validation or role checks (server enforces); handle money as strings; avoid leaking data in URLs/logs.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

DevOps, cloud and security reference

Docker

Image = layered filesystem + metadata; container = isolated process (namespaces, cgroups). Good Dockerfiles: multi-stage (build vs runtime), pinned base tags (ideally digests), dependency layer before source (cache), **non-root** user, minimal runtime (JRE, not JDK), `HEALTHCHECK`, no secrets in layers, `.dockerignore`, `STOPSIGNAL`/graceful shutdown, JVM container awareness (`MaxRAMPercentage`). Compose: local multi-service stacks, networks, health-gated `depends_on`, `read_only`/`cap_drop` hardening, secrets via env/files, named volumes.

Kubernetes

Pod (one or more containers), Deployment (ReplicaSet, rolling update), Service (stable virtual IP), Ingress (HTTP routing/TLS), ConfigMap/Secret, HPA (autoscaling on metrics), PDB (voluntary-disruption budget), NetworkPolicy (default deny), requests/limits (scheduling vs ceiling), probes (startup/liveness/readiness), `preStop` + `terminationGracePeriodSeconds`, topology spread, namespaces + pod security standards, RBAC, service accounts.

- **Rolling update** maths: `maxUnavailable: 0, maxSurge: 1` keeps capacity; readiness gates traffic; failed rollout -> `rollout undo`.
- **Probes:** liveness restarts a stuck process (do not tie it to dependencies); readiness removes a pod from load balancing (can include dependencies); startup protects slow starters.
- **JVM resources:** memory limit = non-negotiable (OOMKill); CPU **limits** cause throttling latency (prefer requests only); set `-XX:MaxRAMPercentage`.
- **Secrets:** base64 is not encryption - use External Secrets/CSI + KMS-encrypted etcd; RBAC to limit reads.

CI/CD

Pipeline stages and gates (see the CI/CD chapter); **trunk-based development**, short-lived branches, PR checks; *build once, promote the same artefact*; environment config outside the artefact; versioning (semver + build metadata); deployment strategies: rolling, blue/green, canary, feature flags; **database migrations** in the pipeline with expand/contract; smoke tests; automated rollback; pipeline security (least-privilege tokens, pinned actions, OIDC federation, protected environments, signed artefacts/SBOM, provenance - SLSA). **Security gates:** SAST (CodeQL/SpotBugs), SCA (Dependency-Check, `npm audit`, Dependabot), secrets scanning, container scanning (Trivy), IaC scanning, DAST against a deployed test environment.

Cloud architecture (AWS flavour)

Region/AZ design (>= 2-3 AZs), VPC with public/private/data subnets, ALB + WAF + CloudFront, EKS or ECS, RDS Multi-AZ with PITR and KMS encryption, Secrets Manager/Parameter Store, IAM least privilege + IRSA, ECR, CloudWatch/X-Ray (or Prometheus/Grafana/Tempo), autoscaling, backups and DR (RPO/RTO, pilot light/warm standby/active-active), cost awareness (right-sizing, Savings Plans, S3 lifecycle), infrastructure as code (Terraform/CDK). **Rollback:** application = redeploy previous image; data = forward-fix migrations, PITR as last resort.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

OAuth 2.0 and OpenID Connect

Analogy: OAuth is a valet key: it lets an app act for you with limited rights without handing over your master key; OIDC adds an ID card (who you are).

- Roles: resource owner, client, authorization server, resource server. **Flows:** authorization code **with PKCE** (browser/mobile - the default today), client credentials (service-to-service), device code; implicit and password grants are obsolete.
- **Tokens:** access token (short-lived, for the API), refresh token (rotate, store safely), ID token (OIDC, proves authentication to the client). Scopes/claims/roles; audience restriction; `iss` and `aud` checks; token revocation/introspection (opaque tokens).
- **Browser apps:** authorization code + PKCE; a backend-for-frontend with HttpOnly, SameSite cookies is the safest pattern; never put tokens in `localStorage` if you can avoid it.

JWT validation

Structure: header.payload.signature (base64url). Validate: **signature** with the issuer's key (`kid` -> JWKS), **algorithm allow-list** (reject `none`, never let the token choose HS vs RS), `exp`, `nbf`, `iss`, `aud`, clock skew; then map claims to authorities. JWTs are *signed, not encrypted* (do not put secrets in them); revocation is hard (short TTL + refresh rotation); keys rotate (cache JWKS, refetch on unknown `kid`). Pitfalls: accepting tokens from any issuer, trusting `roles` from an unsigned source, long-lived tokens, storing in web storage, logging tokens.

OWASP

OWASP Top 10 (2021) and API Security Top 10 (2023) mapped to this project in the security docs: broken (object-level) access control, cryptographic failures, injection, insecure design, misconfiguration, vulnerable components, authentication failures, integrity failures, logging/monitoring failures, SSRF; API-specific: BOLA, broken auth, object-property exposure, unrestricted resource consumption, function-level authorization, sensitive business flows, unsafe API consumption. **Secure SDLC:** threat modelling (STRIDE), secure defaults, code review with a security checklist, SAST/SCA/DAST, dependency management, secrets management, least privilege, defence in depth, incident response.

Secrets management

Never in code/config/images; secret manager (AWS Secrets Manager, Vault) -> injected at runtime (env/files/CSI); rotation without downtime; separate per environment; least-privilege access; audit access; scan for leaks; treat a leak as "rotate first". See the secrets chapter.

Monitoring, logging, tracing

Metrics (Prometheus; RED for services, USE for resources), logs (structured JSON, correlation ids, no PII), traces (OpenTelemetry, sampling), alerts on symptoms with runbooks, SLOs and error budgets, dashboards per audience. *Cardinality* warning: never put unbounded values (user ids, raw URIs) in metric labels.

Performance troubleshooting

The investigation flow in the operations chapter: symptom -> saturated resource -> responsible code/query -> fix -> re-measure. Tools: Prometheus/Grafana, traces, `jcmd`/JFR/async-profiler, `pg_stat_statements`, `EXPLAIN ANALYZE`, `pg_stat_activity`, thread and heap dumps, load tests (k6) with realistic arrival rates.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

AI-assisted development reference

GitHub Copilot workflows

- **Inline completion:** best for boilerplate and well-known patterns; poor for security- and concurrency-sensitive logic. Accept suggestions only after reading them like a pull request from a stranger.
- **Chat/Agent mode:** ask for *explanations*, *test ideas*, *refactor proposals* and *drafts*; keep tasks small; give it the spec and the relevant files; insist on tests first; run the tests yourself.
- **Repository instructions:** `.github/copilot-instructions.md` (always-on), path-scoped `.github/instructions/*.instructions.md` (`applyTo`), reusable `.github/prompts/*.prompt.md`. Short, high-signal, version-controlled, reviewed like code.
- **Pull-request assistance:** summaries and review suggestions are *inputs*, not approvals.

Specification-driven development

Write the spec first (requirements, acceptance criteria, design, API contract, test plan, security/performance notes), let the human approve the design (gate), then implement in small slices with tests; the spec outlives the chat session and is the contract reviewers compare the diff against. (See `specs/001-account-transfer.md` and `ai-engineering/workflow-example.md`.) Benefits: fewer misunderstandings, reviewable intent, better tests (derived from criteria rather than code), reproducibility across tools.

Context engineering

Analogy: give the assistant a small, accurate briefing pack rather than the whole library.

- Tiering: always-on rules (10 lines that cost money if broken) -> path-scoped rules -> on-demand reference files -> source of truth (code, ADRs, tests). Each fact lives in one place; link, do not copy.
- Make instructions **testable** ("money is BigDecimal compared with compareTo" is checkable; "write clean code" is not). Prune when a rule has not prevented a defect in months.
- Give examples of good and bad code from the repository (`review-examples.md`). Prefer pointing at real files over describing them.

Reusable instructions and handoff

Templates for prompts and handoffs reduce variance: the handoff note records what changed, evidence (real output), invariants checked, risks and what not to touch. Optional multi-agent roles (implementation, testing, security review, documentation) are *definitions*; humans own the two gates (design approval, final review).

Reviewing AI-generated code

Five recurrent failure patterns: **races/atomicity**, **missing authorization**, **incorrect money maths**, **unsafe retries/external calls**, **inadequate tests** - each with a bad-to-good example in `ai-engineering/review-examples.md`. Review method: ask "what if it runs twice, concurrently, or dies halfway?", "whose data is this?", "what are the units and rounding?", "what does a timeout mean?", "would this test fail if the code were wrong?" - and run the tests. Other risks: hallucinated APIs/dependencies (check they exist), licence-tainted snippets, subtly outdated idioms, confident but wrong explanations, secrets pasted into prompts, over-engineering.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Test generation

Generate from acceptance criteria and boundaries, not from the implementation; ask for negative, authorization and concurrency cases; use mutation thinking (delete a line, the test must fail); reject tests that only restate the implementation, over-mock, or sleep. Keep human ownership of what "correct" means.

Refactoring with assistants

Refactor only under green tests; small steps; ask for a plan first, then apply one step at a time; verify behaviour with characterisation tests for legacy code; use the IDE's automated refactorings where possible (safer than generated rewrites); keep diffs reviewable.

Safe dependency upgrades

One dependency (or one BOM) at a time; read release notes/migration guides; let the Boot BOM manage versions; run the full test suite including integration and PostgreSQL tests; compare Dependency-Check results before/after; watch for transitive changes (`mvn dependency:tree`); canary in the test environment; automate with Dependabot/Renovate PRs that go through the same gates. An assistant can summarise changelogs and propose code changes, but the green pipeline - not the assistant - decides.

Guardrails (organisation level)

No secrets or customer data in prompts; approved tools and tenants only; disclose AI involvement in PRs; the author remains accountable; security-sensitive areas require a second human reviewer; track defects traced to AI output and feed them back into instructions and checklists; train the team on failure patterns.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Technical leadership and agile delivery

Artefacts that show ownership from requirements to production readiness: product requirements, HLD/LLD, sprint plan and risk register, review checklists, code-review scenarios, troubleshooting exercises, mentoring.

Product requirements (example PRD) and feature specifications

PRD-1: Reliable money movement for internal applications

Problem	Three internal applications each implement their own balance checks and retry logic; duplicate and overdrawn transactions reach the operations team weekly and reconciliation is manual.
Goal	One platform API that makes money movement *correct by construction*: no overdraft, no duplicate application, no half-applied transfer, full audit.
Users	Customers (self-service), operations analysts (exceptions), support agents (look-up), administrators.
Success metrics	duplicate-transaction incidents -> 0; ledger-integrity violations = 0; p95 submission < 500 ms; 99.9 % availability; unknown-outcome transactions resolved < 15 min median; operations time per exception down 50 %.
Non-goals	real brokerage connectivity, order execution, fees/interest, cross-customer transfers, multi-currency conversion.
Constraints	Java/Spring Boot + Angular; PostgreSQL; synthetic data in non-production; audit retention per policy; SOC-2-style change control.
Release plan	Sprint 1-2 foundation, Sprint 3 operations tooling, Sprint 4 hardening and launch readiness (see `sprint-plan.md`).

Feature specifications (one-pagers, each with acceptance criteria)

FS-1 Idempotent submission

As a client developer, I want a retried request to return the original result so that a network glitch never moves money twice. **Acceptance:** (1) `Idempotency-Key` required (400 otherwise); (2) same key + same payload returns the original response with `Idempotent-Replayed: true`; (3) same key + different payload -> 422; (4) concurrent identical requests produce exactly one transaction; (5) keys are scoped per user; (6) records expire after 7 days.

FS-2 Maker-checker for large transactions

As a risk officer, I want amounts >= 50,000 to need a second person so that one insider cannot move large sums. **Acceptance:** (1) such a submission is `PENDING_REVIEW`; (2) the submitter cannot approve/reject; (3) another operator/admin can; (4) a customer cannot; (5) approval/rejection is audited with the reviewer; (6) an approved transaction is then processed normally.

FS-3 Uncertain-outcome handling

As an operations analyst, I want withdrawals with an unknown network outcome to be held and resolved so that customers are neither double-paid nor charged for nothing. **Acceptance:** (1) timeout -> `RECONCILIATION_REQUIRED`, funds stay reserved; (2) the reconciler resolves it from the network's answer automatically; (3) I can see attempts, audit trail and the reconciliation record; (4) I can resolve manually, but not for my own submission; (5) an alert fires if it stays open 15 minutes;

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

(6) the ledger is intact afterwards.

FS-4 Daily withdrawal limit (built test-first)

See `docs/testing/tdd-example.md` for the acceptance criteria AC1-AC8.

FS-5 Account transfer

Specified end to end in `specs/001-account-transfer.md` (specification-driven development example).

High-level and low-level design

High-level design (HLD)

The system context, component architecture, data model, lifecycles and deployment are the chapters *System context and component architecture*, *Customer, account and transaction lifecycles*, *Database design*, and *CI/CD and cloud deployment* of the architecture PDF. One-page summary:

- **Style:** modular monolith (ADR-001) + one simulator service; REST + JSON; PostgreSQL; asynchronous processing through a transactional outbox (ADR-006).
- **Key flows:** synchronous *validate and accept* (202) -> asynchronous *process* (ledger, optional external call) -> *finish* (completed / failed / reconciliation).
- **Invariants and who enforces them:** balance never negative (row lock + `CHECK`), balance == ledger (only `LedgerService` mutates, integrity check), once-only application (unique idempotency key, unique ledger postings), authorization (role + object-level), auditability (same-transaction audit rows, append-only table).
- **Quality attributes:** consistency over availability for money state; availability through stateless replicas, graceful shutdown, probes; observability by design; security by layered controls.
- **Main risks:** hot-account contention, external-network uncertainty, polling throughput of the in-database queue, single database as the scaling limit (see the risk register).

Low-level design (LLD): submit-and-process a withdrawal

Classes and responsibilities

Class	Responsibility	Key collaborators
`TransactionController.submit`	HTTP <-> DTO, status mapping (202 / 422), replay header	`TransactionService`
`TransactionService.submit`	key validation, fingerprint, idempotent create in `TransactionTemplate`, race-loser replay	`AccessPolicy`, `TransactionValidator`, `OutboxService`, `AuditService`, repositories
`TransactionValidator.validate`	pure rule evaluation -> `ACCEPT/REVIEW/REJECT` + violations	`DailyLimitPolicy`, `MoneyPolicy`
`OutboxDispatcher.drain`	lease events, call handler, retry/backoff/dead-letter	`TransactionProcessor`
`TransactionProcessor.process`	Tx1 begin, external call, Tx2 finish; business vs technical failures	`LedgerService`, `ClearingClient`, `ReconciliationService`
`LedgerService`	row locks, authoritative checks, debit/credit/reversal, ledger entries	`AccountRepository.lockById`, `LedgerRepository`
`RestClearingClient`	HTTP call with timeouts, retry+jitter, circuit breaker, outcome mapping	`RetryExecutor`, Resilience4j `CircuitBreaker`
`ReconciliationService`	open/auto-resolve/manual-resolve, compensation	`ClearingClient`, `LedgerService`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Algorithm (processor, withdrawal)

```
process(id):
  try   started = tx { tx = load(id); if status != VALIDATED -> return NOTHING   # idempotent
                       tx.PROCESSING; ledger.reserveWithdrawal(tx)               # lock, re-check,
debit, ledger entry
                       attempt(RESERVED); return (needsClearing, reference, amount, currency) }
  catch ApiException e  ->  tx { reload; PROCESSING -> FAILED(e.code) } ; return      # business
failure: Tx1 rolled back
  # technical exceptions (lock timeout, DB down, optimistic conflict) propagate -> outbox retry
  result = clearing.authorize(reference, amount, currency)                          # NO transaction
open here
  tx { tx = load(id); if status != PROCESSING -> return
       Authorized -> COMPLETED
       Declined | NotSent -> ledger.reverseWithdrawal(tx); FAILED
       Unknown -> RECONCILIATION_REQUIRED; reconciliation.open(tx) }
```

SQL (essentials)

```
SELECT * FROM accounts WHERE id = :id FOR UPDATE;                                    -- lock (5 s
bounded wait)
UPDATE accounts SET balance = :new, updated_at = now(), version = version + 1 WHERE id = :id AND
version = :v;
INSERT INTO ledger_entries (transaction_id, account_id, entry_type, amount, balance_after, created_at)
VALUES (...);  -- UNIQUE per (tx, account, type)
SELECT COALESCE(SUM(amount),0) FROM transactions WHERE source_account_id=:id AND
transaction_type='WITHDRAWAL'
  AND created_at >= :since AND status IN ('PROCESSING','COMPLETED','RECONCILIATION_REQUIRED') AND id
<> :self;   -- daily limit, under the lock
```

Error handling matrix

Failure	Where detected	Result
validation rule	`TransactionValidator`	`REJECTED`, 422, audited
insufficient funds at lock time	`LedgerService` (`ApiException`)	Tx1 rolls back, then `FAILED INSUFFICIENT_FUNDS`
lock wait > 5 s	JPA lock timeout	exception propagates, outbox retries, API-side 503 `LOCK_TIMEOUT`
optimistic conflict (cancel vs process)	`@Version`	loser retries/skips, API 409
clearing decline / not sent / unknown	`ClearingResult`	`FAILED` + refund / `FAILED` + refund / `RECONCILIATION_REQUIRED`
crash between Tx1 and Tx2	recovery job	`RECONCILIATION_REQUIRED` after 5 min
poison event	dispatcher	`DEAD` after 5 attempts + alert

Sequence and boundaries

See the *Account transfer sequence* and *Database transaction boundaries* diagrams.

Sprint task breakdown, dependencies and risk register

Sprint plan (2-week sprints, one team of five; estimates in ideal days)

Sprint	Goal	Stories (estimate)	Definition of done evidence

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Sprint	Goal	Stories (estimate)	Definition of done evidence
1 Foundation	walking skeleton with correct money handling	project skeleton + CI (2); domain model, Flyway V1, constraints (3); `MoneyPolicy` + tests (1); customer/ account CRUD + lifecycle (4); security chain + JWT + roles (3); error envelope, correlation id (2)	tests green; contract generated; threat model v1
2 Transactions	submit -> validate -> process end to end	state machine (2); validator + `AccessPolicy` (3); idempotent submit (3); outbox + dispatcher (3); ledger + ordered locks (3); processor (3); concurrency tests (3); BDD scenarios (2)	invariants asserted under concurrency; idempotency race test
3 Integration and operations	external network, reconciliation, dashboards	clearing client with retry/jitter/breaker (3); simulator + chaos (2); reconciliation (3); maker-checker (2); Angular shell, guards, interceptors (3); dashboard, investigations, audit (4); metrics + Grafana + alerts (3)	chaos e2e; runbooks for every alert
4 Hardening and launch	production readiness	rate/size limits (2); PostgreSQL verification + query plans (3); Dockerfiles, compose, k8s, probes (3); security scans in CI (2); performance baseline + k6 (3); docs, ADRs, runbooks (3); release checklist and rollback drill (2)	readiness checklist signed; rollback rehearsed

Buffer 15 % per sprint. Demo each sprint with the live end-to-end scripts; retrospective actions go to the next sprint backlog.

Dependencies

Dependency	Needed by	Risk if late	Mitigation
OIDC identity provider + client registration	Sprint 3	no real token validation	local issuer (dev only) behind a profile; contract = issuer URL
Clearing network sandbox + idempotency/lookup contract	Sprint 3	integration design unvalidated	simulator with deterministic and chaos behaviours; contract assumptions documented (ADR-011)
PostgreSQL instance, backup policy	Sprint 4	no real-DB verification	embedded PostgreSQL in tests; Testcontainers in CI
Security review slot	Sprint 4	late findings	threat model reviewed from Sprint 1; checklist per PR
Kubernetes namespace + secrets store	Sprint 4	cannot deploy	compose + manifests validated statically; overlay for test

Risk register

#	Risk	Likelihood	Impact	Score	Mitigation	Owner	Evidence/ status
R1	Overspend/duplicate under concurrency	M	critical	high	row locks, idempotency, constraints, concurrency tests on real PostgreSQL	tech lead	tests green

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	Risk	Likelihood	Impact	Score	Mitigation	Owner	Evidence/ status
R2	Unknown external outcome mishandled (double payout)	M	critical	high	reserve-first, 3-way classification, reconciliation, alerts	tech lead	ADR-011, chaos e2e
R3	Broken object-level authorization	M	critical	high	`AccessPolicy` + BOLA tests, review checklist	security champion	`SecurityIT`
R4	Hot-account lock contention limits throughput	M	medium	medium	bounded waits, metrics, design note for batching	tech lead	baseline shows serial behaviour
R5	In-database queue polling limits throughput	M	medium	medium	work-conserving loop done; parallel workers / broker relay next	dev	measured 23 -> 34 tx/s locally
R6	H2/PostgreSQL behavioural drift	M	medium	medium	PostgreSQL ITs (embedded + Testcontainers)	QA lead	`Postgres*IT`
R7	Dependency vulnerability	H	high	high	scans in CI, pinned versions, Dependabot	devops	CI jobs (not yet run on a runner)
R8	Token theft via XSS	L	high	medium	memory-only token, CSP, no innerHTML	frontend lead	ADR-012
R9	AI-generated code introduces races/ auth gaps	M	high	high	AI review checklists, tests-first, human review mandatory	tech lead	`ai-engineering/`
R10	Schema change breaks rollback	L	high	medium	expand/contract, previous-version compatibility check	dba	release checklist
R11	Skills concentration (one person knows the ledger)	M	medium	medium	pairing, ADRs, runbooks, mentoring plan	EM	docs
R12	Unvalidated assumptions about the network contract	M	high	high	explicit assumptions, contract tests when the sandbox exists	architect	ADR-011

Pull-request checklist and code-review checklist

Pull-request checklist (author)

- **Why** is clear (ticket/spec link); the PR is small enough to review in 30 minutes, or split.
- Tests written first or alongside: unit for new rules, integration for new endpoints, regression test for every fixed defect.
- **Authorization:** every new endpoint has a role rule *and* an object-level check; a BOLA test exists.
- **Money:** only `BigDecimal`/strings, `compareTo`, no rounding of input, no `double`/`float` anywhere near amounts.
- **Transactions:** boundaries explicit; no external call inside a transaction; locks in the documented order; no `@Transactional` self-invocation.
- **Idempotency/retries:** any new retry is on an idempotent operation with backoff + jitter; new endpoints that create state accept an idempotency key.
- **Errors:** uses the error envelope; no internals leaked; new codes documented.
- **Data:** migrations additive (expand/contract), never edit a released migration, indexes for new query shapes, plan checked.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Observability:** logs without PII; metrics/alerts/runbook updated if behaviour is operationally relevant.
- **Docs:** OpenAPI regenerated (`-Dopenapi.update=true`) and committed; ADR for a significant decision.
- `mvn verify`, frontend lint/test/build pass locally; no secrets in the diff; dependencies justified.
- If AI assisted: I read and understand every line; the AI checklist in `ai-engineering/review/` was applied.

Code-review checklist (reviewer)

Correctness & invariants - Can an interleaving of two requests break an invariant? What if this runs twice? What if the process dies between these two lines? Are state transitions legal per the state machine? **Security** - who can call this, and on *whose* data? Is input validated by allow-list? Any string-built SQL/HTML? Any new secret, log line, or error that exposes data? Any new dependency (licence, maintenance, CVEs)? **Concurrency** - lock scope and order; `@Version` use; unique constraints as the arbiter; bounded waits; thread-safety of shared state (rate limiter maps, caches). **Resilience** - timeouts on every remote call; retry only idempotent calls; classification of "unknown"; circuit breaker; graceful degradation. **Performance** - N+1 queries, unbounded result sets, missing index, sort without whitelist, deep offset paging, work inside locks, allocation in hot paths. **Design** - single responsibility, naming, dependency direction, duplication, testability (pure logic separated from I/O), no premature abstraction. **Tests** - do they fail if the code is wrong (mutation thinking)? Do they test behaviour, not implementation? Deterministic (no sleeps without need, no clock/time-zone dependence)? Cover negative paths and the invariant afterwards? **Frontend** - typed models, no `any`, OnPush, unsubscribe/`takeUntilDestroyed`, accessible labels, loading/empty/error states, no money arithmetic in JS, no token in storage. **Operations** - config via environment, safe defaults, health/readiness unaffected, rollback possible, migration safe under rolling deploy. **Tone** - comments are specific, kind and actionable; label severity (`blocker`, `suggestion`, `nit`, `question`); approve when risks are acceptable, not when code matches your taste.

Code-review scenarios

Seven realistic pull requests. Each has the **problem**, the **investigation** (how a reviewer finds it), the **solution**, the **trade-offs** and the **regression tests** (existing in this repository where noted).

1. "Check the balance, then update it" (race condition)

Problem. A PR adds withdrawals like this:

```
Account a = accounts.findById(id).orElseThrow();
if (a.getBalance().compareTo(amount) < 0) throw insufficientFunds();
a.setBalance(a.getBalance().subtract(amount));      // read-modify-write, no lock
accounts.save(a);
```

Investigation. Ask: *what if two requests run this at the same moment?* Both read 100, both pass the check for 80, both write 20: 160 left the account. It passes every single-threaded unit test. Look for the missing lock/version, run two threads (the repository's concurrency test), read the DB constraints (`balance >= 0` would turn this into an error, but only for the second writer *after* money logic already assumed success). **Solution.** Take the row lock before reading (`accounts.lockById` -> `SELECT ... FOR UPDATE`), check and change inside the same transaction, keep the invariant also in a `CHECK` constraint; this is `LedgerService.reserveWithdrawal`. **Trade-offs.** Pessimistic locking serialises per account (lock waits, deadlock risk -> ordered locks, bounded timeout) in exchange for simplicity and no retry storms; the alternative `UPDATE ... WHERE balance >= :x` is fine for a single row but cannot also enforce the ledger/limit/status checks in one critical section. **Regression tests.** `ConcurrencyScenarios.concurrentWithdrawalsNeverOverspend` (H2 *and* PostgreSQL), `SchemaConstraintsIT.balanceCannotGoNegative`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. Controller loads by id without an ownership check (BOLA)

Problem. `@GetMapping("/accounts/{id}") Account get(@PathVariable UUID id) { return repo.findById(id).orElseThrow(); }` - and it returns the entity. **Investigation.** For every `{id}` ask *whose is it?* There is no comparison with the caller's customer. Also: it serialises an entity (leaks fields; lazy-loading surprises) and returns `500` for unknown ids. Try it: authenticate as customer A, request customer B's id. **Solution.** Go through a service that loads the object and calls `AccessPolicy.canViewAccount(user, account.customerId)`; return `404` plus an independent `ACCESS_DENIED` audit row; map to a DTO (`AccountService.requireVisible`, `toResponse`). **Trade-offs.** `404` instead of `403` hides existence (good) but makes debugging slightly harder (hence the audit event). Centralising the check costs one extra lookup per request - negligible. **Regression tests.** `SecurityIT.customerCannotReadOthersAccounts`, `searchesAreScoped`, `cannotSubmitAgainstOthersAccounts`; `AccessPolicyTest`.

3. double for fees and totals

Problem. `double fee = amount.doubleValue() * 0.0125; BigDecimal.valueOf(fee)` and `if (total == 100.0)`. **Investigation.** Grep for `double`, `float`, `doubleValue`, `Math.round` near money. Demonstrate: `0.1 + 0.2 != 0.3`; `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is false. **Solution.** `MoneyPolicy.percentageOf(base, basisPoints, currency)` with `HALF_EVEN` and the currency's scale; `compareTo` for comparisons; allocation that preserves cents (`allocate`). Money in JSON as strings. **Trade-offs.** `BigDecimal` is slower and verbose (irrelevant at this volume); minor-unit `long` is an alternative with its own awkwardness for fractional rates. **Regression tests.** `MoneyPolicyTest` (float vs decimal, HALF_EVEN ties, allocation, equals vs compareTo), `DailyLimitPolicyTest.scaleDoesNotMatter`, `AccountDtos` string serialisation (`ApiContractIT.moneyIsAString`).

4. @Transactional around a REST call

Problem. `@Transactional void process(id) { debit(); clearing.authorize(); markCompleted(); }` **Investigation.** Draw the timeline: the row lock and a pooled connection are held for the *duration of an HTTP call*, retries and timeouts included. Under a slow dependency the pool drains (`hikaricp_connections_pending > 0`), every request thread waits, the whole API stalls although the database is idle. Also: if the call succeeds but the commit fails, money left without a record. **Solution.** Split the boundaries (ADR-003): commit "funds reserved", call with no transaction open, commit the outcome; classify the result (`Unknown` -> reconciliation). See `TransactionProcessor`. **Trade-offs.** More states and a reconciliation path; far shorter lock/connection hold times and honest handling of uncertainty. **Regression tests.** `TransactionProcessorTest` (order: reserve before authorize; Unknown does not refund), `TransactionFlowIT.unknownOutcome*`, chaos e2e (breaker opens, pool unaffected).

5. Retrying a non-idempotent call

Problem. `@Retryable` on a `POST /payouts` client method "to be resilient". **Investigation.** Ask what a *timeout* means: the request may have executed. A retry then pays twice. Check whether the remote operation is keyed by a stable id, whether retries have backoff **and jitter**, whether there is a cap and a circuit breaker. **Solution.** Only retry because `authorize` is idempotent on our `reference` (the contract); `RetryExecutor` with exponential backoff and full jitter, bounded attempts; classify post-retry failure as `Unknown`, never as "failed"; breaker to stop hammering. **Trade-offs.** Retries add latency and load during an outage (breaker mitigates); relying on the remote's idempotency is a contract assumption that must be verified with the provider (ADR-011). **Regression tests.** `RestClearingClientTest` (retries 503s, UNKNOWN after exhaustion, NotSent only for pre-delivery failures, breaker short-circuit), `RetryAndRateLimitTest` (jitter bounds, no retry of non-retryable errors).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Entity exposed over the API + mass assignment

Problem. `@PostMapping Customer create(@RequestBody Customer c)`. **Investigation.** The client can set `status`, `version`, `id`, `createdAt`; the response leaks internals (tax id, version); schema changes break clients; unknown fields are silently accepted. **Solution.** Request/response DTOs with Bean Validation, a service mapping layer, `FAIL_ON_UNKNOWN_PROPERTIES`, masked responses by role. **Trade-offs.** Mapping boilerplate (records keep it short); a stable contract and an enforced boundary in exchange. **Regression tests.** `SecurityIT.inputHardening` (mass-assignment attempt -> 400), `piiMasking`, `ApiContractIT.validationEnvelope`.

7. Swallowed exception and a "temporary" catch (Exception e) { log.warn(...) }

Problem. The processor catches everything, logs, and returns - the outbox event is then marked dispatched and the transaction sits in `VALIDATED` forever. **Investigation.** Follow what each exception *means*: business failures must produce a terminal state; technical failures must *propagate* so the retry mechanism runs; and a catch that logs-and-continues turns both into silent data loss. Check metrics: backlog constant, no failures counted. **Solution.** `TransactionProcessor` catches **only** `ApiException` (business) and records `FAILED`; everything else propagates to the dispatcher, which retries with backoff and dead-letters after N attempts (alert). **Trade-offs.** Noisier logs while an outage lasts; no lost work. **Regression tests.** `TransactionProcessorTest.technicalFailuresPropagate`, `.ledgerBusinessFailureFailsTransaction`, `OutboxDispatcherTest` (retry vs dead letter), `OutboxDeadLetterIT`.

Troubleshooting exercises

Six exercises for practising production investigation. Each states the **problem**, the **investigation** (what to look at, in order), the **solution**, the **trade-offs** and the **regression tests**.

Exercise 1 - p95 latency explodes at peak, database is idle

Problem. At 09:00 p95 goes from 80 ms to 12 s; CPU is low; the database dashboard looks calm; no errors for the first minutes, then timeouts. **Investigation.** (1) Latency by `uri`: all endpoints, so a shared resource. (2) `hikaricp_connections_pending` > 0 and `active == max`: **pool exhaustion**. (3) Thread dump: request threads `WAITING` in `HikariPool.getConnection`; the connection holders are in `RestClient`/socket read - a transaction wraps an HTTP call. (4) `pg_stat_activity`: many `idle in transaction`. (5) The clearing network is slow today. **Solution.** Remove the remote call from the transaction (reserve funds, commit, call, commit); keep a hard read timeout (2 s) and the circuit breaker; alert on pool pending and acquire-time p95. **Trade-offs.** Extra states (`RECONCILIATION_REQUIRED`) and a reconciler; bounded blast radius. Increasing the pool "to fix it" would only move the failure to PostgreSQL's `max_connections`. **Regression tests.** `TransactionProcessorTest.withdrawalAuthorized` (reserve precedes call), chaos e2e, alert `ConnectionPoolSaturated`.

Exercise 2 - intermittent duplicate deposits

Problem. Support sees pairs of identical deposits a few seconds apart, only for mobile users on poor networks. **Investigation.** Compare the pair: different transaction ids, different or missing idempotency keys? Audit trail `TRANSACTION_SUBMITTED` details and correlation ids: two requests. Client log: the first request timed out, the app retried **with a freshly generated key**. **Solution.** Client must generate the key once per *logical* action and reuse it on retry (`IdempotencyKeys`); the server already refuses to accept a missing key; add the key to the UI's retry path. Server side nothing was wrong - the replay protection worked for same-key retries. **Trade-offs.** A user pressing the button twice deliberately is *meant* to create two payments; idempotency distinguishes retries from intents only via the key. **Regression tests.** `IdempotencyIT` (replay, concurrent, key reuse), SPA specs "reuses the SAME key when retrying an unchanged payload" and "new key when the payload

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

is edited".

Exercise 3 - sporadic deadlock errors on transfers

Problem. `deadlock detected` from PostgreSQL a few times an hour; the affected transfers succeed on retry. **Investigation.** The PostgreSQL log prints both statements and lock targets: two `UPDATE accounts` on rows A and B in opposite order. Find the code path: a new batch job that applies transfers from a file, locking source first. Compare with `LedgerService.applyTransfer`, which sorts the ids. **Solution.** Route the batch through `LedgerService` (single place that orders locks); never add a second path that mutates balances; keep lock waits bounded and treat deadlock victims as retryable. **Trade-offs.** A total lock order is cheap and complete for a known set of rows; ad-hoc "retry on deadlock" alone hides the bug and costs throughput. **Regression tests.** `LedgerServiceTest.locksInAscendingIdOrder`, `ConcurrencyScenarios.oppositeTransfersDoNotDeadlock` on H2 and PostgreSQL.

Exercise 4 - heap grows until the pod is OOM-killed

Problem. Pods restart every ~6 hours with `OOMKilled`; heap-after-GC rises steadily. **Investigation.** Heap dump (`jcmd GC.heap_dump`) -> MAT dominator tree: a huge `ArrayList<FinancialTransaction>` retained by a request thread; the code is `repo.findAll()` in a new "export" endpoint. RSS vs heap vs limit tells whether it is heap or native. **Solution.** Page every list endpoint (cap 100), stream exports or use keyset batches, add a test that asserts the page cap, add a heap-high alert. **Trade-offs.** Exports become multi-request or asynchronous jobs. **Regression tests.** `ApiContractIT.pagination` (page-size cap), code-review checklist item "unbounded result sets".

Exercise 5 - reconciliation backlog after a network blip

Problem. After a 4-minute provider incident, 37 withdrawals are `RECONCILIATION_REQUIRED` and customers call support about reduced balances. **Investigation.** Confirm funds are *reserved, not lost* (`funds_reserved`, ledger entries `DEBIT`). Provider back? The auto-reconciler resolves records whose lookup returns a definite answer; records the provider never saw are refunded after the grace period (30 s). Anything left needs the provider's portal. **Solution.** Let the reconciler work; manually resolve the remainder (maker-checker); tell support the script ("funds are held, resolved within minutes"); afterwards check ledger integrity. **Trade-offs.** Conservative holding reduces customers' available balance temporarily, in exchange for never double-paying. **Regression tests.** `TransactionFlowIT.unknownOutcomeIsReconciledAutomatically`, `unknownOutcomeResolvedManually`, chaos e2e "every held transaction is resolved automatically".

Exercise 6 - every request returns 401 right after an identity-provider key rotation

Problem. After the IdP rotated its signing keys, valid tokens are rejected with `INVALID_TOKEN` for ~10 minutes, then it recovers by itself. **Investigation.** Decode the token header (`kid` unknown to the service); check JWKS reachability from the pod; the resource server caches the key set and re-fetches on unknown `kid` (rate-limited) - a network policy blocked egress to the IdP so the refresh failed until the cache expired. **Solution.** Allow egress to the IdP in the NetworkPolicy, alert on JWKS fetch failures, keep the IdP overlapping old and new keys during rotation. **Trade-offs.** Longer JWKS cache = fewer calls but slower rotation pick-up. **Regression tests.** `SecurityIT` token validation tests (tampered/forged/expired -> 401 with the standard envelope); `validate-infra.py` keeps the NetworkPolicy reviewed.

Mentoring junior developers

Principles: teach with the codebase (every lesson points at a real class and a test), prefer questions to answers ("what happens if this runs twice?"), review the *thinking* before the code, give small stretch tasks with a safety net (tests, pairing), and make safe failure normal (blameless reviews).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Worked examples

SOLID - "this service does too much"

Observation: a junior's `TransactionService` validates, writes the audit row, calls the clearing API and sends e-mails. *Coaching:* ask them to list the reasons the class would change. Together split into `TransactionValidator` (decides), `LedgerService` (changes balances), `TransactionProcessor` (orchestrates), `ClearingClient` (adapter). *Show:* the validator is pure, so ten rules became ten one-line tests. *Homework:* add a new rule (`DailyLimitPolicy`) without touching any class except the validator - open/closed in practice.

Design patterns - when not to use one

Observation: a factory-of-strategies-of-visitors for three transaction types. *Coaching:* a `switch` over a sealed/enum type with exhaustive checking is simpler and the compiler tells you when a new type is unhandled. Introduce a pattern when it removes duplication you can *point at* (here: the `ClearingClient` port exists because the real adapter, the in-process one and the test double all differ).

Testing - "it passes, but would it fail if the code were wrong?"

Exercise: delete the `lockById` call from `LedgerService` and run the suite: the single-thread tests stay green, `concurrentWithdrawalsNeverOverspend` goes red. Lesson: tests must be able to fail for the property you care about; add a concurrency test whenever the property is about interleavings. Then write a red/green TDD cycle together (`docs/testing/tdd-example.md`).

Secure coding - think like the caller

Exercise: "log in as Alice and try to read Bob's account using only curl". They find nothing blocks it in their draft controller; fix with `AccessPolicy`, then write the BOLA test *first* for the next endpoint. Discuss why the answer is `404`, why masking matters for support, why `FAIL_ON_UNKNOWN_PROPERTIES` is on.

Performance - measure first

Exercise: the history endpoint is "slow" - before changing anything, capture `EXPLAIN (ANALYZE)` (the repository's captured plans), read `Rows Removed by Filter`, hypothesise (OR predicate), write the expected result, change one thing, re-measure. Rule: no optimisation without a before/after number (see how the dispatcher change was measured).

Production troubleshooting - follow one correlation id

Exercise: trigger a failing transaction via the simulator (`.66`), then investigate as on-call: dashboard -> `/ops/health` -> transaction attempts -> audit trail -> reconciliation record -> runbook. Rotate the on-call shadow role weekly.

The scenario: "the unit tests pass, but it fails under concurrent load"

A junior implements *withdraw* as read, check, write (scenario 1 in the code-review scenarios). My approach:

1. **Don't lead with the answer.** Ask them to run two requests at once (the k6/Node script) and look at the final balance; let them see -40.
2. **Build the mental model:** draw the two timelines on a whiteboard, name the anomaly (lost update / write skew), connect it to isolation levels.
3. **Present the option space, not just the fix:** row lock, atomic conditional `UPDATE`, optimistic version with retry, serialisable isolation, single writer; discuss trade-offs.
4. **Make the failure reproducible as a test first** (the concurrency scenario), watch it fail, apply the fix, watch it pass - on the real database, not only H2.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5. **Generalise:** add "what if this runs twice / concurrently / is retried?" to their personal review checklist and to the PR template.
6. **Follow up:** have them review the next person's concurrency-sensitive PR with the checklist (teaching reinforces learning), and pair on the next lock-related task.

Feedback style

Specific, kind, actionable, tied to impact; one or two themes per review, not twenty nits; praise good decisions explicitly (it is how norms spread); label comments (`blocker`/`suggestion`/`nit`/`question`).

Release planning, release-readiness checklist and rollback plan

Release-readiness checklist

Code and tests

- CI is green on the release commit: unit, integration (incl. real PostgreSQL), frontend lint/tests/build, SpotBugs, CodeQL.
- `specs/openapi.json` regenerated and reviewed; any breaking change has a new version, not an edit of v1.
- No open critical/high vulnerabilities (Dependency-Check, npm audit, Trivy) without a documented, time-boxed exception.
- No secrets in the diff (gitleaks green).

Data

- Migrations are forward-only, additive (expand phase), reviewed by a second engineer, and tested against a copy of production-size data for lock duration (`CREATE INDEX CONCURRENTLY`, no table rewrites).
- The *previous* application version runs against the *new* schema (verified in the test environment).
- Backup/PITR verified recent; restore time known.

Operations

- Dashboards and alerts cover the new behaviour; new alerts have runbook sections.
- Capacity impact estimated; HPA/limits reviewed.
- Feature flags / configuration switches for risky behaviour exist and are documented.
- On-call briefed; support has the customer-facing change notes.

Security

- Threat-model delta reviewed; new endpoints have role and object-level rules and tests (BOLA test added).
- Logging reviewed for PII.

Rollout

- Deployed to test; smoke tests passed; end-to-end (API + browser) suites passed against test.
- Rollout window avoids month-end/peak; owner and rollback decision-maker named.

Release plan (for a typical change)

1. Merge to `main` -> pipeline builds and scans images (`1.0.<run>`), deploys to **test**, runs smoke tests.
2. Manual approval on the `production` environment promotes the **same image** (no rebuild).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. Rolling update (`maxUnavailable 0`): new pods become ready (readiness), old pods drain (`preStop` + graceful shutdown).
4. Watch for 30 minutes: error rate, p95, outbox backlog, reconciliation count, ledger integrity, business KPIs.
5. Announce completion; keep the previous ReplicaSet for fast rollback; run the post-release checks.

Rollback plan

Situation	Action	Time	Data impact
Bad application release, schema compatible	`kubectl rollout undo deployment/platform-service` (or Argo CD revert)	~1-2 min	none; outbox and in-flight work continue
Bad configuration	revert the ConfigMap/overlay, restart	~2 min	none
Risky behaviour only	switch it off with the feature flag/config (e.g. `app.processing.dispatcher-enabled=false` to pause processing while keeping intake)	seconds	submissions queue in the outbox
Migration caused a problem	**do not roll the schema back by default**: ship a forward fix; the previous app version still works (expand/contract)	-	-
Data corruption (last resort)	point-in-time recovery to a new instance, replay/repair from the audit trail and ledger, reconcile before switching traffic	hours	requires an incident commander

Triggers (decide in advance): error rate > 5 % for 5 min, p95 > 3x baseline for 10 min, any ledger-integrity violation, any dead-lettered event caused by the release. Money safety is why rollback is rarely scary here: a transaction is either fully applied or not applied; unknown outcomes are held, not lost.

Post-incident review (blameless)

Field	
Incident id / title	
Date, start, detection, mitigation, resolution (UTC)	
Severity	SEV1 / SEV2 / SEV3
Incident commander / scribe / responders	
Customer impact (who, how many, what, how long)	
Money impact (any ledger-integrity violation? any manual correction? amounts, accounts)	

Summary

Two or three sentences a non-engineer can read.

Timeline

Time	Event	Source (alert, dashboard, log, person)

Detection

How was it detected? Was it an alert or a customer? **Time to detect**: . Could we have detected it earlier (missing alert, noisy alert, wrong threshold)?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Root cause and contributing factors

Use "5 whys"; separate the *trigger* (what happened) from the *conditions* (why it was possible). No names, no blame - systems and decisions.

What went well / what went badly / where we got lucky

Resolution and recovery

What stopped the bleeding? What was restored when? Which data had to be repaired and how was it verified (ledger-integrity check, reconciliation, audit trail)?

Action items

#	Action	Type (prevent / detect / mitigate / process)	Owner	Due	Ticket
1					

Every action has an owner and a date; at least one is a **regression test or alert** so this exact failure cannot recur silently.

Lessons for the playbook

What should the runbook say next time? Which runbook sections changed?

Appendix

Graphs, queries, log excerpts (with correlation ids), related incidents.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Job-description traceability matrix

Maps each major requirement of a **Senior/Lead Java Full-Stack Engineer** role (as described for SS&C FinTech Services India, Hyderabad - full-stack Java with Spring Boot, Angular or React, REST, microservices, SQL/Oracle/MongoDB, automated testing, TDD/BDD, secure coding, CI/CD, cloud, AI-assisted engineering, technical ownership) to **concrete evidence in this repository**. Rule applied: *a requirement counts as demonstrated only when it is tied to code, tests, scripts, or clearly labelled conceptual material.* "Evidence" states whether it was **executed** (E), **statically validated** (S), or is **conceptual** (C). The frontend exists twice, in Angular (`frontend/`) and in React (`frontend-react/`), against the same API. No claim here is made about any vendor's proprietary systems.

1 - Senior full-stack Java development

Aspect	Evidence
Implemented feature / artefact	end-to-end platform: Spring Boot API + Angular UI + React UI + PostgreSQL
Source-code location	`backend/platform-service`, `frontend/`
Test / validation evidence	full builds and live runs (E): backend unit + integration, Angular 58 tests, React 90 tests, API e2e 50/50, browser e2e 19/19 for each UI
Interview talking point	"I owned requirements -> design -> code -> tests -> pipeline -> runbooks"
Remaining limitations	single-team scale; no real client integrations

2 - Spring Boot (3.x, Security, Data JPA, Actuator, validation)

Aspect	Evidence
Implemented feature / artefact	Boot 3.5, Security resource server, JPA/Hibernate + Flyway, Actuator + Micrometer, OpenAPI
Source-code location	`SecurityConfig`, `*Service`, `application*.yml`
Test / validation evidence	`SecurityIT`, `ApiContractIT`, `PostgresFeaturesIT` (E)
Interview talking point	explicit transaction boundaries, `MANDATORY` propagation, fail-closed config
Remaining limitations	Spring Cloud intentionally not used (ADR-001)

3 - Angular (TypeScript, Router, RxJS/Signals, Reactive Forms, HttpClient, component library)

Aspect	Evidence
Implemented feature / artefact	Angular 20 standalone + signals + Material, lazy routes, guards, interceptors, typed forms
Source-code location	`frontend/src/app/**`
Test / validation evidence	58 Karma tests, ESLint a11y rules, prod build, browser e2e (E)
Interview talking point	state views, safe retries with idempotency keys, token in memory, money as strings
Remaining limitations	no i18n, no offline support, UI e2e is a script not Playwright suite

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3b - React (TypeScript, hooks, Router, controlled forms, fetch, testing library)

Aspect	Evidence
Implemented feature / artefact	React 19 + Vite console with the same screens, contract and security behaviour as the Angular one: lazy routes, route guards, one HTTP wrapper (bearer, correlation id, normalised errors, 401 handling), `useAsync` for race-free loading, idempotent submit
Source-code location	`frontend-react/src/**`, ADR-013
Test / validation evidence	90 Vitest tests, strict tsc, ESLint (react-hooks, jsx-a11y), prod build, browser e2e 19/19 on the same suite as Angular (E)
Interview talking point	two frontends, one contract: same types, same e2e suite, same wording; the 403 found when the dev proxy rewrote the Host header
Remaining limitations	no React Query, no SSR, Edge only, no visual-regression or axe audit

4 - REST API design

Aspect	Evidence
Implemented feature / artefact	resource model, DTOs, standard error envelope, pagination/sorting whitelist, idempotency, versioning, OpenAPI, correlation ids
Source-code location	`docs/architecture/05-api-design.md`, controllers, `specs/openapi.json`
Test / validation evidence	`ApiContractIT`, `SecurityIT`, Postman/`.http`, live examples `docs/api/examples.md` (E)
Interview talking point	404 vs 403, 202 vs 201, 422 vs 400, compatibility rules
Remaining limitations	no HATEOAS/ETag; v2 not exercised

5 - Microservices and distributed systems

Aspect	Evidence
Implemented feature / artefact	modular monolith + simulator; outbox, idempotent consumer, retry+jitter, circuit breaker, reconciliation, eventual consistency
Source-code location	ADR-001/003/006/011, `RestClearingClient`, `TransactionProcessor`, `ReconciliationService`
Test / validation evidence	`RestClearingClientTest`, `TransactionProcessorTest`, chaos e2e (E)
Interview talking point	when *not* to use microservices; db tx vs distributed workflow; "unknown" outcome
Remaining limitations	no broker/service mesh; one real service boundary only

6 - SQL and relational databases

Aspect	Evidence
Implemented feature / artefact	normalised schema, constraints, partial/composite indexes, row locks, keyset/OR trade-offs, query plans
Source-code location	`V1__core_schema.sql`, `V2__postgresql_hardening.sql`, `docs/architecture/06-database.md`
Test / validation evidence	`SchemaConstraintsIT`, `PostgresFeaturesIT` (real PostgreSQL 14, EXPLAIN plans) (E)
Interview talking point	isolation levels, deadlocks, lock ordering, expand/contract

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Aspect	Evidence
Remaining limitations	no partitioning implemented; PostgreSQL 16 only in compose/CI (S)

7 - Oracle knowledge

Aspect	Evidence
Implemented feature / artefact	Oracle porting guide (syntax/transaction differences)
Source-code location	`docs/architecture/06-database.md` (Oracle section)
Test / validation evidence	none - **conceptual** (C)
Interview talking point	`'' IS NULL`, no partial indexes, DDL auto-commit, `FETCH FIRST`
Remaining limitations	Oracle was not run; no Oracle Flyway folder

8 - MongoDB knowledge

Aspect	Evidence
Implemented feature / artefact	where Mongo fits (read models, documents) and where it does not (ledger)
Source-code location	database chapter + interview guide
Test / validation evidence	none - **conceptual** (C)
Interview talking point	one system of record per fact; projections
Remaining limitations	no Mongo code

9 - Unit, integration and end-to-end testing

Aspect	Evidence
Implemented feature / artefact	81 backend unit tests, Mockito, integration suites on H2 + real PostgreSQL, Angular specs, React tests, API + browser e2e
Source-code location	`src/test/**`, `*.spec.ts`, `tests/e2e`
Test / validation evidence	recorded runs (E) - counts in the validation report
Interview talking point	invariants after every scenario; mutation check
Remaining limitations	no property-based tests; coverage is a signal only

10 - TDD and BDD

Aspect	Evidence
Implemented feature / artefact	red/green/refactor with real output; Gherkin feature bound to tests
Source-code location	`DailyLimitPolicyTest`, `docs/testing/tdd-example.md`, `tests/bdd/transactions.feature`
Test / validation evidence	red run (6 errors) and green run (6 pass) recorded (E)
Interview talking point	tests that fail for the right reason
Remaining limitations	no Cucumber runtime by design

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

11 - Secure coding

Aspect	Evidence
Implemented feature / artefact	JWT validation, role + object-level authorization, rate/size limits, masking, audit, threat model, OWASP mapping
Source-code location	`security/`, `AccessPolicy`, `docs/security/*`
Test / validation evidence	`SecurityIT` (18), `AccessPolicyTest`, `RateLimitIT`, `ServerErrorIT` (E)
Interview talking point	BOLA, maker-checker, fail-closed, no info leakage
Remaining limitations	no pen test, no field encryption, no distributed rate limiter

12 - CI/CD and DevOps

Aspect	Evidence
Implemented feature / artefact	GitHub Actions with build, tests, SAST, SCA, secrets scan, image scan, versioning, deploy, smoke, rollback
Source-code location	`.github/workflows/ci.yml`, `scripts/smoke-test.sh`, `validate-infra.py`
Test / validation evidence	local equivalents executed (E); workflow **not run on a runner** (S)
Interview talking point	gates, build-once-promote, expand/contract
Remaining limitations	pipeline unverified on GitHub; actions not SHA-pinned

13 - Cloud deployment

Aspect	Evidence
Implemented feature / artefact	AWS reference architecture; Kubernetes manifests (probes, HPA, PDB, NetworkPolicy, secrets by reference); Dockerfiles; Compose
Source-code location	`infrastructure/`, ADR-009
Test / validation evidence	structure asserted by `validate-infra.py` (S); **no cluster/Docker run**
Interview talking point	EKS+RDS+Secrets Manager+IRSA, rolling updates, rollback
Remaining limitations	never deployed; Terraform not included

14 - AI-assisted engineering

Aspect	Evidence
Implemented feature / artefact	Copilot instructions, path-scoped rules, prompts, agent definitions, handoff, review recipes, spec-driven example
Source-code location	`.github/copilot-instructions.md`, `ai-engineering/`, `specs/001-account-transfer.md`
Test / validation evidence	the project itself was AI-assisted and verified by recorded runs; **agents not executed** (C)
Interview talking point	context tiers, guardrails, reviewing AI output (5 failure patterns)
Remaining limitations	process artefacts, not a measured productivity study

15 - SOLID principles and design patterns

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Aspect	Evidence
Implemented feature / artefact	state machine, port/adapter, pure policies, outbox, idempotent receiver, compensation, specification
Source-code location	backend architecture chapter
Test / validation evidence	unit tests of the pure units (E)
Interview talking point	when a pattern earns its place; sealed types and exhaustive switch
Remaining limitations	patterns are illustrative at this scale

16 - Technical documentation

Aspect	Evidence
Implemented feature / artefact	two PDFs, ADRs (12), HLD/LLD, runbooks, API examples, threat model, diagrams (17)
Source-code location	`docs/`
Test / validation evidence	PDF structural verification (E)
Interview talking point	docs as code, one fact in one place
Remaining limitations	PDF is generated, not hand-typeset

17 - Performance and reliability engineering

Aspect	Evidence
Implemented feature / artefact	metrics, dashboards, alerts, k6 scripts, local baseline, investigation guides
Source-code location	`performance/`, `observability/`, `docs/operations/performance-troubleshooting.md`
Test / validation evidence	baseline executed (E); k6 **not executed**; numbers are dev-machine only
Interview talking point	measure -> change one thing -> re-measure; 23 -> 34 tx/s example
Remaining limitations	no capacity test on production-like hardware

18 - Observability

Aspect	Evidence
Implemented feature / artefact	structured logs, correlation ids, RED + business metrics, tracing (OTel), health, Grafana, Prometheus alerts
Source-code location	`PlatformMetrics`, `observability/`
Test / validation evidence	metrics endpoint asserted in `ApiContractIT` (E); Grafana/Prometheus/Jaeger **not run** (S)
Interview talking point	alert -> runbook -> trace -> audit
Remaining limitations	dashboards unrendered; no log-aggregation stack run

19 - Technical ownership and leadership

Aspect	Evidence
Implemented feature / artefact	ADRs, sprint plan, risk register, review checklists, 7 code-review scenarios, 6 troubleshooting exercises, mentoring plan, post-incident template
Source-code location	`docs/decisions`, `docs/leadership`, `docs/operations`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Aspect	Evidence
Test / validation evidence	documents (C) cross-referenced to code/tests
Interview talking point	trade-off articulation, blameless culture, mentoring a junior on concurrency
Remaining limitations	process documents are templates until used with a real team

20 - Financial-domain integrity

Aspect	Evidence
Implemented feature / artefact	ledger, idempotency, state machine, reconciliation, maker-checker, audit
Source-code location	`LedgerService`, `TransactionService`, `ReconciliationService`
Test / validation evidence	concurrency + chaos scenarios (E)
Interview talking point	never claim distributed exactly-once; reserve-first
Remaining limitations	simplified domain (no fees/FX/settlement cycles)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Interview question bank

The bank has **142 questions**; 52 are marked **[Scenario]**. Every answer follows the same shape: *short answer, reasoning and implementation, alternatives and trade-offs, risks and pitfalls, likely follow-ups, and where it lives in the project*. Practise saying the short answer aloud first.

Java and Spring Boot

Q1. How does the JVM manage memory, and what happens during garbage collection?

Short answer: objects live on the heap, split (conceptually) into young and old generations; G1 collects young regions frequently with short stop-the-world pauses and reclaims old regions in mixed collections; a Full GC is the failure mode. **Reasoning and implementation:** most objects die young, so a small frequently-collected young area is cheap. Survivors are copied between survivor regions and promoted. Besides the heap the process holds metaspace, thread stacks, code cache and direct buffers, so container memory must exceed the heap (`MaxRAMPercentage=75`). Evidence comes from `-Xlog:gc*`, `jvm_gc_pause_seconds`, JFR and heap dumps. **Alternatives and trade-offs:** ParallelGC (throughput, longer pauses), ZGC/Shenandoah (very low pauses, more memory and CPU). Pick by SLO: latency-bound services may justify ZGC, but only after measurement. **Risks and pitfalls:** tuning flags before reducing allocation; ignoring non-heap memory so the pod is OOMKilled although the heap looks fine; humongous allocations. **Likely follow-ups:** "how do you find a memory leak?" (heap-after-GC trend, heap dump, dominator tree); "G1 vs ZGC?" **In this project:** `platform-service.Dockerfile` sets `MaxRAMPercentage=75`, `ExitOnOutOfMemoryError`, heap dump to `/tmp`; the `JvmHeapHigh` and `LongGcPauses` alerts link to runbook sections.

Q2. [Scenario] Pods are OOMKilled every six hours. How do you investigate?

Short answer: separate heap growth from native/non-heap growth, capture a heap dump, find the retained set, fix the cause, and verify with the same load. **Reasoning and implementation:** (1) chart `jvm_memory_used_bytes{area="heap"}` low points and `process_resident_memory` against the limit - rising low points = leak, flat-but-high = under-sized; (2) `jcmd <pid> GC.heap_dump` (or the automatic dump) into Eclipse MAT: dominator tree and leak suspects; (3) typical culprit in a service: an unpaged `findAll()` in a new endpoint, a cache without eviction, a static collection; (4) fix (paging, eviction, streaming), add a test that asserts the page cap, keep an alert on heap; (5) repeat the load test and watch the heap-after-GC line stay flat. **Alternatives and trade-offs:** raising the limit buys time but hides a leak; restarting on a schedule is a mitigation, not a fix. **Risks and pitfalls:** analysing a dump from a different version than production; forgetting thread stacks and direct buffers when many threads/Netty buffers exist. **Likely follow-ups:** "what if the heap is fine but RSS grows?" (native memory tracking, metaspace, direct buffers, glibc arenas); "how do you do this without downtime?" **In this project:** troubleshooting exercise 4 and the `high-memory` runbook; page size capped at 100 by configuration.

Q3. Explain the Java memory model and why synchronized in the application does not protect a bank balance in a multi-instance deployment.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: the JMM defines visibility and ordering between threads of **one JVM**; a balance shared by several application instances needs a lock that lives where all instances can see it - the database. **Reasoning and implementation:** `synchronized` establishes mutual exclusion and happens-before only inside the JVM. With two pods, each has its own monitor; both can read the same row and both write. The authoritative serialisation point is the database: `SELECT ... FOR UPDATE` (pessimistic), a version column (optimistic), or an atomic conditional `UPDATE`. A unique constraint arbitrates creation races. **Alternatives and trade-offs:** distributed locks (Redis/ZooKeeper) add failure modes (expiry during GC pause) and cannot match the safety of a database constraint; single-writer partitioning avoids contention but needs partitioned infrastructure. **Risks and pitfalls:** assuming a single instance "for now"; Hibernate first-level cache hiding concurrent changes; thinking `@Transactional` alone provides isolation from other readers. **Likely follow-ups:** "what does `volatile` guarantee?"; "what is a happens-before edge?" **In this project:** `LedgerService` locks rows (`AccountRepository.lockById`), `ConcurrencyScenarios` runs 20 competing threads.

Q4. What is a deadlock and how do you prevent it in a transfer between two accounts?

Short answer: two transactions each hold a lock the other needs; prevent it by acquiring locks in a single global order. **Reasoning and implementation:** transfer A->B locks A then B; transfer B->A locks B then A; each waits forever (until the database's detector aborts one). If every operation locks the lower id first, a cycle is impossible. Add bounded lock waits (`lock.timeout=5000`) so contention becomes a retryable error, and keep critical sections short. **Alternatives and trade-offs:** lock a single "account pair" row (extra contention); optimistic retries (livelock risk); `NOWAIT` + retry. **Risks and pitfalls:** one new code path that locks in a different order reintroduces the bug; ids compared with the wrong ordering function (any total order works if used *consistently*; `UUID.compareTo` is signed - fine, as long as all code uses it). **Likely follow-ups:** "how would you detect a deadlock in production?" (database log shows both statements, `pg_locks`); "what is livelock?" **In this project:** `LedgerService.applyTransfer`, `LedgerServiceTest.locksInAscendingIdOrder`, `ConcurrencyScenarios.oppositeTransfersDoNotDeadlock` (H2 and PostgreSQL), mutation check: removing the sort fails the unit test.

Q5. Compare ArrayList, LinkedList, HashMap, ConcurrentHashMap and EnumMap. When do you use each?

Short answer: `ArrayList` by default, `HashMap` for lookup, `ConcurrentHashMap` for shared mutable maps, `EnumMap/EnumSet` for enum keys, `LinkedList` almost never. **Reasoning and implementation:** `ArrayList` has contiguous memory (cache friendly) and O(1) index access; `LinkedList` pays per-node overhead and O(n) index; `HashMap` is O(1) average but needs correct `equals/hashCode` and is not thread-safe; `ConcurrentHashMap` uses CAS and fine-grained locking and offers atomic `compute*` operations; `EnumMap` is a plain array indexed by ordinal - the state machine uses `EnumSet`. **Alternatives and trade-offs:** `TreeMap` for ordering/ranges (O(log n)); immutable `List.of` for safety; `CopyOnWriteArrayList` for rarely-written listener lists. **Risks and pitfalls:** mutable keys; iterating while modifying; assuming `HashMap` ordering; check-then-put races on a plain map. **Likely follow-ups:** "how does HashMap resolve collisions?" (buckets, treeification at 8); "why is `BigDecimal` a bad key?" (`equals` is scale-sensitive). **In this project:** `TokenBucketRateLimiter` uses `ConcurrentHashMap.computeIfAbsent`; `TransactionStatus.allowedNext()` returns `EnumSet`.

Q6. How does dependency injection work in Spring and what are the injection pitfalls?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: the container constructs beans and supplies their dependencies (by type, then qualifier/name); constructor injection is preferred. **Reasoning and implementation:** beans are registered from component scanning/`@Bean` methods; the container resolves constructor parameters by type; ambiguity is resolved by `@Primary`, then `@Qualifier`, then parameter name. Constructor injection makes dependencies explicit, immutable and testable without a container. **Alternatives and trade-offs:** field injection (hides dependencies, hard to test), setter injection (optional dependencies), service locator (avoid). **Risks and pitfalls:** circular dependencies (design smell - break with a port/interface, as `AccountClosureGuard` does); `@Primary` silently overriding name matching (a real near-miss with two `TransactionTemplate` beans); prototype beans injected into singletons; proxies and self-invocation. **Likely follow-ups:** "what happens at startup if no `JwtDecoder` exists?" (fails closed); "how do you override a bean in tests?" **In this project:** `AppConfig` documents the two-template rule; `AccountClosureGuard` breaks the account<->transaction cycle.

Q7. How do Spring transactions work, and what are the common ways they silently fail?

Short answer: `@Transactional` wraps a bean in a proxy that opens/commits/rolls back a transaction around the call; anything that bypasses the proxy bypasses the transaction. **Reasoning and implementation:** the proxy binds a connection to the thread; unchecked exceptions mark rollback; propagation decides join vs new. Self-invocation (`this.other()`), `private`/`final` methods and calls from non-managed objects skip the proxy. Swallowing an exception inside leaves the transaction rollback-only or, worse, commits partial work. **Alternatives and trade-offs:** `TransactionTemplate` (explicit, no proxy surprises, can wrap part of a method), AOP-based declarative style (concise, hides boundaries). **Risks and pitfalls:** I/O inside a transaction; lazy loading outside it; `readOnly` mistaken for security; long transactions holding connections. **Likely follow-ups:** "difference between REQUIRED and REQUIRES_NEW?"; "what is `MANDATORY` good for?" **In this project:** `LedgerService`, `AuditService.record` and `OutboxService.enqueue` are `MANDATORY` so a missing boundary fails loudly; `TransactionProcessor` uses `TransactionTemplate` because the external call must sit *between* two transactions.

Q8. [Scenario] A teammate says "I annotated the service method @Transactional but the audit row is still written when the request fails." What do you check?

Short answer: is the audit write in a *separate* transaction (`REQUIRES_NEW`) or did the exception get swallowed, or is the call self-invoked? **Reasoning and implementation:** check propagation on `AuditService`; if it is `REQUIRES_NEW` the row commits independently by design (denials must be audited even though the request rolls back). If it was meant to roll back, look for `catch` blocks converting exceptions into return values, calls via `this.`, and whether the exception type rolls back by default (checked exceptions do not). A debugger on `TransactionSynchronizationManager.isActualTransactionActive()` or transaction logging (`logging.level.org.springframework.transaction=TRACE`) shows boundaries. **Alternatives and trade-offs:** if audit must be atomic with the change use `REQUIRED`/`MANDATORY`; if it must survive failure use `REQUIRES_NEW` (costs an extra connection - a pool-sizing consideration). **Risks and pitfalls:** `REQUIRES_NEW` inside a loop under a small pool can deadlock the pool (outer holds one connection, inner needs another). **Likely follow-ups:** "how many connections does a request hold in that case?" **In this project:** `recordIndependent` (`REQUIRES_NEW`) for `ACCESS_DENIED`; `record` (`MANDATORY`) for state changes.

Q9. Explain the Hibernate first-level cache, dirty checking and the N+1 problem.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: the persistence context caches loaded entities per transaction and flushes changes by comparing snapshots; N+1 is one query for parents plus one per child collection access. **Reasoning and implementation:** within a transaction `findById` twice returns the same instance; modifications are detected at flush (before commit/queries). N+1 appears when iterating lazy associations; fix with fetch joins, `@EntityGraph`, batch fetching (`default_batch_fetch_size`) or DTO projections. Detect with SQL logging/`hibernate.generate_statistics` and tests that count statements. **Alternatives and trade-offs:** avoid associations (this project stores ids and loads explicitly), read-model queries (jOOQ/native), caching. **Risks and pitfalls:** `EAGER` mappings causing cartesian products; `open-in-view` hiding lazy loads in controllers; large persistence contexts in batch jobs (clear/flush). **Likely follow-ups:** "difference between `save`, `persist`, `merge`?"; "why did `save()` throw an optimistic-locking exception for a new entity?" **In this project:** entities reference other aggregates by id; wrapper `Long` version avoids the merge trap.

Q10. [Scenario] repository.save(new Entity()) throws ObjectOptimisticLockingFailureException on the first insert. Why?

Short answer: Spring Data decided the entity was *not new* (assigned id and a non-null/primitive version), called `merge`, and Hibernate found no row with that version. **Reasoning and implementation:** `isNew` uses the version attribute when present: a **wrapper** `Long` that is null means new -> `persist`; a primitive `long` (always 0) or a getter that returns 0 for null makes it look existing. Fix: wrapper `Long version`, or implement `Persistable<ID>`, or let the database generate ids. Verify with a test inserting a fresh entity. **Alternatives and trade-offs:** `Persistable` makes intent explicit at the cost of boilerplate; generated ids avoid the issue but change locality/portability. **Risks and pitfalls:** "fixing" by catching the exception; adding `@Version` later to an existing entity without migration. **Likely follow-ups:** "what does `@Version` do on update?" (adds `WHERE version = ?`, increments, throws on 0 rows). **In this project:** the bug appeared on the first integration run (every submit returned `409`); the wrapper-version comment in `Customer/Account/FinancialTransaction` records why.

Q11. How do you design exception handling for a REST API?

Short answer: one `@RestControllerAdvice` mapping every exception to a stable, safe error envelope; business errors carry a code; unexpected errors are generic and logged once with a correlation id. **Reasoning and implementation:** an `ApiException(status, code, message)` for domain failures; handlers for validation (itemised violations), malformed JSON (never echo the payload), media type, optimistic lock (409), lock timeout (503 + `Retry-After`), security (403), and a catch-all (500, generic message). Filters outside MVC use the same envelope writer. **Alternatives and trade-offs:** RFC 7807 problem details (standard, slightly more verbose); exceptions-as-flow-control versus result types for expected business outcomes. **Risks and pitfalls:** leaking stack traces/SQL; inconsistent shapes between filters and controllers; logging the same exception at every layer. **Likely follow-ups:** "which status for a business rule violation?" (422 vs 400/409 discussion). **In this project:** `GlobalExceptionHandler`, `ErrorResponses`, `ServerErrorIT` (a fake JDBC URL and password never reach the client).

Q12. What is the difference between authentication and authorization in Spring Security, and where do object-level checks belong?

Short answer: authentication establishes *who*, authorization decides *what* - and "what" has a role layer (annotations) and an object layer (domain policy). **Reasoning and implementation:** the resource-server filter validates the JWT and builds an `Authentication`; `@PreAuthorize("hasRole('ADMIN')")` guards endpoints; but "may this customer see *this* account" depends on data, so it belongs in a pure `AccessPolicy` called by services with the loaded object. **Alternatives and trade-offs:** `@PostAuthorize`/`PermissionEvaluator` (fine, but logic scattered in expressions), ABAC engines (OPA/Cedar) for larger rule sets, gateway-only checks (cannot see domain data). **Risks and pitfalls:** relying on "authenticated" as authorization (BOLA); leaking

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

existence with 403; testing only the happy path. **Likely follow-ups:** "how do you test it exhaustively?" (the policy is pure: matrix tests). **In this project:** `AccessPolicy` + `AccessPolicyTest` + 18 `SecurityIT` tests.

Q13. [Scenario] A security reviewer says customers can read each other's transactions through /transactions/{id}. How do you confirm and fix it?

Short answer: reproduce with two customers, add a failing test, add an object-level check, return 404 and audit the attempt. **Reasoning and implementation:** authenticate as Alice, request Bob's transaction id: if it returns 200, BOLA is confirmed. Write the test first (`SecurityIT.customerCannotTouchOthersTransactions`). Fix in the service: load, compute the owners of source/target accounts, call `AccessPolicy.canViewTransaction`; if false record an independent `ACCESS_DENIED` audit event and respond 404. Audit other endpoints that take ids (cancel, attempts, audit, ledger, search filters) - the same flaw is rarely single. **Alternatives and trade-offs:** 403 is also defensible but confirms existence; database row-level security as a second layer (PostgreSQL RLS) - stronger but needs per-request context plumbing. **Risks and pitfalls:** fixing one endpoint and missing list/search endpoints; relying on UI hiding. **Likely follow-ups:** "how do you prevent regressions?" (a test per endpoint, review checklist, shared policy). **In this project:** `TransactionService.requireVisible`, `SecurityIT`.

Q14. How do you validate input in a Spring Boot API beyond @NotNull?

Short answer: layered validation: syntactic (Bean Validation on DTOs), semantic/business (domain rules), and storage constraints. **Reasoning and implementation:** DTO records with `@Valid`: sizes, patterns (allow-list for free text), `@Digits`, enums, UUIDs; cross-field rules with `@AssertTrue`; strict JSON (unknown properties rejected) to stop mass assignment; business validation in a pure validator returning all violations; database `CHECK`/`UNIQUE` as the last line; return all violations with field names. **Alternatives and trade-offs:** custom constraint annotations for reusable rules; programmatic validation for rules needing I/O. **Risks and pitfalls:** validating only in the UI; deny-list regexes; echoing invalid payloads in errors; trusting `Content-Length`. **Likely follow-ups:** "validation vs authorization order?" (authenticate -> authorize -> validate business rules that depend on data). **In this project:** `SubmitTransactionRequest` (`accountsConsistentWithType`), `TransactionValidator`, constraints in V1.

Q15. What is a good unit test, and how do you decide between mocking and using real objects?

Short answer: a good test fails when the code is wrong, is deterministic, and checks behaviour at the right level; mock *collaborators with side effects*, use real small objects otherwise. **Reasoning and implementation:** pure logic needs no mocks (validator, policy). Orchestration with branches on outcomes (processor) uses mocks to force each branch (`Declined`, `Unknown`) and verifies only interactions that *are* the behaviour (no refund on `Unknown`; reserve before authorize). Use real `Account`/`FinancialTransaction` rather than mocking them. Check by deleting a line and seeing the test fail. **Alternatives and trade-offs:** fakes (in-memory implementations) over mocks for stateful collaborators; integration tests for wiring. **Risks and pitfalls:** over-mocking (tests restate the implementation); verifying every call; sleeping in tests. **Likely follow-ups:** "how do you test code that uses `TransactionTemplate`?" (a no-op transaction manager). **In this project:** `TransactionProcessorTest`, `LedgerServiceTest`, `NoOpTransactions`.

Q16. How do you make time-dependent and random-dependent code testable?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: inject the clock, random source and sleeper. **Reasoning and implementation:** `Clock` bean (`Clock.systemUTC()` in production, `Clock.fixed` in tests), `RandomGenerator` and a `Sleeper` functional interface in `RetryExecutor` so tests assert delays without sleeping; the rate limiter takes a nano-clock supplier so refill is tested by advancing a counter. **Alternatives and trade-offs:** libraries that mock static time (fragile), `Thread.sleep` in tests (slow and flaky). **Risks and pitfalls:** `Instant.now()` scattered in domain code; time zones (store UTC); tests that pass only at certain times of day. **Likely follow-ups:** "how do you test a scheduler?" (call the method directly; make intervals configurable). **In this project:** `RetryAndRateLimitTest`, `Clock` in services.

Q17. [Scenario] A scheduled job processes a queue but under load the backlog grows although CPU is idle. Diagnose.

Short answer: the job's *scheduling pattern* limits throughput (one batch per tick), or work is serialised on a hot resource; measure, then make it work-conserving or parallel. **Reasoning and implementation:** compute the ceiling: batch 20 per 500 ms = 40/s at best; measured drain was ~23 tx/s because per-transaction latency plus the tick gap. Change to a work-conserving loop (keep draining while work exists, with a time budget) - measured ~34 tx/s on the same machine; next steps: parallel workers (safe thanks to lease + optimistic version), `SKIP LOCKED`, or a broker relay. Verify with the same load script before/after. **Alternatives and trade-offs:** more instances (scale-out) vs smarter polling; push (LISTEN/NOTIFY, broker) vs poll. **Risks and pitfalls:** unbounded loops starving other scheduled tasks (time budget), double processing without idempotency. **Likely follow-ups:** "how do you make sure two workers do not process the same event?" (lease + `@Version`). **In this project:** `ProcessingScheduler.drainWithinBudget`, `docs/validation/perf-baseline*.json`.

Q18. How do you externalise configuration and keep secrets out of the application?

Short answer: typed properties with safe defaults, overridden by environment variables; secrets come from a secret store at runtime. **Reasoning and implementation:** `application.yml` holds defaults and every tunable is `${ENV:default}`; `@ConfigurationProperties` records bind and validate; profiles switch behaviour (`local` seeds and the demo issuer); production profile has no issuer so the app fails closed without `OIDC_ISSUER_URI`; secrets live in Secrets Manager/Vault, surfaced as Kubernetes Secrets and referenced with `secretKeyRef`. **Alternatives and trade-offs:** Spring Cloud Config/Vault integration (more moving parts), mounted files vs env (files avoid process listings; env is simpler). **Risks and pitfalls:** defaults that are secrets; profile-specific behaviour that weakens security in production; secrets in logs/actuator `env`. **Likely follow-ups:** "how do you rotate a DB password?" (dual-user overlap + rolling restart). **In this project:** `.env.example` (placeholders), compose `:?` required variables, `external-secret.example.yaml`.

Q19. What do you expose through Actuator and how do you protect it?

Short answer: health (liveness/readiness), info and Prometheus metrics only, on a separate management port in containers, never through the ingress. **Reasoning and implementation:** `management.endpoints.web.exposure.include=health, info, prometheus`, probes enabled, `show-details: never`, management port 9090 (NetworkPolicy allows only the monitoring namespace), no `env/heapdump/threaddump` exposure. A *business* health view is a separate staff-only endpoint. **Alternatives and trade-offs:** securing actuator with a role on the main port (credentials for Prometheus scraping), service-mesh policy. **Risks and pitfalls:** exposing `env`/`heapdump` (secrets, memory), putting dependency checks in the liveness probe (restart storms). **Likely follow-ups:** "difference between liveness and readiness?" **In this project:** `scripts/check-management-port.mjs` proved actuator answers on the management port and 404s on the public one.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q20. How do you configure graceful shutdown and why does it matter?

Short answer: stop accepting new requests, finish in-flight work within a bounded time, then exit - so deployments do not break requests. **Reasoning and implementation:** `server.shutdown=graceful`, `spring.lifecycle.timeout-per-shutdown-phase=20s`, scheduler `await-termination`; in Kubernetes add `preStop: sleep 10` so the load balancer deregisters before SIGTERM, and a `terminationGracePeriodSeconds` larger than both. The processor is also crash-safe (outbox redelivery, recovery), so a hard kill loses no money. **Alternatives and trade-offs:** connection draining at the proxy; shorter grace for faster deploys. **Risks and pitfalls:** grace period shorter than the drain; long-running background jobs ignoring interruption. **Likely follow-ups:** "what happens to a request that arrives after SIGTERM?" **In this project:** `application.yml`, `platform-service.yaml` (`terminationGracePeriodSeconds: 45`).

Q21. [Scenario] Explain how you would add a new transaction type (e.g. "fee") without breaking existing behaviour.

Short answer: extend the closed set deliberately and let the compiler and tests show every place that must change. **Reasoning and implementation:** add the enum value; the exhaustive `switch` statements in validator/ledger/processor fail to compile until handled; add DB `CHECK` update via a new migration (expand: widen constraint), validator rules, ledger postings and entry types, API DTO rules (`accountsConsistentWithType`), UI form rules, BDD scenario and tests (unit, integration, concurrency if balances change); keep the API additive (new enum value is a *client-visible* change - document tolerant-reader expectations). **Alternatives and trade-offs:** strategy per type with registration (more open, less exhaustive checking), or a type table in the DB. **Risks and pitfalls:** forgetting the DB constraint or the frontend; clients with exhaustive enums breaking on a new value. **Likely follow-ups:** "is adding an enum value a breaking change?" **In this project:** `TransactionType`, `TransactionValidator.validate` switch, `ck_tx_type`.

Q22. What are Java records and sealed interfaces good for, and where do you use them?

Short answer: records are immutable data carriers with generated `equals/hashCode/toString`; sealed interfaces model closed alternatives that the compiler can check exhaustively. **Reasoning and implementation:** DTOs, results and config are records (less boilerplate, immutability); `ClearingResult` is a sealed interface with `Authorized | Declined | NotFound | NotSent | Unknown`, so the processor's pattern-matching `switch` has no `default` and a new variant fails compilation until handled. **Alternatives and trade-offs:** Lombok (annotation processing, less explicit), enums with fields (no per-variant data), class hierarchies with visitors (verbose). **Risks and pitfalls:** records are shallowly immutable (mutable components), JPA entities cannot be records, serialisation of records needs a recent Jackson. **Likely follow-ups:** "what is pattern matching for switch?" **In this project:** `ClearingClient.ClearingResult`, DTO records in `*Dtos`.

Q23. How would you profile and fix a slow Spring Boot endpoint?

Short answer: find where the time goes (trace/metrics), then fix the biggest contributor with a before/after measurement. **Reasoning and implementation:** (1) `http_server_requests` by `uri` and percentile; (2) a trace to split time into DB, remote call and application; (3) database: statement timings, `EXPLAIN ANALYZE`, N+1 counts; (4) resource saturation: pool pending, threads, GC, CPU throttling; (5) JFR/async-profiler for CPU; (6) fix (index/paging/batching/cache/timeouts) and re-measure. **Alternatives and trade-offs:** caching first (hides a bad query), scaling out (only helps if the bottleneck is per-instance). **Risks and pitfalls:** optimising without a baseline; testing on tiny data; averages instead of percentiles. **Likely follow-ups:** "p99 is bad but p50 fine?" (queuing, GC, lock waits, noisy neighbours). **In this project:** `performance-troubleshooting.md`, captured query plans, baseline script.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q24. [Scenario] Your service starts but every API call returns 401 after a deployment. What are the likely causes?

Short answer: token validation configuration: issuer, keys, clock, audience or egress to the identity provider. **Reasoning and implementation:** decode a token (header `kid`, `iss`, `exp`) and compare with `OIDC_ISSUER_URI`; check the pod can reach the JWKS endpoint (NetworkPolicy/egress/DNS/TLS trust); check clock skew (NTP); check role-claim mapping (401 vs 403 distinction); confirm the active profile did not enable the local issuer or remove the decoder. The service is fail-closed, so a missing validator crashes at startup rather than returning 200s. **Alternatives and trade-offs:** shorter JWKS cache for faster key rotation vs more IdP calls. **Risks and pitfalls:** "fixing" by disabling validation; logging tokens while debugging. **Likely follow-ups:** "how do you roll keys at the IdP safely?" (publish new key, switch signing, retire old after TTL). **In this project:** troubleshooting exercise 6; `SecurityIT` token tests.

Databases

Q25. Explain the SQL isolation levels and which anomalies each allows.

Short answer: READ COMMITTED prevents dirty reads; REPEATABLE READ also prevents non-repeatable reads (and phantoms in PostgreSQL); SERIALIZABLE prevents all anomalies including write skew, at the cost of aborts. **Reasoning and implementation:** the anomalies are dirty read, non-repeatable read, phantom, lost update and write skew. PostgreSQL uses MVCC snapshots: READ COMMITTED takes a new snapshot per statement; REPEATABLE READ one per transaction; SERIALIZABLE adds serialisable snapshot isolation with conflict detection (`40001`, retry). Lost updates under READ COMMITTED are prevented by row locks (`FOR UPDATE`), conditional updates or a version column. **Alternatives and trade-offs:** raising the isolation level everywhere turns contention into retries; targeted locks give the same safety for the invariants you care about. **Risks and pitfalls:** assuming `@Transactional` alone prevents lost updates; MySQL REPEATABLE READ gap locks and deadlocks. **Likely follow-ups:** "what is write skew?" (two transactions read overlapping data and write disjoint rows, violating an invariant); "how does Oracle differ?" **In this project:** default READ COMMITTED + row locks + constraints (ADR-005, integrity chapter).

Q26. [Scenario] How would you prevent two concurrent withdrawals from overspending an account?

Short answer: serialise on the account row (`SELECT ... FOR UPDATE`), re-check the balance under the lock, change it in the same transaction, and keep a `CHECK (balance >= 0)` as the last line of defence. **Reasoning and implementation:** read-modify-write without a lock lets both requests read 100 and write 20. With `FOR UPDATE` the second waits (bounded to 5 s), then reads the *new* balance and fails with `INSUFFICIENT_FUNDS`. Validation at submit is advisory; the authoritative check happens inside `LedgerService` under the lock. Prove it with 20 concurrent withdrawals against a balance for 10: exactly 10 complete, balance 0.00, ledger integrity empty - on H2 and on real PostgreSQL. **Alternatives and trade-offs:** atomic `UPDATE ... SET balance = balance - :x WHERE id = :id AND balance - :x >= :min` (excellent for a single row, but we also need ledger entry, limit and status checks in one critical section); optimistic version + retry (retry storms on hot rows); SERIALIZABLE (aborts everywhere); single-writer queue per account (needs partitioned infrastructure). **Risks and pitfalls:** checking the balance in the UI/controller only; holding the lock across a network call; inconsistent lock order across accounts. **Likely follow-ups:** "what if the lock wait times out?" (retryable 503); "how do you scale a hot account?" **In this project:** `LedgerService.reserveWithdrawal`, `ConcurrencyScenarios.concurrentWithdrawalsNeverOverspend`, mutation check (removing the lock fails 4 of 5 concurrency tests).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q27. How would you choose between optimistic and pessimistic locking?

Short answer: optimistic when conflicts are rare and retries are cheap; pessimistic when the same row is expected to be contended and the critical section is short. **Reasoning and implementation:** optimistic (`@Version`, `WHERE version = ?`) holds no lock, so readers and writers proceed; losers retry/fail with 409. Pessimistic (`FOR UPDATE`) makes later requests wait, avoiding wasted work and retry storms but risking lock waits/deadlocks. Decide by conflict probability, cost of conflict, read/write ratio and latency tolerance. Many systems use both: pessimistic for balances, optimistic for status changes and edits. **Alternatives and trade-offs:** atomic conditional updates, `SKIP LOCKED` queues, advisory locks. **Risks and pitfalls:** optimistic locking on hot rows (livelock), pessimistic without ordering/timeouts (deadlock, pool exhaustion), forgetting to handle the optimistic exception. **Likely follow-ups:** "what does the client see on a lost optimistic race?" (409 `CONCURRENT_MODIFICATION`, re-read and retry). **In this project:** ADR-005; `@Version` on all entities; cancel-vs-process race resolved by `@Version`.

Q28. How do indexes work, and how do you choose the column order of a composite index?

Short answer: a B-tree maps key order to rows; put equality-filtered columns first, then the range/sort column, and only index what queries need. **Reasoning and implementation:** `(source_account_id, created_at DESC)` serves `WHERE source_account_id = ? ORDER BY created_at DESC LIMIT n` without a sort. A leading column that the query does not constrain makes the index unusable for that query. Selectivity and data distribution matter: the planner may prefer a sequential scan for unselective predicates. Partial indexes cover only interesting rows (`WHERE status IN (...)`). **Alternatives and trade-offs:** covering indexes (`INCLUDE`) for index-only scans; hash/GIN/BRIN for special cases; every index slows writes and takes space. **Risks and pitfalls:** functions on indexed columns, leading wildcards, implicit casts, redundant indexes, stale statistics. **Likely follow-ups:** "why can `OR` on two columns be slow?" (see Q29). **In this project:** V1/V2 migrations and the captured plans (database chapter, appendix).

Q29. [Scenario] The account-history query WHERE source = :id OR target = :id ORDER BY created_at DESC LIMIT 20 is slow for some accounts. How do you analyse it?

Short answer: read the `EXPLAIN (ANALYZE)` plan, look at rows removed by the filter, and consider splitting the `OR` or paginating by keyset. **Reasoning and implementation:** with 100,000 rows the plan for a busy account walks `ix_tx_created` newest-first and filters (20 matches after ~400 entries - cheap); for a sparse account PostgreSQL switches to a bitmap OR of the two account indexes. The risk case is an account whose recent rows are few: the global-index walk discards many rows. Fixes: `UNION ALL` of two indexed queries with `ORDER BY ... LIMIT`, a participants table `(account_id, created_at, transaction_id)`, keyset pagination instead of deep `OFFSET`, `ANALYZE` for fresh statistics. **Alternatives and trade-offs:** denormalised participant table (faster reads, extra write/consistency work); materialised views. **Risks and pitfalls:** optimising on tiny data; trusting estimates instead of actuals. **Likely follow-ups:** "what is keyset pagination?" (`WHERE (created_at, id) < (:c, :i)`). **In this project:** `PostgresFeaturesIT.queryPlans` regenerates the plans; the database chapter explains them.

Q30. How do you design safe, zero-downtime schema migrations?

Short answer: forward-only, additive steps (expand/contract) so the previous and next application versions both work against the schema during rollout and rollback. **Reasoning and implementation:** expand (add nullable column/table/index concurrently), deploy code that writes both, backfill in batches, switch reads, contract (drop) in a later release. Flyway versioned migrations are immutable (checksum-validated); privileged migration role separate from the app role; rehearse against production-sized data to measure lock time; avoid `ALTER` that rewrites large

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

tables. **Alternatives and trade-offs:** blue/green databases (heavy), online schema change tools (gh-ost/pg_repack), Liquibase (richer changelog formats). **Risks and pitfalls:** editing a released migration (startup failure), long exclusive locks, destructive change in the same release as the code. **Likely follow-ups:** "how do you roll back?" (application rollback only; schema stays - that is why changes are additive). **In this project:** `db-migration-guide.md`; a real defect (column too short) was fixed *before* any release by editing V1 - after release a V3 would be required.

Q31. How would you size and troubleshoot a database connection pool?

Short answer: size by what the database can run concurrently, not by request threads; troubleshoot by looking at pending threads, acquire time and what active connections are doing. **Reasoning and implementation:** start near `cores x 2` per database instance shared across all replicas (`replicas x pool <= max_connections`), set a short `connectionTimeout` (3 s) so exhaustion fails fast, enable leak detection. Exhaustion symptoms: `hikaricp_connections_pending > 0`, acquire-time p95 rising. Investigate `pg_stat_activity` (`active`, `idle in transaction`), locks, slow queries, and whether any transaction wraps I/O. **Alternatives and trade-offs:** PgBouncer/RDS Proxy (transaction pooling), virtual threads (do not remove the pool limit). **Risks and pitfalls:** raising the pool to "fix" slow queries; `REQUIRES_NEW` inside loops consuming two connections per request. **Likely follow-ups:** "what does `idle in transaction` tell you?" **In this project:** `application.yml` Hikari settings, `ConnectionPoolSaturated` alert, runbook and troubleshooting exercise 1.

Q32. [Scenario] A withdrawal request hangs for 30 seconds, then returns 503 LOCK_TIMEOUT. What happened and what do you do?

Short answer: another transaction holds the account's row lock for too long; find the holder, understand why, and fix the code path or kill the session. **Reasoning and implementation:** the lock wait is bounded (5 s in code; here it was exceeded by queued waiters). Query `pg_stat_activity` joined with `pg_blocking_pids` to find the head blocker and its statement/state. Common causes: a transaction doing I/O while holding the lock, an abandoned session `idle in transaction`, a batch job locking many accounts. Mitigate (terminate the blocker), then fix (shorten the critical section, remove I/O, ordered locks). **Alternatives and trade-offs:** `NOWAIT` + client retry; shorter lock timeout to shed load faster. **Risks and pitfalls:** killing the wrong session; raising timeouts (worse pile-ups). **Likely follow-ups:** "how would you alert on this?" (lock-wait metric, 503 rate). **In this project:** `ServerErrorIT.lockTimeoutIs503`, blocking-locks query in the database chapter.

Q33. What is the difference between UNIQUE constraints, application checks and idempotency keys?

Short answer: only a database constraint can decide a race between application instances; application checks are optimisations, idempotency keys *use* a unique constraint. **Reasoning and implementation:** "select then insert" is a race; two instances both see "no row" and insert. A `UNIQUE (principal, key)` makes the second insert fail atomically; the application catches the violation, rolls back, and replays the winner's result. **Alternatives and trade-offs:** `INSERT ... ON CONFLICT DO NOTHING` (PostgreSQL), distributed locks (weaker). **Risks and pitfalls:** catching the violation inside the same transaction and continuing (the transaction is aborted/rollback-only); not scoping keys by principal. **Likely follow-ups:** "why roll back the whole transaction?" (the loser's other rows - outbox, audit - must not survive). **In this project:** `TransactionService.submit`, `IdempotencyIT.concurrentSameKeyCreatesOneTransaction`.

Q34. When would you use MongoDB or another document store alongside PostgreSQL?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: for flexible-schema documents and read-optimised projections, never as the system of record for the ledger. **Reasoning and implementation:** KYC document metadata, notification payloads, event/audit sinks with TTL, and denormalised customer-activity views fit documents. Feed them from events (outbox) so the relational store stays authoritative and projections can be rebuilt. **Alternatives and trade-offs:** PostgreSQL JSONB (one system, good enough for many cases), Elasticsearch for search. **Risks and pitfalls:** dual writes without an outbox, treating a projection as truth, multi-document transactions as a crutch. **Likely follow-ups:** "how do you keep them consistent?" (idempotent projectors, rebuild from the log). **In this project:** conceptual only (database chapter); no Mongo code.

Q35. How would you port this design to Oracle?

Short answer: change the dialect/driver and Flyway vendor folder, then fix the semantic differences: empty strings are NULL, no partial indexes, DDL auto-commits, different pagination syntax and booleans. **Reasoning and implementation:** `'' IS NULL` affects validation/uniqueness; partial unique index becomes a function-based unique index on a `CASE` expression; `LIMIT/OFFSET` becomes `FETCH NEXT`; boolean as `NUMBER(1)`; `RAW(16)` for UUIDs; plans via `EXPLAIN PLAN`/AWR; deadlock and isolation behave similarly at READ COMMITTED. Retest the concurrency suite on Oracle. **Alternatives and trade-offs:** keep PostgreSQL where allowed; use Oracle-compatible abstractions (JPA) but own the migration folder per vendor. **Risks and pitfalls:** native SQL left in code; assuming migrations are atomic. **Likely follow-ups:** "what does `ORA-08177` mean?" (serialisation failure). **In this project:** `docs/architecture/06-database.md` (Oracle section) - conceptual, not executed.

Q36. [Scenario] A migration that added a NOT NULL column locked the table for minutes in production. How do you avoid that?

Short answer: add the column nullable, backfill in batches, add the constraint in a validated-in-two-steps way, and do it across releases. **Reasoning and implementation:** PostgreSQL 11+ adds a column with a constant default instantly, but a volatile default or a `NOT NULL` check on existing rows scans the table under a strong lock. Safer: `ADD COLUMN ... NULL`; deploy code writing it; backfill with small batches (`UPDATE ... WHERE id BETWEEN`); `ADD CONSTRAINT ... CHECK (col IS NOT NULL) NOT VALID` then `VALIDATE CONSTRAINT` (weaker lock); finally `SET NOT NULL`. Create indexes with `CONCURRENTLY`. **Alternatives and trade-offs:** maintenance window (simplest, but downtime), online migration tools. **Risks and pitfalls:** running migrations inside the application at startup on every replica (race, long locks) - run them as a controlled job. **Likely follow-ups:** "what lock does `CREATE INDEX` take?" **In this project:** guidance in the migration guide and release checklist.

Q37. How do you design data retention for audit, idempotency and transaction data?

Short answer: keep each class of data for as long as its purpose requires, with cheap archival for the long-lived ones. **Reasoning and implementation:** idempotency keys: days (longest client retry window) with a purge job; audit: years (regulatory), append-only, partitioned by month, cold partitions archived; transactions/ledger: partition by time, archive after the reporting window; PII: minimise, mask, support erasure by design; outbox: delete dispatched rows after N days. **Alternatives and trade-offs:** soft delete (complex queries), separate archive database. **Risks and pitfalls:** deleting audit data needed for investigations; unbounded growth of the outbox; partitions without maintenance. **Likely follow-ups:** "how does GDPR erasure interact with an immutable audit trail?" (store identifiers not PII; crypto-shredding). **In this project:** `TransactionMaintenance.purgeExpiredIdempotencyRecords`, append-only audit trigger.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q38. [Scenario] The ledger and a customer's balance disagree by 0.01. How do you find the cause?

Short answer: run the integrity query, locate the account, and walk the ledger entries and transactions to find the posting that broke the invariant. **Reasoning and implementation:** `GET /ops/ledger-integrity` (or the SQL: balance vs signed sum) identifies the account; list its ledger entries with `balance_after` to find where the running balance diverges; match entries to transactions and their states (a `RECONCILIATION_REQUIRED` withdrawal has a `DEBIT` and no payout; a reversal exists once). Causes: a code path that changes the balance without a ledger entry, rounding applied on one side, a manual SQL change. The database constraints (unique postings, non-negative balance) narrow the suspects. **Alternatives and trade-offs:** periodic automated integrity checks and alerts (cheap, recommended). **Risks and pitfalls:** "fixing" by editing the balance (breaks the audit story) instead of posting a correcting entry. **Likely follow-ups:** "how do you prevent it?" (single mutation path, tests asserting the invariant after every scenario). **In this project:** `OpsService.ledgerIntegrity`, asserted after every concurrency and e2e scenario.

Microservices, distributed systems and financial transaction integrity

Q39. [Scenario] How would you design a highly available financial transaction service?

Short answer: stateless service instances across zones, a highly-available database with synchronous replication, idempotent APIs, asynchronous processing with at-least-once delivery and idempotent consumers, explicit failure states and reconciliation, and tested recovery. **Reasoning and implementation:** (1) *Availability of compute:* >= 2-3 replicas across AZs behind a load balancer, readiness probes, graceful shutdown, rolling deploys with zero unavailable pods, PDBs. (2) *Data:* a Multi-AZ PostgreSQL (synchronous standby, automatic failover, PITR backups); the ledger is the system of record; consistency over availability for money state. (3) *Correctness under retries and crashes:* idempotency keys, transactional outbox, idempotent processor, unique ledger postings. (4) *External dependencies:* timeouts, retry with jitter for idempotent calls, circuit breaker, "unknown" outcomes held for reconciliation. (5) *Operations:* SLOs with error budgets, alerts with runbooks, chaos and failover drills, capacity headroom, backups tested by restoring. **Alternatives and trade-offs:** active-active multi-region (higher availability, much harder consistency - needs partitioning by customer and conflict-free design); event sourcing; managed services versus self-run. **Risks and pitfalls:** claiming "exactly-once", forgetting the database as the single point of failure, untested failover, HA that loses acknowledged writes (async replication). **Likely follow-ups:** "RPO/RTO?"; "what happens during failover to in-flight requests?"; "multi-region?" **In this project:** stateless replicas, probes/PDB/HPA in the manifests, outbox + reconciliation, the chaos e2e; DB HA described for RDS (not run).

Q40. [Scenario] How do you handle duplicate transaction submissions?

Short answer: require an idempotency key, persist it atomically with the effect under a unique constraint, replay the original result for the same key and payload, and reject reuse with a different payload. **Reasoning and implementation:** client generates the key once per logical action and reuses it on retries. Server: fast lookup by `(principal, key)`; on miss, one DB transaction inserts the transaction and the idempotency record (unique constraint arbitrates races); the loser's transaction rolls back and it replays the winner's record. A SHA-256 fingerprint detects key reuse with a different body (422). Records are retained for the retry window (7 days) then purged. Downstream, the processor is idempotent (status check + unique ledger postings). **Alternatives and trade-offs:** time-window deduplication (false positives/negatives), Redis keys (not atomic with DB), exactly-once brokers (do not extend to the database/external effects). **Risks and pitfalls:** key scope (per principal), regenerating a key on retry (defeats the purpose), caching only successes

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

(replay rejections too). **Likely follow-ups:** "what if the first request is still in flight when the retry arrives?" (the unique insert blocks until the first commits, then the retry replays). **In this project:** `TransactionService.submit`, `IdempotencyIT`, SPA `IdempotencyKeys`.

Q41. [Scenario] How would you guarantee that an account transfer is atomic?

Short answer: do the debit, the credit and the ledger entries in one database transaction on one database, with locks and constraints guarding the invariants. **Reasoning and implementation:** lock both account rows in ascending id order, re-check status and funds, debit and credit, insert two ledger entries (unique per tx/account/type), move the transaction to `COMPLETED`, commit. Any failure rolls everything back; business failures are recorded afterwards in a fresh transaction. The `CHECK balance >= 0`, unique postings and a ledger-integrity check make a half-applied transfer both impossible and detectable. **Alternatives and trade-offs:** if debit and credit were in different services/databases you would need a saga with compensations (half-applied states become visible and must be reconciled) or 2PC (blocking, rarely available) - the reason accounts and ledger share one service here. **Risks and pitfalls:** splitting the two updates across transactions "for performance"; auto-commit; locking in inconsistent order; JPA batch updates hiding partial failure. **Likely follow-ups:** "what if the process crashes between debit and credit?" (the transaction rolls back - neither is applied). **In this project:** `LedgerService.applyTransfer`, `ConcurrencyScenarios.oppositeTransfersDoNotDeadlock`, money-conservation checks in the baseline.

Q42. What is the difference between a database transaction and a distributed workflow, and what can each guarantee?

Short answer: a database transaction gives atomicity/isolation within one database; a distributed workflow coordinates several independent systems with explicit states and compensations and can offer only eventual consistency. **Reasoning and implementation:** you cannot roll back an HTTP call to another company. So local changes are transactional; the external step is modelled as states (`PROCESSING -> COMPLETED | FAILED | RECONCILIATION_REQUIRED`), idempotent calls, retries, compensation (reversal credit) and reconciliation. Delivery guarantees compose as: at-least-once + idempotent consumer = effectively-once *effects*; never claim distributed exactly-once. **Alternatives and trade-offs:** 2PC/XA (limited support, availability cost), sagas (orchestrated or choreographed), outbox + CDC. **Risks and pitfalls:** treating timeouts as failure; hiding the unknown state; compensations that can themselves fail (need retries and alerts). **Likely follow-ups:** "orchestration vs choreography?" **In this project:** ADR-003, ADR-011, integrity chapter.

Q43. [Scenario] A call to a third-party payment network times out. What state is the transaction in and what do you do?

Short answer: *unknown*; keep the funds reserved, mark `RECONCILIATION_REQUIRED`, and ask the network what happened - never guess. **Reasoning and implementation:** classify results: authorized/declined (definite), not-sent (connection refused/DNS/4xx/circuit open - provably not executed: refund and fail fast), unknown (read timeout, 5xx after retries - may have executed). For unknown: open a reconciliation record, reconcile automatically with a lookup by our reference (idempotent read), refund after a grace period if the network has no record, alert after 15 minutes, allow manual resolution with maker-checker. **Alternatives and trade-offs:** assume failure (risk of double payout), assume success (phantom payout), synchronous retries until certain (blocks resources). **Risks and pitfalls:** refunding on timeout; retrying a non-idempotent call; no alert on stuck records. **Likely follow-ups:** "what if the network has no lookup API?" (negotiate one, or use settlement reports for daily reconciliation). **In this project:** `TransactionProcessor`, `ReconciliationService`, chaos e2e (`.66` cents case).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q44. Explain the transactional outbox pattern and its alternatives.

Short answer: write the event to an outbox table in the same transaction as the state change; a relay delivers it later, so the change and its announcement cannot diverge. **Reasoning and implementation:** solves the dual-write problem (DB commit succeeded but the message send failed, or vice versa). The dispatcher claims events (lease + version), invokes the handler, marks dispatched, retries with backoff, dead-letters after N attempts. Delivery is at-least-once, so consumers must be idempotent. **Alternatives and trade-offs:** change-data-capture (Debezium) publishing the table to Kafka (no polling, more infrastructure); listen/notify; "listen to yourself" event publishing (loses events on crash); 2PC (no). **Risks and pitfalls:** unbounded outbox growth, polling load, ordering assumptions (per aggregate only), non-idempotent handlers. **Likely follow-ups:** "how do you keep ordering?" (partition by aggregate id; single claimer per aggregate). **In this project:** `OutboxService`, `OutboxDispatcher`, ADR-006, `OutboxDeadLetterIT`.

Q45. [Scenario] Messages are redelivered and a handler runs twice. How do you prevent double effects?

Short answer: make the handler idempotent: check state first and back it with a unique constraint. **Reasoning and implementation:** the processor loads the transaction and returns if the status is not `VALIDATED`; the ledger insert is unique per `(transaction_id, account_id, entry_type)`; reversal checks `funds_reserved` and the existing reversal entry. Redelivery after a crash between debit and outcome is handled by recovery, not by re-running the debit. **Alternatives and trade-offs:** dedup table of processed message ids (extra storage, generic), natural idempotency of the operation. **Risks and pitfalls:** idempotency by "have I seen this id in memory"; side effects outside the dedup scope (e-mails). **Likely follow-ups:** "how do you test it?" (process the same id twice - `duplicateDeliveryIsHarmless`). **In this project:** `TransactionProcessorTest.nonValidatedTransactionIsSkipped`, `ConcurrencyScenarios.duplicateDeliveryIsHarmless`.

Q46. When should you use microservices versus a modular monolith?

Short answer: start with a modular monolith; extract services when independent scaling, team ownership or release cadence justifies the distributed-systems cost - and never split things that share an invariant. **Reasoning and implementation:** the costs of microservices (network failure, distributed transactions, N pipelines, observability, contract management) must be paid by a benefit you can name. Boundaries follow business capabilities and data ownership. Keep module dependencies one-directional so extraction is possible later. **Alternatives and trade-offs:** a monolith with a separate worker deployment (cheap first step), serverless functions for edge tasks. **Risks and pitfalls:** "distributed monolith"; shared database across services; premature splitting; resume-driven architecture. **Likely follow-ups:** "how would you extract the first service?" (outbox dispatcher/reporting). **In this project:** ADR-001 and the migration path.

Q47. How do you define service boundaries and data ownership?

Short answer: by business capability and by who changes which data together; each datum has exactly one owning service. **Reasoning and implementation:** event-storm the domain, find aggregates and invariants, keep an invariant inside one boundary, expose other services' data via APIs/events (not shared tables), version contracts, and make cross-boundary consistency eventual and observable (reconciliation). Anti-signals: two services editing the same table, long synchronous chains, joint deployments. **Alternatives and trade-offs:** shared-library "domain" modules (couples releases), data mesh for analytics. **Risks and pitfalls:** boundaries by technical layer ("DAO service"), chatty fine-grained services. **Likely follow-ups:** "what about reporting across services?" (read models/CQRS). **In this project:** table ownership in the architecture chapter; `AccountClosureGuard` keeps account independent of transaction persistence.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q48. What is a saga and when would you not use one?

Short answer: a sequence of local transactions where each step has a compensating action; use it when a business process spans services that cannot share a transaction - and avoid creating that situation when a single transaction would do. **Reasoning and implementation:** orchestration (a coordinator drives steps and compensations) vs choreography (services react to events). Requirements: idempotent steps and compensations, durable saga state, timeouts, visibility, handling of compensation failure (retry + alert). Example here (single external step): reserve funds (local) -> external call -> complete or reverse - a minimal saga. **Alternatives and trade-offs:** one transaction in a monolith; 2PC; TCC (try-confirm-cancel). **Risks and pitfalls:** compensations that cannot undo (e-mail sent), isolation anomalies (others see intermediate state - e.g. reserved funds), complexity explosion. **Likely follow-ups:** "how do you show intermediate state to users?" (explicit statuses). **In this project:** ADR-003/011.

Q49. [Scenario] Design retry behaviour for a flaky downstream API.

Short answer: retry only idempotent calls, a few times, with exponential backoff and full jitter, a timeout per attempt, an overall deadline, and a circuit breaker; classify the final failure honestly. **Reasoning and implementation:** delay = random(0, min(cap, base x 2^attempt)); 3 attempts; connect 500 ms / read 2 s; breaker opens at 50 % failures over 10 calls and half-opens after 10 s; metrics for retries/breaker state; after exhaustion return *unknown* (not failed) when the request may have executed. **Alternatives and trade-offs:** hedged requests (lower tail latency, extra load), retry budgets (limit retries to a % of traffic), queue-and-retry later (asynchronous). **Risks and pitfalls:** retry storms (no jitter), amplification across layers (each layer retrying), retrying non-idempotent calls. **Likely follow-ups:** "why full jitter?" (decorrelates clients; best recovery behaviour in published analyses); "retry budgets?" **In this project:** `RetryExecutor`, `RestClearingClientTest`, `RetryAndRateLimitTest`.

Q50. How does a circuit breaker work and what do you do while it is open?

Short answer: it counts failures in a sliding window; above a threshold it opens and rejects calls immediately for a cool-down, then half-opens to probe. **Reasoning and implementation:** protects the caller's threads/connections and gives the dependency time to recover. While open the platform answers withdrawals with a fast `CLEARING_UNAVAILABLE` and refunds (the request was provably not sent), deposits and transfers continue, health shows `DEGRADED`, an alert fires. After the cool-down a probe call decides whether to close. **Alternatives and trade-offs:** bulkheads, rate limiting, load shedding, fallback data. **Risks and pitfalls:** thresholds too sensitive (flapping) or too lax; counting client errors as failures; hiding open state from operators. **Likely follow-ups:** "which exceptions count as failures?" (transport failures only). **In this project:** `ClearingConfig` (Resilience4j), `platform_clearing_circuit_open` gauge, chaos e2e.

Q51. How do you design backward-compatible REST APIs?

Short answer: only additive changes within a version, tolerant readers, contract tests, and a deprecation process for breaking changes. **Reasoning and implementation:** allowed: add optional request fields, add response fields, add endpoints, add error codes; breaking: remove/rename, change types/scale/meaning, new required input, tighter validation, new status codes for existing flows. Detect drift automatically (OpenAPI diff in CI), version in the URI (`/api/v1`), support old and new side by side, announce with `Deprecation`/`Sunset` headers, monitor usage per client, then remove. **Alternatives and trade-offs:** header/media-type versioning, date-based versions, consumer-driven contracts (Pact). **Risks and pitfalls:** "additive" enum values breaking strict clients; changing money scale; silently changing defaults. **Likely follow-ups:** "is adding a new enum value breaking?" (depends on client tolerance - document it). **In this project:** ADR-007, `ApiContractIT`, CI `git diff --exit-code specs/openapi.json`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q52. [Scenario] How would you secure a multi-tenant financial application?

Short answer: make the tenant a first-class, server-derived part of every request and enforce isolation in several layers: token claims, authorization policy, query scoping, and ideally database row-level security. **Reasoning and implementation:** (1) tenant id comes from the validated token, never from the request body; (2) every query is scoped by tenant (a mandatory filter/specification, or PostgreSQL RLS with `SET LOCAL app.tenant`); (3) object-level checks on every id; (4) per-tenant rate limits and quotas; (5) separate encryption keys per tenant for sensitive data; (6) audit includes tenant; (7) tests that attempt cross-tenant access on *every* endpoint; (8) consider silo (database per tenant) vs pool (shared schema) vs bridge (schema per tenant) by risk/cost. **Alternatives and trade-offs:** DB-per-tenant (strongest isolation, highest cost, operational scale issues), schema-per-tenant, shared tables with RLS (best density, requires discipline). **Risks and pitfalls:** missing a filter on one query (BOLA across tenants), caches keyed without tenant, background jobs without tenant context, noisy neighbours. **Likely follow-ups:** "how do you test isolation?"; "how do you handle a super-admin?" **In this project:** the customer-scoping model (`customer_id` claim, `AccessPolicy`, forced scope in searches) is the single-level version of this design; multi-tenancy itself is out of scope.

Q53. [Scenario] Two microservices must both update data for one business operation. What do you do?

Short answer: first ask whether they should be one service; if not, use the outbox + idempotent consumers (+ saga/compensation) and make intermediate states explicit. **Reasoning and implementation:** the initiating service commits its change and an outbox event atomically; the second service consumes idempotently and acknowledges; failure of the second step triggers a compensating action or a visible `PENDING`/`FAILED` state with reconciliation; use timeouts and alerts for stuck workflows; never use a distributed lock as the consistency mechanism. **Alternatives and trade-offs:** merge the services; synchronous call with compensation (simpler, couples availability); 2PC where genuinely supported. **Risks and pitfalls:** hidden coupling, lost messages (dual write), duplicate effects. **Likely follow-ups:** "how do you test it?" (fault injection between steps). **In this project:** the external clearing step is exactly this situation, solved with reservation + reconciliation.

Q54. How do you achieve eventual consistency and keep users informed?

Short answer: make asynchronous states first-class resources and give clients a way to observe progress. **Reasoning and implementation:** `202 Accepted` with a resource URL, explicit statuses (`VALIDATED`, `PROCESSING`, `RECONCILIATION_REQUIRED`), polling/server-sent events/webhooks, idempotent retries, dashboards showing stuck work, SLOs on end-to-end latency. For read models, show "as of" timestamps and handle read-your-writes by reading from the primary or using version tokens. **Alternatives and trade-offs:** synchronous completion (simpler client, couples latency to slowest dependency). **Risks and pitfalls:** hiding intermediate states; unbounded staleness without alerts. **Likely follow-ups:** "polling vs push?" **In this project:** transaction status endpoint, 15-second dashboard refresh, end-to-end latency histogram.

Q55. [Scenario] A customer says a withdrawal reduced their balance but the money never arrived. How do you investigate?

Short answer: follow the transaction through its states, attempts and audit trail, check the reconciliation record and the network's own record. **Reasoning and implementation:** find the transaction by reference; check status: `RECONCILIATION_REQUIRED` means funds are reserved while the outcome is unknown; `COMPLETED` means the network authorized (check authorization id in the attempt detail and the network's settlement); `FAILED` should show a reversal credit in the ledger. Verify ledger entries (`DEBIT` then maybe `REVERSAL_CREDIT`) and the integrity check. Resolve manually

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

if needed (maker-checker), then reply to the customer with the facts. **Alternatives and trade-offs:** reconcile against the network's daily settlement file as a second detection net. **Risks and pitfalls:** editing balances directly; promising refunds before the facts are known. **Likely follow-ups:** "how long can funds stay reserved?" (alert after 15 minutes; policy for escalation). **In this project:** `transactions/{id}/attempts`, `/audit`, `/ops/reconciliations`, runbook.

Q56. What consistency guarantees does the dashboard have and why is that acceptable?

Short answer: it reads the same database as the API, so it is strongly consistent for what it shows at query time, but values are computed from a snapshot that may be seconds old and aggregates are not transactional with writes. **Reasoning and implementation:** counts and sums run as separate queries (each statement sees a committed snapshot), so a transaction may be counted as `PROCESSING` in one query and `COMPLETED` in another; for operational awareness that is fine, for financial reporting you would use a consistent snapshot (`REPEATABLE READ` read-only) or a dedicated reporting store. **Alternatives and trade-offs:** replicas (lag), materialised views, event-fed read models. **Risks and pitfalls:** treating dashboard numbers as accounting truth. **Likely follow-ups:** "how would you make it consistent?" **In this project:** `OpsService.summary`.

Q57. How do you handle cancellation vs processing races?

Short answer: make the state transition itself the arbiter with an optimistic version so exactly one of "cancel" and "start processing" wins. **Reasoning and implementation:** both operations load the transaction at version *n* and attempt to write a transition (`VALIDATED -> CANCELLED` or `-> PROCESSING`); the second update fails the `WHERE version = n` check. The loser of cancel gets 409; the loser of processing finds a non-`VALIDATED` status on retry and skips. Tests: cancel then drain leaves the transaction cancelled and the balance untouched. **Alternatives and trade-offs:** row lock on the transaction (more blocking), cancel as a request that the worker honours (eventual). **Risks and pitfalls:** cancelling after funds are reserved (not allowed by the state machine). **Likely follow-ups:** "why not allow cancel in `PROCESSING`?" (external effects may be in flight). **In this project:** `TransactionStatus.isCancellable`, `TransactionFlowIT.cancellation`.

Q58. [Scenario] Operations asks for a maker-checker control on large transactions. How do you implement it?

Short answer: a review threshold sends large submissions to `PENDING_REVIEW`; approval requires a different authenticated operator than the submitter; everything is audited. **Reasoning and implementation:** validator returns `REVIEW`; state machine allows `PENDING_REVIEW -> VALIDATED | REJECTED | CANCELLED`; `AccessPolicy.canReview` requires an operator role and `subject != submittedBy` (applies to admins as well); approval writes `reviewedBy`, the outbox event and an audit row; tests cover self-approval (403), customer (403), second operator (OK). **Alternatives and trade-offs:** multi-level approvals, dual control with time windows, amount tiers per role. **Risks and pitfalls:** shared accounts defeating maker-checker; approving a stale request (re-validate at approval - processing re-checks authoritative rules anyway). **Likely follow-ups:** "what about collusion?" (separation of duties + audit analytics). **In this project:** `TransactionService.approve`, `AccessPolicyTest.makerChecker`, e2e scenario.

Q59. How do you scale a system where one account is extremely hot?

Short answer: accept serial processing per account but shrink the critical section, or restructure the hot spot (batching, sharded counters, partitioned queues). **Reasoning and implementation:** the row lock serialises per account; per-account throughput = 1 / (critical-section time). Measure it; remove I/O from the lock; batch many small postings into one transaction per account per tick; for a house/fee account use sub-accounts (shards) whose totals are aggregated; or route by account id to a single-writer partition. **Alternatives and trade-offs:** optimistic with retries (poor under hot contention), eventual-consistent counters (not for balances that gate spending). **Risks and pitfalls:**

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

hidden hot rows (shared totals, sequence rows), premature sharding. **Likely follow-ups:** "what do you monitor?" (lock waits, per-account queue depth). **In this project:** baseline scenario "concurrent withdrawals on one account"; risk R4.

Q60. What is reconciliation and how often should it run?

Short answer: comparing two sources of truth and repairing differences; continuous for unknown outcomes, periodic for end-to-end consistency. **Reasoning and implementation:** *Event-driven:* the open-records reconciler every 15 s. *Periodic:* daily ledger-vs-balance integrity and a settlement-file comparison against the network; mismatches raise tickets. Always prefer *detect and alert* before *auto-correct*; corrections are new postings, never edits. **Alternatives and trade-offs:** real-time verification (costly), sampling. **Risks and pitfalls:** auto-fixing errors you do not understand; no ownership for exceptions. **Likely follow-ups:** "what do you do with unexplained differences?" **In this project:** `ReconciliationService.autoReconcile`, `/ops/ledger-integrity`.

Q61. How do you design for partial failure of a dependency you do not control?

Short answer: assume it will be slow, wrong, or down; bound waiting, isolate it, degrade gracefully, and make every outcome representable. **Reasoning and implementation:** timeouts and breaker; async processing so request threads are not tied up; separate failure states for *not sent* and *unknown*; keep unaffected capabilities (deposits/transfers) working; health and alerts; a simulator with chaos modes to rehearse; documented runbooks. **Alternatives and trade-offs:** queue and retry later (customer waits), alternative provider (cost/complexity). **Risks and pitfalls:** happy-path-only integration tests; missing timeouts. **Likely follow-ups:** "how do you test it?" (fault injection). **In this project:** `clearing-simulator` chaos modes and `e2e-api.mjs --with-chaos`.

Q62. What would you change first if this platform had to handle 100x the load?

Short answer: measure to find the first bottleneck; expect it to be the database and the single-threaded in-database queue. **Reasoning and implementation:** (1) parallel outbox workers (`SKIP LOCKED`) and a broker relay; (2) read replicas for dashboards/history; (3) partition large tables; (4) keyset pagination and the participants table; (5) shard by customer if one primary is not enough; (6) cache reference data; (7) tune pools and use PgBouncer; (8) load-test with arrival-rate tools on production-like hardware; (9) per-account batching for hot accounts. **Alternatives and trade-offs:** CQRS/event sourcing, specialised stores for read models. **Risks and pitfalls:** scaling before measuring; breaking invariants while sharding. **Likely follow-ups:** "how do you keep transfers atomic across shards?" (route by customer; cross-shard transfers use a saga with reservation). **In this project:** risk register R4/R5 and the dispatcher measurement (23 -> 34 tx/s locally).

Angular and REST APIs

Q63. What does OnPush change detection do, and how do signals relate to it?

Short answer: an `OnPush` component is checked only when an input reference changes, an event fires inside it, a signal it reads changes, or it is marked dirty; signals make that precise. **Reasoning and implementation:** Default change detection walks the whole component tree after every async event (zone.js). `OnPush` prunes subtrees. Signals record which components read them, so a `signal.set()` schedules only those views. Together: immutable-style updates, `computed` derived state, no manual `markForCheck` in most cases. **Alternatives and trade-offs:** zoneless Angular (no zone.js; signals/`markForCheck` drive updates), Default CD (simplest, slowest). **Risks and pitfalls:** mutating objects in place under `OnPush` (no update); reading non-signal mutable fields in templates. **Likely follow-ups:** "what triggers a check in OnPush?" **In this project:** every component is `OnPush`; view state uses `signal` + `computed`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q64. RxJS: when do you use switchMap, mergeMap, concatMap and exhaustMap?

Short answer: `switchMap` cancels the previous inner (typeahead, route-driven loads), `mergeMap` runs in parallel, `concatMap` queues in order, `exhaustMap` ignores new triggers while busy (submit buttons). **Reasoning and implementation:** HTTP cancellation happens when you unsubscribe, so `switchMap` prevents stale responses overwriting fresh ones. `exhaustMap` protects against double submit at the stream level (the form here also disables the button and relies on idempotency keys server-side). `forkJoin` waits for all to complete and **cancels siblings on the first error**. **Alternatives and trade-offs:** signals + `toSignal` for simple loads; async/await with `firstValueFrom` for one-shot calls. **Risks and pitfalls:** nested subscribes, leaking subscriptions (use `takeUntilDestroyed`), using `mergeMap` for ordered work. **Likely follow-ups:** "what is the difference between cold and hot observables?" **In this project:** dashboard `forkJoin`, form submit handling; tests learnt that sibling requests are cancelled on error.

Q65. How do signals differ from RxJS observables, and when do you use which?

Short answer: signals hold current synchronous state with automatic dependency tracking; observables model asynchronous event streams over time. **Reasoning and implementation:** state shown in templates (loading flag, rows, derived `viewState`) is signal territory (`signal`, `computed`); async sources (HTTP, timers, user event streams) remain observables and are bridged at the edge (`toSignal`, `toObservable`). `effect` is for side effects (sparingly - prefer `computed`). **Alternatives and trade-offs:** NgRx/SignalStore for large shared state; plain services with subjects. **Risks and pitfalls:** writing signals inside `computed`; overusing `effect` as a hidden data flow; converting everything for its own sake. **Likely follow-ups:** "is `computed` lazy?" (yes, memoised). **In this project:** `AuthService` signals (`user`, `isStaff`...), `toSignal(form.valueChanges)` in the transaction form.

Q66. [Scenario] A form submits twice when the user double-clicks "Submit" on a slow network. How do you make that safe end to end?

Short answer: disable the button while pending, make the request idempotent with a stable key, and let the server enforce it. **Reasoning and implementation:** client: ignore submit while `busy`, generate the idempotency key once per logical payload (`IdempotencyKeys.keyFor`) and reuse it on retry; reset only after a definitive outcome. Server: unique `(principal, key)` constraint replays the original result. After an *unknown* failure (timeout) the user retries with the *same* key and cannot create a duplicate; after editing the amount a new key is used. **Alternatives and trade-offs:** `exhaustMap` in the stream (helps the client only), server-side time-window dedup (fragile). **Risks and pitfalls:** regenerating the key on retry; keys that survive across different payloads (server answers 422). **Likely follow-ups:** "what does the user see on replay?" (a notice that no second transaction was created). **In this project:** `TransactionNewComponent`, specs for "reuses the SAME key" and "new key on edit", `IdempotencyIT`.

Q67. How do Angular interceptors work, and how do you order them?

Short answer: functional interceptors wrap `HttpClient` calls in a chain; the first in the list is the outermost. **Reasoning and implementation:** `provideHttpClient(withInterceptors([error, auth, correlation]))`: request flows correlation -> auth -> error-handling in reverse of the list for the *response*; the outermost interceptor sees the final error after inner ones added headers. Keep them cross-cutting and small; attach tokens only to your API URLs; normalise errors once; sign out on 401 except for the login call. **Alternatives and trade-offs:** per-service header handling (duplication), HTTP client wrapper. **Risks and pitfalls:** sending tokens to third parties, mutating the request instead of cloning, swallowing errors, retry interceptors on non-idempotent calls. **Likely follow-ups:** "how would you add a global loading indicator?" **In this project:** `core/interceptors.ts`, tested in `core.spec.ts`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q68. How do you implement role-aware navigation and why is it not security?

Short answer: filter menu items and guard routes by role for usability, but the API must enforce every permission because the client is untrusted. **Reasoning and implementation:** `AuthService` exposes role signals; `ShellComponent` filters nav items; `roleGuard` redirects to a "not permitted" page; the API independently returns 401/403/404. A malicious user can edit the JS or call the API directly, so the server decides. **Alternatives and trade-offs:** server-driven UI (permissions returned by the API) to avoid duplicating rules. **Risks and pitfalls:** hiding buttons and assuming safety; role names drifting between client and server. **Likely follow-ups:** "where should the role list come from?" (token claims / `/me`). **In this project:** `ShellComponent`, `roleGuard`, E2E "a customer is redirected from /audit".

Q69. [Scenario] Users complain the dashboard flickers "Loading..." every 15 seconds. How do you fix it?

Short answer: separate *initial loading* from *background refresh* and keep the last good data visible. **Reasoning and implementation:** the view state is derived from `summary()` presence: spinner only until the first data arrives; a refresh failure sets a non-blocking warning instead of replacing the content; refresh only while the tab is visible; avoid overlapping requests (`switchMap`/ ignore while in flight). **Alternatives and trade-offs:** server-sent events/WebSocket push (less polling, more infrastructure). **Risks and pitfalls:** stale data without a visible timestamp; polling storms with many users. **Likely follow-ups:** "how would you add push updates?" **In this project:** `DashboardComponent` (tests: stale-data warning, error + retry).

Q70. How do you design Reactive Forms with cross-field validation and accessible errors?

Short answer: typed controls, field validators for syntax, a group validator for cross-field rules, errors shown after touch and announced to assistive tech. **Reasoning and implementation:** `transactionShapeValidator` checks type vs accounts (deposit needs target, transfer needs two different accounts); amount uses a pattern and a non-zero validator; `markAllAsTouched` on submit; errors use `mat-error` and `role="alert"`; the submit handler returns early if invalid or busy; the same rules mirror the server's, which stays authoritative. **Alternatives and trade-offs:** template-driven forms (simple forms), schema-driven forms. **Risks and pitfalls:** client-only validation; unlabeled controls; errors only by colour. **Likely follow-ups:** "async validators?" (debounce, cancel). **In this project:** `transaction-new.component.ts` and its spec.

Q71. How do you test Angular components and services?

Short answer: `TestBed` with real templates, `HttpTestingController` for HTTP, assert user-visible behaviour, keep tests deterministic. **Reasoning and implementation:** services/interceptors/ guards: configure providers and flush requests; components: create the fixture, set inputs with `setInput`, trigger events, flush HTTP, assert DOM text/roles/requests; use `fakeAsync` for timers; destroy fixtures to stop intervals; use `ctl.verify()`. A browser E2E script covers whole journeys. **Alternatives and trade-offs:** Jest/Vitest (faster, jsdom), Playwright/Cypress for E2E and component tests. **Risks and pitfalls:** testing private fields; unflushed requests; forgetting `forkJoin` cancels siblings. **Likely follow-ups:** "how do you test a route guard?" (`runInInjectionContext`). **In this project:** 58 specs; `core.spec.ts`, `transaction-new.component.spec.ts`, `dashboard.component.spec.ts`.

Q72. What are the main secure-frontend practices for an Angular app handling financial data?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: avoid XSS sinks, keep tokens out of web storage, use a strict CSP, same-origin API, validate redirects, and never trust the client for authorization. **Reasoning and implementation:** rely on Angular's sanitisation (no `bypassSecurityTrust*`/`innerHTML`); hold the access token in memory (or HttpOnly cookie via a BFF); CSP `default-src 'self'` from nginx; proxy `/api` so no CORS and no third-party origins; validate `returnUrl`; `npm audit` and lockfile; money as strings; no secrets in the bundle; errors without internals. **Alternatives and trade-offs:** BFF with cookies (needs CSRF protection), service-worker token handling. **Risks and pitfalls:** third-party scripts (analytics), `localStorage` tokens, open redirects. **Likely follow-ups:** "what is Trusted Types?" **In this project:** ADR-012, `nginx.conf`, login open-redirect tests.

Q73. How do you optimise Angular performance?

Short answer: reduce work per change detection, reduce bytes, reduce requests. **Reasoning and implementation:** `OnPush` + signals; `track` in `@for`; lazy routes and `@defer`; bundle budgets (the build fails over budget); pure pipes/`computed` instead of template function calls; virtual scroll for long lists; HTTP caching and request de-duplication; measure with Angular DevTools/Lighthouse; avoid polling storms. **Alternatives and trade-offs:** SSR/hydration for first paint; zoneless. **Risks and pitfalls:** premature optimisation; huge Material imports; memory leaks from subscriptions. **Likely follow-ups:** "how do you find what is slow?" **In this project:** initial bundle ~341 kB raw (~97 kB transferred), lazy features.

Q74. How would you design REST endpoints for transactions (naming, verbs, status codes)?

Short answer: nouns for resources, verbs only for commands, standard codes, and an error envelope. **Reasoning and implementation:** `POST /transactions` (202 + Location), `GET /transactions/{id}`, `GET /accounts/{id}/transactions`, `POST /transactions/{id}/cancel|approve|reject` as commands; `422` for valid-but-rejected business outcomes; `409` for state conflicts; `404` for foreign objects; idempotency header; pagination with whitelisted sort. **Alternatives and trade-offs:** `PATCH /transactions/{id} {status}` (generic but hides rules), GraphQL mutations. **Risks and pitfalls:** leaking internal states as free-form PATCHes, using 200 for everything, unclear 400 vs 422. **Likely follow-ups:** "400 vs 422?" (malformed/invalid input vs semantically valid but rejected by rules). **In this project:** API chapter table.

Q75. How do you implement pagination, filtering and sorting safely?

Short answer: cap page size, whitelist sortable fields, parameterise filters, and add a deterministic tie-break. **Reasoning and implementation:** `size <= 100`; `PageableSupport.restrictSort` rejects unknown properties (no internal columns, no unindexed sorts) and appends `id`; filters are explicit query parameters mapped to JPA Specifications (parameterised); the envelope is stable (`items, page, size, totalElements, totalPages`); for deep pages use keyset pagination. **Alternatives and trade-offs:** cursor pagination (stable under inserts), `count(*)` cost avoidance (omit totals). **Risks and pitfalls:** deep `OFFSET`, unstable ordering causing duplicates/missing rows between pages, exposing Spring's `Page` JSON. **Likely follow-ups:** "why a tie-break on id?" **In this project:** `PageableSupport`, `TransactionFlowIT.searchAndPagination`.

Q76. [Scenario] A client reports that after your release their integration breaks. What might have caused it and how do you prevent it?

Short answer: a breaking change slipped into a non-breaking version: removed/renamed field, changed type/scale, tightened validation, new required input, new enum value, changed status code. **Reasoning and implementation:** compare the OpenAPI document before/after; run consumer contract tests; check logs by client id for the failing calls. Prevent with a committed contract and CI diff, additive-only rules, tolerant-reader guidance, deprecation headers, and a canary/ring rollout for risky changes. Roll back quickly if the change is not additive. **Alternatives**

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

and trade-offs: consumer-driven contract tests (Pact) give early warnings from real consumers. **Risks and pitfalls:** "harmless" changes like different decimal scale or ordering of arrays. **Likely follow-ups:** "how do you version?" **In this project:** `ApiContractIT`, CI step, ADR-007.

Q77. How do you handle rate limiting and what do you return?

Short answer: token bucket per caller; `429` with `Retry-After`; stricter limits on login. **Reasoning and implementation:** key by authenticated subject (or address when anonymous); bucket capacity allows bursts, refill rate sets sustained throughput; return the standard error envelope and `Retry-After`; per-instance in-memory here - distributed deployments need a shared store (Redis) or gateway enforcement; also protect expensive endpoints separately. **Alternatives and trade-offs:** fixed/sliding window counters, leaky bucket, gateway-level limiting (WAF). **Risks and pitfalls:** limiting by IP behind a proxy (everyone looks the same - configure forwarded headers), unbounded key maps. **Likely follow-ups:** "how do you test it?" (tiny limits in a separate test context). **In this project:** `TokenBucketRateLimiter`, `RateLimitIT`.

Q78. What goes into a good error response, and what must never be in it?

Short answer: stable code, safe human message, field violations, correlation id; never stack traces, SQL, class names, secrets or echoed payloads. **Reasoning and implementation:** clients branch on `code`; humans read `message`; operations use `correlationId` to find logs; validation errors list `field` + message; 500s are generic. Tests assert that internal exception text never appears. **Alternatives and trade-offs:** RFC 7807 problem details (`type`, `title`, `detail`, `instance`). **Risks and pitfalls:** user enumeration through distinct messages ("user not found" vs "wrong password"). **Likely follow-ups:** "should 404 reveal existence?" **In this project:** `ApiError`, `ServerErrorIT`, login tests.

Q79. [Scenario] How would you expose a long-running operation (e.g. a statement export) through REST?

Short answer: accept the request asynchronously (`202` + job resource), let the client poll or be notified, and make the job idempotent and cancellable. **Reasoning and implementation:** `POST /exports` with an idempotency key -> `202 Accepted` + `Location: /exports/{id}`; job states `QUEUED/RUNNING/COMPLETED/FAILED`; the result is a time-limited download link (pre-signed URL); progress via polling with `Retry-After` or webhooks; authorisation on the job and the file; limits per customer; stream, never load all rows in memory. **Alternatives and trade-offs:** synchronous streaming for small exports; server-sent events. **Risks and pitfalls:** unbounded memory, orphaned jobs, links that outlive permissions. **Likely follow-ups:** "how do you resume after a crash?" (job state in the DB, idempotent steps). **In this project:** the transaction flow itself follows this shape (`202` + status resource).

Q80. How do you version and document an API for several internal consumers?

Short answer: URI version, generated and committed OpenAPI, examples from the real system, changelog and deprecation policy. **Reasoning and implementation:** springdoc generates the contract from code (so it cannot lie), the file is committed and checked in CI, examples are captured from a live run (`docs/api/examples.md`), Postman/`.http` collections are provided, breaking changes require a new version. **Alternatives and trade-offs:** design-first (write OpenAPI, generate server stubs) - better for cross-team negotiation, needs discipline to keep code in sync. **Risks and pitfalls:** hand-written docs that drift; undocumented error codes. **Likely follow-ups:** "code-first or design-first?" **In this project:** `specs/openapi.json`, `scripts/capture-api-examples.mjs`, `tests/api`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Security

Q81. Explain OAuth 2.0 and OpenID Connect and which flow you would use for an Angular SPA.

Short answer: OAuth 2.0 delegates limited access to an API; OIDC adds authentication (an ID token) on top; the SPA should use the authorization-code flow with PKCE. **Reasoning and implementation:** the SPA redirects to the authorization server, the user authenticates there, the SPA receives a one-time code and exchanges it (with the PKCE verifier) for tokens; the API validates the access token as a resource server. Implicit and password grants are obsolete. A backend-for-frontend can hold tokens server-side and give the browser an HttpOnly, SameSite cookie. **Alternatives and trade-offs:** BFF + cookies (strongest browser story, needs CSRF protection and a server), SPA-held tokens (simpler, more XSS exposure), client credentials for service-to-service. **Risks and pitfalls:** tokens in `localStorage`, missing state/nonce validation, long-lived access tokens, wildcard redirect URIs. **Likely follow-ups:** "what does PKCE protect against?" (code interception); "ID token vs access token?" **In this project:** local demo issuer for convenience only; production-style validation uses `OIDC_ISSUER_URI`; ADR-008/012.

Q82. How do you validate a JWT correctly?

Short answer: verify the signature with the issuer's key, restrict the algorithm, check `exp/nbf/iss/aud`, then map claims to authorities. **Reasoning and implementation:** resolve the key by `kid` from the JWKS (cached, refetched on unknown `kid`); allow-list algorithms (never `none`, never let the token pick HS vs RS); validate time claims with small skew; check issuer and audience; ignore unknown role names; treat the token as signed-but-readable (no secrets inside). **Alternatives and trade-offs:** opaque tokens with introspection (easy revocation, extra hop), mTLS-bound tokens. **Risks and pitfalls:** accepting any issuer, trusting claims from an unverified source, long TTLs without revocation, logging tokens. **Likely follow-ups:** "how do you revoke a JWT?" (short TTL + refresh rotation, deny-list for emergencies). **In this project:** `SecurityConfig`, `LocalIssuerConfig` (timestamp + issuer validators), `SecurityIT` (forged, tampered, expired tokens).

Q83. [Scenario] A pen-tester shows that changing the id in /accounts/{id} returns another customer's data. How do you respond?

Short answer: treat it as a high-severity BOLA defect: reproduce, fix with an object-level policy, add tests for every endpoint taking an id, review all similar endpoints, and check logs for past exploitation. **Reasoning and implementation:** reproduce with two users; write the failing test; add `AccessPolicy.canViewAccount` in the service that loads the object; return 404 and audit; sweep `balance`, `history`, `ledger`, transaction read/cancel, search filters and list endpoints (force the customer scope server-side). Look at access logs/audit for unusual id probing (many 404s per subject) and assess disclosure obligations with the security team. **Alternatives and trade-offs:** database row-level security as defence in depth; gateway rules cannot see ownership. **Risks and pitfalls:** patching one endpoint only; relying on unguessable UUIDs ("security through obscurity"). **Likely follow-ups:** "how do you prevent recurrence?" (review checklist, a BOLA test per endpoint, shared policy). **In this project:** `SecurityIT` BOLA tests, `AccessPolicy`.

Q84. How do you protect against SQL injection in a JPA application?

Short answer: never concatenate input into queries; use bound parameters (JPQL/Criteria/Spring Data), validate input, and give the database account least privilege. **Reasoning and implementation:** search strings are passed as parameters (`cb.like(..., pattern)`), sort fields come from a whitelist, `LIKE` wildcards are stripped from user text; native queries use named parameters; the DB role cannot alter schema; tests send injection strings and assert they behave as data. **Alternatives and trade-offs:** stored procedures (not inherently safer), ORM-generated queries (safe if used correctly). **Risks and pitfalls:** `@Query` with string concatenation, dynamic

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

`ORDER BY` from user input, second-order injection. **Likely follow-ups:** "are `ORDER BY` columns parameterisable?" (no - whitelist). **In this project:** `SecurityIT.inputHardening`, `PageableSupport`.

Q85. [Scenario] How do you design secrets management for this system across local, CI and production?

Short answer: no secrets in the repository; local uses an untracked `.env`; CI uses encrypted repository/environment secrets (prefer OIDC federation); production uses a secret manager surfaced to pods at runtime. **Reasoning and implementation:** compose refuses to start without required variables; Kubernetes references `secretKeyRef`, populated by External Secrets from AWS Secrets Manager (IRSA) with rotation; the app fails closed without token validation; gitleaks and a pattern scan run in CI; a leak is handled by rotating first. **Alternatives and trade-offs:** Vault dynamic credentials (short-lived DB users), sealed secrets (GitOps-friendly), mounted files vs env. **Risks and pitfalls:** secrets in images/logs/actuator `env`, shared credentials across environments. **Likely follow-ups:** "how do you rotate the DB password without downtime?" **In this project:** `secrets-management.md`, `.env.example`, `external-secret.example.yaml`.

Q86. What are the OWASP API Security Top 10 risks most relevant to a payments API?

Short answer: BOLA, broken authentication, broken function-level authorization, unrestricted resource consumption, unrestricted access to sensitive business flows, and excessive data exposure. **Reasoning and implementation:** map each to controls: object-level policy; JWT validation + login throttling; `@PreAuthorize`; rate/body/page limits; idempotency + review thresholds + maker-checker; DTOs and masking. Add logging/monitoring for abuse patterns (repeated `ACCESS_DENIED`). **Alternatives and trade-offs:** API gateway/WAF for generic controls (cannot replace domain checks). **Risks and pitfalls:** treating the list as a checklist instead of testing each. **Likely follow-ups:** "what is 'sensitive business flow' abuse?" (bulk account creation, scripted redemption, transaction spamming). **In this project:** `owasp-mapping.md` with evidence per risk.

Q87. [Scenario] You must secure the admin operations (drain, requeue, resolve). What controls do you apply?

Short answer: role restriction, separation of duties where money is affected, full audit, network isolation and rate limiting. **Reasoning and implementation:** `ADMIN`-only `@PreAuthorize` for outbox/drain; reconciliation resolution by operators with maker-checker; every action audited with actor/role/correlation id; management endpoints on a separate port restricted by NetworkPolicy; step-up authentication/MFA at the IdP for privileged roles; break-glass accounts monitored; no destructive bulk operations without confirmation. **Alternatives and trade-offs:** four-eyes approval for every admin action (slower), just-in-time privilege elevation. **Risks and pitfalls:** shared admin accounts, admin APIs on the public ingress, unlogged actions. **Likely follow-ups:** "how do you detect misuse?" (SIEM rules on audit events). **In this project:** `OpsController`, `SecurityIT.roleRules`, `TransactionFlowIT` (support cannot resolve).

Q88. How do you prevent sensitive data from leaking through logs and errors?

Short answer: do not log it, mask it where you must show it, and sanitise errors centrally. **Reasoning and implementation:** no tokens/bodies/PII in logs; account numbers masked; audit text stores field names not values; one error handler returns generic 500s; correlation id lets support find details server-side; structured logs with allow-listed fields; the OTel collector strips auth headers; tests assert that internals do not reach the client. **Alternatives and trade-offs:** log redaction filters (safety net, not the plan), tokenisation of identifiers. **Risks and pitfalls:** `toString()` of entities in logs, logging request bodies for debugging, third-party SDK logging. **Likely follow-ups:** "how do you debug production issues without PII?" (correlation ids, synthetic repro). **In this project:** `Masking`, `ServerErrorIT`, `AuditService` usage.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q89. What is CSRF and does this API need protection?

Short answer: CSRF abuses automatically attached credentials (cookies); an API that authenticates with an `Authorization` header does not need CSRF tokens, but the decision must be explicit and revisited if cookies are introduced. **Reasoning and implementation:** browsers add cookies to cross-site requests but never add custom headers on their own, so a forged form post carries no bearer token. CORS allow-list limits which origins can read responses. If a BFF with cookie sessions were used, enable CSRF protection (SameSite cookies + tokens). **Alternatives and trade-offs:** double-submit cookie, SameSite=Strict. **Risks and pitfalls:** disabling CSRF "to make it work" in a cookie-based app. **Likely follow-ups:** "what about CORS preflight?" **In this project:** `SecurityConfig` comment, `SecurityIT.cors`.

Q90. [Scenario] Login is being brute-forced. What do you do?

Short answer: throttle by client and account, add monitoring/lockout carefully, require stronger factors at the IdP, and keep error messages uniform. **Reasoning and implementation:** per-address token bucket on the login endpoint (10/min), identical 401 messages for unknown user and wrong password, constant-time comparison path, alert on spikes of `LOGIN_FAILED` audit events, temporary per-account backoff (avoid permanent lockout - it enables denial of service), MFA/WAF bot rules at the edge. **Alternatives and trade-offs:** CAPTCHA, device reputation, credential-stuffing databases checks. **Risks and pitfalls:** lockouts that let an attacker lock victims out; IP limits behind a proxy. **Likely follow-ups:** "how do you test rate limits?" **In this project:** `RateLimitFilter`, `RateLimitIT.loginIsThrottled`, `SecurityIT.localLogin`.

Q91. How do you secure containers and Kubernetes workloads?

Short answer: minimal non-root images, read-only filesystem, dropped capabilities, restricted pod security, network policies, no mounted service-account tokens, scanned images and externalised secrets. **Reasoning and implementation:** multi-stage builds with a JRE-only runtime and a dedicated user; `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`, `seccompProfile: RuntimeDefault`; namespace with the *restricted* pod-security standard; default-deny NetworkPolicy with explicit allows; Trivy in CI; RBAC least privilege; admission policies (OPA/Kyverno) to enforce. **Alternatives and trade-offs:** distroless/chiselled images (smaller attack surface, harder debugging), gVisor/Kata for stronger isolation. **Risks and pitfalls:** running as root "temporarily", privileged containers, broad egress. **Likely follow-ups:** "how do you debug a read-only container?" (ephemeral debug containers). **In this project:** manifests and `validate-infra.py` assertions.

Q92. [Scenario] How would you handle a dependency with a critical CVE the day before a release?

Short answer: assess exploitability in your context, patch or mitigate, run the full gates, and communicate; do not skip the pipeline. **Reasoning and implementation:** identify the affected component with the SCA report and `mvn dependency:tree`; check whether the vulnerable code path is reachable; upgrade (BOM-managed if possible) or apply a mitigation (config flag, WAF rule); run the full test suite and the scans again; if no safe fix, document a time-boxed exception with compensating controls and an owner; consider delaying the release. **Alternatives and trade-offs:** hot-patch only the vulnerable library vs full dependency refresh (risk of regressions). **Risks and pitfalls:** blindly bumping major versions; suppressing findings without analysis. **Likely follow-ups:** "what is an SBOM and why generate one?" **In this project:** CI `security-scan` job, ADR-009, release checklist.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Testing, CI/CD and cloud

Q93. How would you structure a test strategy for a financial transaction service?

Short answer: a pyramid biased toward fast unit tests of pure rules, targeted integration tests on real infrastructure for locks/constraints/security, and a few end-to-end journeys - with the money invariants asserted after every scenario. **Reasoning and implementation:** unit: money maths, state machine, validators, policies, retry logic, orchestration branches with mocks; integration: Spring context + Flyway + real JWT validation, security/BOLA, idempotency races, concurrency on a real PostgreSQL, constraint tests, contract test; E2E: live stack incl. a simulator with chaos; performance: arrival-rate load with invariant checks. Every defect gets a regression test. **Alternatives and trade-offs:** contract tests with consumers (Pact), property-based tests for money, mutation testing tools (PIT) to measure test strength. **Risks and pitfalls:** testing only happy paths, mocking the database where locking matters, coverage as the goal. **Likely follow-ups:** "how do you test concurrency?" **In this project:** `docs/testing/test-strategy.md`, `ConcurrencyScenarios`, `assertLedgerIntegrity`.

Q94. [Scenario] Walk through a TDD cycle for a new business rule.

Short answer: write a failing test that fails for the right reason, make it pass with the simplest code, refactor under green, then add integration tests and document acceptance criteria. **Reasoning and implementation:** daily withdrawal limit: acceptance criteria first; six unit tests against a stub that throws (6 errors - the red evidence); one-line implementation using `compareTo` (6 pass); wire into the validator; refactor so advisory and authoritative checks share the rule; move the authoritative check under the account lock; integration and concurrency tests prove the HTTP path and the race. **Alternatives and trade-offs:** test-after (faster initially, weaker design feedback), BDD outside-in. **Risks and pitfalls:** tests that pass for the wrong reason, testing implementation details, skipping refactor. **Likely follow-ups:** "what is the smallest step?" **In this project:** `docs/testing/tdd-example.md` with the real red/green output.

Q95. How do you write good BDD scenarios and avoid brittle ones?

Short answer: express business rules with concrete examples in domain language, one rule per scenario, always including the invariant that must still hold. **Reasoning and implementation:** Given/When/Then with numbers (balance 100.00, withdraw 100.01); scenario outlines for boundary tables; bind each scenario to automated tests (here by name) so wording and behaviour stay in sync; avoid UI details in business scenarios. **Alternatives and trade-offs:** Cucumber/Behave glue code (readable specs, extra layer to maintain), plain JUnit with descriptive names. **Risks and pitfalls:** scenarios that describe implementation steps; huge Backgrounds; scenarios nobody reads. **Likely follow-ups:** "who writes them?" (three amigos: product, dev, QA). **In this project:** `tests/bdd/transactions.feature`, `bdd-scenarios.md`.

Q96. [Scenario] A test passes locally but fails intermittently in CI. How do you approach it?

Short answer: treat flakiness as a bug: find the shared state, timing or ordering dependency, reproduce deterministically, and fix the cause. **Reasoning and implementation:** look for time/clock dependence, shared database state between tests, unbounded waits, concurrency without proper synchronisation, port/ephemeral resource collisions, test-order dependence; rerun with a seed/ordering and with CPU contention; replace sleeps with bounded waits or deterministic drains; isolate data (each test creates its own accounts); add `@Timeout`. Never add blind retries. **Alternatives and trade-offs:** quarantine with an owner and deadline (temporary). **Risks and pitfalls:** normalising flakiness; retrying until green. **Likely follow-ups:** "what if it only fails under load?" **In this project:** examples: the latch deadlock (pool smaller than task count), the cold-start

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

timeout in the HTTP client test (warm-up added), port collisions with other local services (port guard in `run-local.sh`).

Q97. How do you test code that calls an unreliable external system?

Short answer: put it behind a port, test the adapter against a real HTTP server for protocol behaviour, test the domain with a scripted double for outcomes, and rehearse failure with a simulator. **Reasoning and implementation:** `RestClearingClientTest` uses an in-JVM HTTP server to produce 503s, delays, dropped connections and verifies retries/timeouts/breaker/classification; `ScriptedClearingClient` (cents-driven) drives integration scenarios deterministically; the simulator service offers chaos modes for end-to-end runs; contract tests when a sandbox exists. **Alternatives and trade-offs:** WireMock/MockServer (feature-rich), consumer-driven contracts. **Risks and pitfalls:** mocks that return only success; assuming the real system behaves like the mock. **Likely follow-ups:** "how do you keep the simulator honest?" (contract tests against the provider's sandbox). **In this project:** the three tests above plus chaos e2e.

Q98. [Scenario] How would you design a CI/CD pipeline with security gates?

Short answer: fast feedback first, then deeper scans, then build-once-promote with scans on the artefact, protected deployment, smoke tests and automatic rollback. **Reasoning and implementation:** PR: compile, unit and integration tests, lint, SAST (SpotBugs/CodeQL), SCA (OWASP Dependency-Check, `npm audit`), secrets scan (gitleaks), config validation; main: build images once, scan (Trivy) with a severity gate, tag with a version, push; deploy to test via kustomize with `rollout status`, run smoke tests, roll back on failure; promote the *same* image to production with manual approval; pipeline hygiene: least-privilege tokens, pinned actions, OIDC federation, protected environments, SBOM/provenance. **Alternatives and trade-offs:** Jenkins/GitLab CI; DAST in a deployed environment; admission control to enforce signed images. **Risks and pitfalls:** gates that are always overridden, scans that only warn, building different artefacts per environment. **Likely follow-ups:** "what if the scan finds a CVE with no fix?" (documented exception, compensating controls). **In this project:** `.github/workflows/ci.yml` (reviewed, not run on a runner), ADR-009.

Q99. What is the difference between continuous delivery and continuous deployment, and which fits a bank?

Short answer: delivery keeps every build releasable with a manual gate to production; deployment releases automatically. Regulated environments usually choose delivery with controlled approvals. **Reasoning and implementation:** audit and change-control requirements favour an explicit approval, change records and segregation of duties, while the pipeline still automates everything else (tests, scans, deploys to test, smoke checks). Feature flags decouple deployment from release. **Alternatives and trade-offs:** fully automated deployment with progressive delivery (canary + automated analysis) once risk controls are automated. **Risks and pitfalls:** manual steps that are skipped under pressure; approvals without information. **Likely follow-ups:** "how do you do emergency fixes?" **In this project:** `deploy-test` automatic, production promotion manual by design.

Q100. [Scenario] A release must roll back after a bad deploy that included a database migration. What is your plan?

Short answer: roll back the application (the previous version still works) and fix forward on the schema; use point-in-time recovery only as a last resort. **Reasoning and implementation:** migrations are additive (expand/contract), so the old application tolerates the new schema; `kubectl rollout undo` restores the previous ReplicaSet in minutes; the ledger-integrity check, reconciliation queue and outbox backlog confirm health; data written by the new version in new columns is ignored by the old; if a migration corrupted data, restore a copy via PITR and repair from the audit trail before switching traffic. **Alternatives and trade-offs:** blue/green with a replicated database

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

(heavy), down-migrations (rarely safe). **Risks and pitfalls:** destructive migration in the same release; migrations run concurrently by every replica at startup. **Likely follow-ups:** "how do you test rollback?" (rehearse in the test environment). **In this project:** `rollback-and-release.md`, migration guide.

Q101. How do Kubernetes probes and rolling updates provide zero-downtime deployments?

Short answer: readiness gates traffic to new pods, `maxUnavailable: 0` keeps old pods until new ones are ready, and graceful shutdown drains old pods. **Reasoning and implementation:** a new pod starts (startup probe tolerates Flyway/warm-up), becomes ready, receives traffic; an old pod gets `preStop: sleep 10` (load balancer deregisters), then SIGTERM -> Spring drains in-flight requests (20 s); `terminationGracePeriodSeconds` exceeds both; PDB prevents voluntary eviction below one pod; HPA handles scale. **Alternatives and trade-offs:** blue/green, canary with a service mesh. **Risks and pitfalls:** readiness that depends on flaky dependencies (flapping), liveness too aggressive (restart loops), grace period shorter than drain. **Likely follow-ups:** "what happens to long-lived connections?" **In this project:** `platform-service.yaml`.

Q102. How do you design the AWS deployment for this platform?

Short answer: ALB + WAF in front, EKS (or ECS) in private subnets across AZs, RDS PostgreSQL Multi-AZ with KMS and PITR, Secrets Manager via External Secrets/IRSA, observability through managed Prometheus/Grafana or CloudWatch/X-Ray. **Reasoning and implementation:** security groups per tier, VPC endpoints for AWS APIs, ECR with immutable tags and scan-on-push, GitHub Actions OIDC to AWS (no static keys), autoscaling (HPA, cluster autoscaler), backups and DR (RPO/RTO targets), environment-specific config through overlays, cost controls. **Alternatives and trade-offs:** ECS Fargate (simpler), Lambda for edge functions, Aurora PostgreSQL (faster failover, cost). **Risks and pitfalls:** public database, overly broad IAM, single-AZ, no restore tests. **Likely follow-ups:** "how do you fail over the database?" **In this project:** AWS deployment diagram and chapter (conceptual; no cloud account was used).

Q103. [Scenario] The pipeline takes 40 minutes. How do you speed it up without reducing quality?

Short answer: parallelise independent jobs, cache dependencies, run the cheapest/most informative checks first, and push detail down to faster tests. **Reasoning and implementation:** split into jobs (backend, frontend, scans, config validation) that run in parallel; cache Maven/npm; fail fast on lint/unit before integration; reduce Spring context starts (shared context, fewer classes); use Testcontainers reuse; run heavy scans on main or nightly while keeping PR-critical gates; shard long suites; measure each stage first. **Alternatives and trade-offs:** larger runners, test-impact analysis. **Risks and pitfalls:** moving security scans out of the merge path entirely; flaky parallel tests sharing state. **Likely follow-ups:** "what do you never skip?" **In this project:** jobs are parallel in `ci.yml`; integration tests share one cached context (~45 s each new context locally).

Q104. What do you put in a smoke test, and what do you leave out?

Short answer: cheap, read-only, fast checks that the release is alive and security posture did not regress; leave business logic to automated suites. **Reasoning and implementation:** frontend served, health endpoint, protected endpoints answer 401, actuator not exposed, security headers present; no data creation in production; run after each deployment and automatically trigger rollback on failure. **Alternatives and trade-offs:** synthetic transactions (canary customers) for deeper confidence. **Risks and pitfalls:** smoke tests that mutate production data, or that always pass. **Likely follow-ups:** "synthetic monitoring?" **In this project:** `scripts/smoke-test.sh`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q105. How do you measure test quality beyond coverage?

Short answer: by whether tests fail when the code is wrong - mutation thinking or tooling - and by defects escaping to production. **Reasoning and implementation:** delete/invert a protected line and verify a test fails (done here: removing the row lock fails 4 of 5 concurrency scenarios; removing the lock ordering fails the lock-order unit test); mutation testing tools (PIT) automate it; track escaped defects and add regression tests; review tests for restating implementation. **Alternatives and trade-offs:** coverage thresholds (cheap signal), property-based testing. **Risks and pitfalls:** coverage targets driving meaningless tests. **Likely follow-ups:** "what coverage do you require?" **In this project:** mutation check results in the validation report.

Q106. How do you test database migrations?

Short answer: apply them from scratch in tests against the real database type, verify constraints/objects exist, and rehearse upgrades with realistic data. **Reasoning and implementation:** Flyway runs in every integration test (H2 and PostgreSQL); `PostgresFeaturesIT` asserts partial indexes and the audit trigger; constraint tests bypass the application layer; Hibernate `validate` checks mappings; for upgrades, run migrations on a copy of production-size data to measure locks and duration. **Alternatives and trade-offs:** schema diff tools, shadow databases. **Risks and pitfalls:** testing only against H2; migrations that pass on empty tables and lock large ones. **Likely follow-ups:** "how do you test a data backfill?" **In this project:** `SchemaConstraintsIT`, `PostgresFeaturesIT`.

Q107. [Scenario] Your Spring context takes 50 seconds to start in tests and the suite is slow. What do you do?

Short answer: share and cache contexts, trim the context, and move logic into unit tests. **Reasoning and implementation:** Spring caches contexts by configuration, so use one base class and avoid per-class property tweaks (each variation is a new context - the project deliberately has separate contexts only for the rate-limit and PostgreSQL variants); disable unnecessary auto-configuration; lazy initialisation for tests; slice tests (`@WebMvcTest`, `@DataJpaTest`) for focused cases; push rule coverage into pure unit tests. **Alternatives and trade-offs:** `@DirtiesContext` (avoid), parallel test execution (needs isolation). **Risks and pitfalls:** hidden shared state between tests when sharing contexts (here every test creates its own accounts). **Likely follow-ups:** "when is a new context justified?" **In this project:** `AbstractIntegrationTest` design; documented in the integration guidance.

Q108. What would you check in a pull request that changes the CI pipeline itself?

Short answer: that security gates are not weakened, permissions are not widened, secrets are not exposed, and the change is reversible. **Reasoning and implementation:** diff of `permissions`, new third-party actions (pin by SHA, provenance), changes to `if:` conditions (do gates still run on PRs?), secret usage (never echoed, not available to untrusted forks), thresholds (CVSS cut-off), caching keys, environment protection; run the workflow on a branch first. **Alternatives and trade-offs:** CODEOWNERS requiring security review for `.github/**`. **Risks and pitfalls:** `pull_request_target` with untrusted code, long-lived tokens. **Likely follow-ups:** "how do you secure forked-PR workflows?" **In this project:** pipeline design notes in `ci.yml` and ADR-009.

Performance and reliability

Q109. [Scenario] How would you investigate a slow Spring Boot API?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: go top-down from the user-visible symptom to the saturated resource and the responsible code, measuring at each step, then fix one thing and re-measure. **Reasoning and implementation:** (1) which endpoint and which percentile (`http_server_requests` by `uri`; fat tail = queueing/GC/locks, uniform = work per request); (2) a trace to split time into application, database and remote calls; (3) resource saturation: Hikari pending/acquire time, thread pool, GC pauses, CPU throttling; (4) database: `pg_stat_statements`, `EXPLAIN (ANALYZE, BUFFERS)`, lock waits; (5) code: profile with JFR/async-profiler; (6) fix (index, paging, N+1, timeouts, batching) and compare before/after under the same load; (7) add an alert/regression test. **Alternatives and trade-offs:** adding caches or instances first (can hide the cause), APM tools for automatic tracing. **Risks and pitfalls:** optimising without a baseline, averages instead of percentiles, tiny test data. **Likely follow-ups:** "p99 is high but p50 is fine?"; "how do you correlate a user report with a trace?" (correlation id). **In this project:** `performance-troubleshooting.md`, correlation ids, captured plans, baseline script.

Q110. [Scenario] How would you troubleshoot a database connection-pool bottleneck?

Short answer: confirm saturation (`active == max`, `pending > 0`), then find *why connections are held* before touching pool size. **Reasoning and implementation:** metrics: `hikaricp_connections_active/pending/acquire`; database: `pg_stat_activity` (long-running statements, `idle in transaction`), blocking locks; application: thread dump (threads waiting in `getConnection`, others inside a remote call?), leak-detection log. Typical causes: slow queries, a transaction wrapping an HTTP call (the anti-pattern), lock waits, a leak, too many replicas. Fix the cause; scale replicas only if the DB has headroom; consider PgBouncer. **Alternatives and trade-offs:** larger pool (moves the bottleneck to the DB), per-feature pools/bulkheads. **Risks and pitfalls:** treating symptoms; `REQUIRES_NEW` inside loops doubling connection demand. **Likely follow-ups:** "how big should the pool be?" **In this project:** runbook "connection pool exhaustion", troubleshooting exercise 1, `ConnectionPoolSaturated` alert.

Q111. How do you investigate high JVM memory usage or frequent garbage collection?

Short answer: classify (leak vs under-sizing vs allocation churn), capture evidence (heap-after-GC trend, GC log, heap dump, JFR), fix the root cause, verify with load. **Reasoning and implementation:** heap-after-GC low points rising = leak -> heap dump + MAT dominator tree; frequent young GCs = high allocation rate -> JFR allocation profiling, reduce churn; Full GC/humongous allocations = size or large arrays; non-heap growth -> native memory tracking/metaspace/direct buffers; container limit vs `MaxRAMPercentage`. **Alternatives and trade-offs:** change collector (G1/ZGC) after reducing allocation; bigger heap buys time. **Risks and pitfalls:** tuning flags blindly; analysing the wrong build; dumps containing PII (handle carefully). **Likely follow-ups:** "what does `to-space exhausted` mean?" **In this project:** runbook sections "high memory" and "excessive GC".

Q112. [Scenario] CPU is low but latency is high and requests queue up. What is happening?

Short answer: the service is waiting, not computing: thread or connection starvation, lock waits, or a slow dependency. **Reasoning and implementation:** thread dumps show many threads `WAITING` (pool, lock, socket); check `hikaricp_connections_pending`, lock waits in the database, remote-call latency and timeouts, Tomcat thread saturation; confirm with traces. Fix with timeouts, bulkheads/async processing, shorter critical sections, right-sized pools. **Alternatives and trade-offs:** virtual threads (remove thread scarcity, not connection scarcity), more instances (if the bottleneck is per-instance). **Risks and pitfalls:** scaling out against a database bottleneck (makes it worse). **Likely follow-ups:** "how do you prove it is the database?" **In this project:** performance guide "thread starvation".

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q113. How do you load-test correctly and avoid misleading results?

Short answer: use realistic workloads and an open-loop (arrival-rate) generator, warm up, measure percentiles, check invariants, and report the environment honestly. **Reasoning and implementation:** closed-loop tools (fixed VUs) slow down when the system slows, hiding latency (*coordinated omission*); use constant/ramping arrival rate; separate the generator from the system; test with production-like data volumes and database; verify correctness during load (balance never negative, money conserved); record environment and versions; repeat runs. **Alternatives and trade-offs:** replaying production traffic, chaos plus load. **Risks and pitfalls:** laptop numbers presented as capacity, tiny datasets, caches warmed unrealistically. **Likely follow-ups:** "what is Little's law?" (concurrency = rate x latency). **In this project:** k6 arrival-rate scenarios; the local baseline is labelled "developer machine, not a benchmark" and includes invariant checks.

Q114. [Scenario] Your optimisation "should" improve throughput. How do you prove it did?

Short answer: measure before and after with the same workload and environment, change one thing, repeat, and report variance and caveats. **Reasoning and implementation:** the dispatcher change (drain while work exists instead of one batch per tick) was measured on the same machine with the same script: before ~23 transactions/s processed, after ~34 transactions/s; both numbers kept in the repository with the caveat that the machine was also running other workloads; the correctness invariants (balance equals submissions, ledger integrity) were checked in both runs. **Alternatives and trade-offs:** statistical comparison over several runs, controlled hardware. **Risks and pitfalls:** attributing noise to the change; comparing different loads (here the submitted counts differed, so the drain *rate* rather than elapsed time was compared). **Likely follow-ups:** "what would you do next?" (parallel workers, SKIP LOCKED, broker). **In this project:** `docs/validation/perf-baseline-before-drain-loop.json` vs `perf-baseline.json`.

Q115. What is the difference between availability, reliability and resilience, and how do you measure them?

Short answer: availability = fraction of time the service works; reliability = consistent correct behaviour over time; resilience = ability to absorb and recover from faults. Measure with SLIs/SLOs and error budgets. **Reasoning and implementation:** SLIs: successful requests, latency percentiles, correctness checks (ledger integrity = 0 violations), freshness (reconciliation age); SLOs set targets (99.9 %); error budgets guide release pace; resilience is demonstrated by fault injection and game days; MTTR matters as much as MTBF. **Alternatives and trade-offs:** synthetic probes vs real-user metrics. **Risks and pitfalls:** measuring uptime of the process rather than user success; alert fatigue. **Likely follow-ups:** "what is an error budget policy?" **In this project:** `docs/operations/slo.md` (illustrative targets).

Q116. [Scenario] The database goes down for five minutes. What should happen and what do you verify afterwards?

Short answer: requests fail fast, readiness fails to drain traffic, no data is lost, and processing resumes automatically; verify integrity and backlog afterwards. **Reasoning and implementation:** connection acquisition times out in 3 s; API returns 5xx/503 quickly; readiness goes red; the outbox lives in the DB so queued work waits; on return Hikari reconnects, the dispatcher resumes, stuck `PROCESSING` rows move to reconciliation after 5 minutes, idempotent retries by clients are safe. Verify: ledger integrity, outbox backlog and dead letters, reconciliation count, error-rate recovery; write the post-incident review. **Alternatives and trade-offs:** read replicas for degraded read-only service, retry buffers at the edge. **Risks and pitfalls:** connection storms on recovery (cap pools, jitter), assuming failover is transparent without testing. **Likely follow-ups:** "RPO/RTO?"; "how do you practise this?" (game day). **In this project:** performance guide "database outage".

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q117. How do health checks, readiness and liveness differ, and what should each include?

Short answer: liveness answers "restart me?" (process is wedged), readiness answers "send me traffic?" (can serve now), startup protects slow initialisation. **Reasoning and implementation:** liveness should be cheap and independent of dependencies (otherwise a database outage restarts every pod); readiness may include essential dependencies (database) but not optional ones; an additional business health view (staff dashboard) shows degraded components without affecting traffic routing. **Alternatives and trade-offs:** deep health endpoints (useful for humans, dangerous as probes). **Risks and pitfalls:** restart storms, readiness flapping. **Likely follow-ups:** "should readiness include the external network?" (no - degrade, do not remove capacity for deposits). **In this project:** actuator probes on the management port plus `/ops/health`.

Q118. [Scenario] A downstream provider is slow (p95 8 s) but not failing. How do you keep your service healthy?

Short answer: enforce timeouts, isolate the call from request threads, shed or fail fast via the circuit breaker, and degrade only the affected capability. **Reasoning and implementation:** read timeout 2 s with bounded retries; withdrawals are asynchronous (request threads return `202` immediately), so slowness does not consume API capacity; slow timeouts count as breaker failures and open it, after which withdrawals fail fast and are refunded while deposits/transfers continue; ops dashboard shows degraded health; reconciliation resolves in-flight unknowns later. **Alternatives and trade-offs:** adaptive concurrency limits, bulkheads with separate pools, caching provider responses where valid. **Risks and pitfalls:** unbounded queues in front of the slow provider; retries multiplying load. **Likely follow-ups:** "what if it is slow but succeeds?" (timeouts treat it as unknown - reconciliation completes it). **In this project:** chaos e2e (`SLOW`/`FAIL_5XX` modes), `RestClearingClient`.

AI-assisted development and technical leadership

Q119. [Scenario] How would you use GitHub Copilot without compromising code quality?

Short answer: treat it as a fast junior pair: give it precise context, work in small slices with tests first, review every line like a stranger's pull request, and let the pipeline - not the assistant - decide what is done. **Reasoning and implementation:** short always-on repository instructions with the critical rules (money, locks, authorization, retries), path-scoped rules, reusable prompts; specification-driven slices (spec -> failing tests -> minimal implementation); use it most for boilerplate, test scaffolding, explanations and refactors under green tests; require human review with a checklist targeting AI failure patterns (races, missing authorization, money errors, unsafe retries, weak tests); never paste secrets or customer data; disclose AI involvement in PRs; keep the Definition of Done (recorded test runs, scans) unchanged. **Alternatives and trade-offs:** banning assistants (loses productivity), unrestricted use (quality and security risk). **Risks and pitfalls:** automation bias (trusting plausible code), hallucinated APIs/dependencies, licence issues, over-engineering, skills atrophy. **Likely follow-ups:** "what would you never delegate?" (design decisions on invariants and security boundaries). **In this project:** `.github/copilot-instructions.md`, `ai-engineering/`, `review-examples.md`.

Q120. [Scenario] How would you mentor a junior developer whose code passes unit tests but fails under concurrent load?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: make the failure visible and reproducible, teach the mental model (interleavings, atomicity, isolation), explore the option space together, fix test-first, and generalise into habits. **Reasoning and implementation:** (1) have them run two parallel requests and observe the wrong balance - experience beats explanation; (2) whiteboard the timelines and name the anomaly (lost update); (3) compare fixes: row lock, atomic conditional update, optimistic version + retry, serialisable - with trade-offs; (4) write the concurrency test first on the real database, watch it fail, fix, watch it pass; (5) add "what if this runs twice/concurrently/halfway?" to their checklist and the PR template; (6) pair on the next lock-sensitive task, then have them review someone else's. **Alternatives and trade-offs:** giving the answer immediately (fast but shallow), reading material only. **Risks and pitfalls:** blame, overload (one lesson at a time), fixing it for them. **Likely follow-ups:** "how do you know it landed?" (their next PR). **In this project:** `docs/leadership/mentoring.md`, `ConcurrencyScenarios`, mutation check of removing the lock.

Q121. What is specification-driven development and how does it work with AI assistants?

Short answer: write the specification (requirements, acceptance criteria, design, API, tests) first; implementation - by a human or an assistant - is then checked against it. **Reasoning and implementation:** the spec is reviewed before code (design gate), tests are derived from criteria, the assistant implements small slices, reviews compare the diff to the spec, and the spec stays in the repository as living documentation. It reduces misunderstandings, makes assistant output reviewable and avoids the "chat history as requirements" problem. **Alternatives and trade-offs:** ad-hoc prompting (fast, unreviewable), heavy up-front design (slow). **Risks and pitfalls:** specs that rot; over-specifying; treating the spec as immutable. **Likely follow-ups:** "what belongs in a spec?" **In this project:** `specs/001-account-transfer.md`, `workflow-example.md`.

Q122. What is context engineering for coding assistants?

Short answer: designing the information the assistant sees - small, accurate, tiered and testable - so it follows the rules that matter. **Reasoning and implementation:** tier 1 always-on rules (the ten critical ones), tier 2 path-scoped instructions, tier 3 on-demand reference files, tier 4 the source of truth (code, ADRs, tests); each fact lives once; rules are checkable; examples come from the repository; prune rules that never prevent defects; version and review them like code. **Alternatives and trade-offs:** one giant instruction file (diluted attention, stale), none (inconsistent output). **Risks and pitfalls:** prose drifting from code; secrets in instructions. **Likely follow-ups:** "how do you measure whether instructions help?" (defect trends, review findings). **In this project:** `ai-engineering/README.md` design section.

Q123. How do you review AI-generated code for race conditions?

Short answer: ask "what if this runs twice, concurrently, or dies halfway?" for every state change, and demand a test that interleaves. **Reasoning and implementation:** look for read-modify-write without a lock/version/constraint, check-then-act, two writes that must commit together, locks in inconsistent order, work inside a lock, shared mutable state in singletons; demand a latch-based concurrency test asserting final invariants, preferably on the real database; apply a mutation check. **Alternatives and trade-offs:** static analysis helps little here; property-based concurrent testing (jcstress) for in-JVM races. **Risks and pitfalls:** unit tests with mocks that cannot interleave. **Likely follow-ups:** "show me an example." **In this project:** `review-examples.md` pattern 1, `ConcurrencyScenarios`.

Q124. How do you review AI-generated code for missing authorization?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: for each new endpoint or query ask "who can call this, and on whose data?" and require negative tests for a different customer and a different role. **Reasoning and implementation:** authentication being enabled globally gives false comfort; look for ids taken from the path/body without ownership checks, lists without scope, entities returned directly, filters that can be widened; require a role rule and an object-level rule; expect 404 for foreign objects and an audit event. **Alternatives and trade-offs:** centralise policies (one `AccessPolicy`) so assistants reuse them. **Risks and pitfalls:** only testing with an admin token. **Likely follow-ups:** "how do you automate this?" (a test helper that iterates endpoints with a second user). **In this project:** `SecurityIT`, `review-examples.md` pattern 2.

Q125. How do you review AI-generated code for incorrect monetary calculations?

Short answer: grep for `double/float/doubleValue`, check `BigDecimal` comparisons and scale, rounding policy and string handling at boundaries. **Reasoning and implementation:** `valueOf(double)` and `new BigDecimal(double)` bake in binary error; `equals` vs `compareTo`; implicit rounding of input; JSON numbers; percentages and allocations losing cents; tests with 0.1/0.2, ties (half-even), scale differences. **Alternatives and trade-offs:** `long` minor units, a `Money` value type. **Risks and pitfalls:** formatting for display used as a computation. **Likely follow-ups:** "how do you test rounding?" **In this project:** `MoneyPolicy`, `MoneyPolicyTest`, ADR-010.

Q126. How do you review AI-generated retry and timeout code?

Short answer: confirm idempotency, backoff with jitter, caps, timeouts, and honest classification of unknown outcomes. **Reasoning and implementation:** is the operation safe to repeat (keyed by a stable id)? exponential backoff with full jitter? bounded attempts and total time? per-attempt timeout? circuit breaker? what is reported after exhaustion - *unknown* not *failed*? tests that simulate 503s, timeouts and dropped connections against a real HTTP server. **Alternatives and trade-offs:** libraries (Resilience4j/Spring Retry) vs small hand-written executors. **Risks and pitfalls:** annotation-based retries on non-idempotent methods; nested retries across layers. **Likely follow-ups:** "what is a retry storm?" **In this project:** `RetryExecutor`, `RestClearingClientTest`, `review-examples.md` pattern 4.

Q127. [Scenario] An assistant proposes adding Kafka "for reliability". How do you respond?

Short answer: ask what problem it solves that the current design does not, quantify the cost, and decide with an ADR. **Reasoning and implementation:** the transactional outbox already provides atomic hand-off and retry; a broker adds an operational system, failure modes and dual-write risks; it becomes justified with multiple independent consumers, replay needs, cross-service fan-out or throughput beyond a polled table - then publish the outbox to Kafka via a relay without changing producers. **Alternatives and trade-offs:** RabbitMQ (simpler routing), CDC with Debezium. **Risks and pitfalls:** resume-driven design; assistants favour popular stacks regardless of need. **Likely follow-ups:** "what would change your mind?" **In this project:** ADR-006 (no broker/Redis yet, with triggers for adding one).

Q128. How do you upgrade dependencies safely, with or without AI help?

Short answer: one change at a time, through the full pipeline, using the BOM, with release-note review and a scan comparison. **Reasoning and implementation:** Dependabot/Renovate PRs, Boot BOM for aligned versions, read changelogs/migration guides, run unit + integration (incl. PostgreSQL) + E2E, compare Dependency-Check/Trivy results, canary in test, keep upgrades small and frequent; an assistant can summarise notes and propose code changes but the green pipeline decides. **Alternatives and trade-offs:** scheduled batch upgrades, pinning and ignoring (security debt). **Risks and pitfalls:** transitive surprises, major-version jumps, ignoring licence changes.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Likely follow-ups: "how do you handle a breaking Spring Boot major?" **In this project:** pinned versions, CI gates, ADR-009.

Q129. [Scenario] How do you run an architecture decision for a contested choice (e.g. optimistic vs pessimistic locking) within a team?

Short answer: write the problem and constraints, list options with measurable criteria, prototype/measure the risky ones, decide with the people accountable, record it as an ADR and revisit on stated triggers. **Reasoning and implementation:** define forces (contention, latency, correctness cost), create a spike with the concurrency tests on the real database, compare results, capture the decision and consequences (including what would make us change it), publish in the repo, and make tests encode the decision. **Alternatives and trade-offs:** decision by seniority (fast, brittle), consensus forever (slow). **Risks and pitfalls:** ADRs written after the fact; decisions without owners. **Likely follow-ups:** "how do you handle disagreement?" (disagree and commit, with a review date). **In this project:** ADR-005 with the alternatives table.

Q130. What belongs in a production runbook and who writes it?

Short answer: symptoms, impact, first checks, diagnosis steps, mitigations, escalation, verification and links to dashboards/queries - written by the people who build the feature, reviewed by those on call. **Reasoning and implementation:** every alert links to a runbook section (enforced by a validation check), steps are copy-pasteable, they state the *safe default* and what not to do (e.g. do not restart to reset the breaker), include "how to verify recovery", and are updated after every incident. **Alternatives and trade-offs:** automated remediation (when safe). **Risks and pitfalls:** stale runbooks, steps requiring tribal knowledge. **Likely follow-ups:** "how do you test a runbook?" (game days). **In this project:** `runbooks.md`, alert-to-runbook check in `validate-infra.py`.

Q131. [Scenario] During an incident you find the root cause was a change you merged last week. How do you handle it?

Short answer: own it, mitigate first, communicate clearly, then run a blameless review focused on how the system allowed it. **Reasoning and implementation:** roll back or disable the change, state facts and impact without speculation, preserve evidence (logs, correlation ids), after resolution write the review: timeline, contributing factors (missing test, review gap, alert gap), concrete actions with owners and dates including at least one regression test or alert; share learnings. **Alternatives and trade-offs:** none that include hiding it. **Risks and pitfalls:** defensiveness; fixing without understanding; no follow-through on actions. **Likely follow-ups:** "what did you change in the process?" **In this project:** `post-incident-review-template.md`.

Q132. How do you balance delivery speed with engineering rigour as a technical lead?

Short answer: make the safe path the easy path - automation, small batches, clear Definition of Done - and spend rigour where the risk is. **Reasoning and implementation:** risk-tier changes (money/security paths get design review, concurrency tests, second reviewer; copy changes do not); automate gates so rigour costs minutes; keep PRs small; use feature flags; track DORA metrics and incident learnings; protect time for reliability work through error budgets. **Alternatives and trade-offs:** uniform heavy process (slows everything), no process (incidents). **Risks and pitfalls:** hero culture, skipping tests "just this once". **Likely follow-ups:** "how do you say no to a deadline?" (present options with risks, not refusals). **In this project:** Definition of Done, release checklist, risk register.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Q133. [Scenario] A stakeholder asks you to "just add exactly-once delivery". How do you respond?

Short answer: explain what is and is not achievable, and offer effectively-once effects with idempotency, plus reconciliation for what we cannot control. **Reasoning and implementation:** exactly-once is impossible across independent systems in the presence of failures; achievable: at-least-once delivery + idempotent consumers + unique constraints give *effectively-once* effects on our data; for external systems use idempotent requests and reconciliation; show the failure modes and the evidence (concurrency, chaos tests); agree on the SLA for resolving unknown outcomes. **Alternatives and trade-offs:** transactional messaging within one broker (exactly-once semantics inside its boundary only). **Risks and pitfalls:** overpromising; hiding unknown states. **Likely follow-ups:** "what does Kafka's exactly-once cover?" (within Kafka transactions, not external side effects). **In this project:** integrity chapter's callout, ADR-011.

Q134. How do you document a system so that a new engineer is productive in a week?

Short answer: a runnable quick start, a short architecture story, ADRs for the "why", runbooks, and tests that read like specifications. **Reasoning and implementation:** README with one-command setup and verification; architecture overview and diagrams generated from text; module tour mapping concepts to classes; ADRs; API examples captured from the running system; threat model and runbooks; "how to run the tests" and "how to debug" notes; keep each fact in one place; generate what can be generated (OpenAPI, examples) so it cannot lie; onboarding buddy. **Alternatives and trade-offs:** wiki-only docs (drift), no docs (tribal knowledge). **Risks and pitfalls:** diagrams only in someone's drawing tool; outdated screenshots. **Likely follow-ups:** "how do you keep docs current?" (docs in the repo, reviewed with code, validated in CI). **In this project:** `README.md`, `docs/`, both PDFs generated from the same sources.

React frontend edition

Q135. The same API has an Angular and a React client. What is identical, what differs, and how do you keep them from drifting?

Short answer: the contract, the security behaviour and the wording are identical; the component model, state handling and tooling differ; a single end-to-end suite that drives both is what prevents drift. **Reasoning and implementation:** both projects use the same `core/models.ts` (money as strings, `ApiProblem` error shape) and the same `IdempotencyKeys`; both keep the token in memory, add the bearer token to `/api/` calls only, tag calls with a correlation id and end the session on a 401. The React edition replaces signals with hooks, interceptors with one `createHttp()` wrapper, guards with `RequireAuth`/`RequireRole`, and Material with native controls. `tests/e2e/ui-e2e.mjs --flavor angular|react` runs the same 19 checks against each. **Alternatives and trade-offs:** generate both clients from `openapi.json` (best once the API grows), a shared UI kit (couples the builds), a single frontend (no drift, less reach). **Risks and pitfalls:** fixing a bug in one edition only; types that diverge silently; wording differences that make the shared test brittle. **Likely follow-ups:** "why not share a package between the two?" (different compilers and component models; the shared part is 180 lines of types). **In this project:** `docs/architecture/12-react-frontend.md`, ADR-013.

Q136. [Scenario] A list page shows the wrong rows after the user changes a filter twice quickly. What is happening and how do you fix it in React?

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Short answer: an older request finished after a newer one and overwrote its result; ignore any response that no longer matches the current inputs. **Reasoning and implementation:** each `useEffect` run starts a request; responses can arrive out of order. `useAsync` sets a `cancelled` flag in the effect cleanup and drops results when it is set, so only the latest inputs can update state. `AbortController` additionally cancels the network request itself. The Angular edition gets the same effect from `switchMap`. **Alternatives and trade-offs:** `AbortController` plus the flag (cancels the work, slightly more code), React Query or SWR (handle this and caching, add a dependency), debouncing the filter (reduces requests, does not fix ordering). **Risks and pitfalls:** forgetting the cleanup, which also causes state updates after unmount; putting unstable objects in the dependency array so the effect re-runs every render. **Likely follow-ups:** "what does StrictMode do to effects?" (runs setup, cleanup, setup in development to expose exactly this bug). **In this project:** `core/useAsync.ts`; the accounts, transactions, customers and audit pages.

Q137. Where does the React app keep the access token and why not localStorage?

Short answer: in memory (React state and a ref inside the auth provider), so script injected through XSS cannot read it from storage and it disappears with the tab. **Reasoning and implementation:** `AuthProvider` holds the session; `createHttp()` reads the token through a getter on every request and checks expiry; a timer ends the session when the token expires. A unit test asserts that nothing is written to `localStorage` or `sessionStorage`. Cost: a reload requires signing in again. **Alternatives and trade-offs:** HttpOnly, SameSite cookies through a backend-for-frontend (best; needs CSRF protection and a BFF), authorization-code with PKCE and silent renewal, `sessionStorage` (survives reload, readable by XSS). **Risks and pitfalls:** logging tokens, putting them in URLs, assuming memory storage stops XSS (it limits theft, it does not stop actions performed inside the page). **Likely follow-ups:** "how would you add silent renewal?" (BFF or PKCE with refresh-token rotation). **In this project:** `core/auth.tsx`, `core.test.tsx` ("keeps the token in memory only").

Q138. Which useEffect mistakes matter most, and where does this code avoid them?

Short answer: missing or unstable dependencies, effects used for derived data, no cleanup for timers and requests, and unconditional work that should be an event handler. **Reasoning and implementation:** derived values (the status bars, the currency of the selected account) are computed during render, not stored through an effect. The dashboard's 15-second refresh creates its timer in an effect and clears it in the cleanup. Loading goes through `useAsync`, which owns the dependency handling in one place; the two intentional exceptions carry a comment explaining why the lint rule is disabled. **Alternatives and trade-offs:** a data-fetching library, `useSyncExternalStore` for external stores, the React compiler to remove manual memoisation. **Risks and pitfalls:** objects and functions recreated each render in a dependency list (infinite loops), stale closures, effects that set state which re-triggers the same effect. **Likely follow-ups:** "when is `useMemo` worth it?" (measured cost, or to keep a referentially stable dependency). **In this project:** `Home.tsx` (dashboard timer), `useAsync.ts`, `eslint.config.js` (`react-hooks/exhaustive-deps` as an error).

Q139. [Scenario] A user double-clicks "Submit transaction" on a slow network. Disabling the button did not prevent two requests in the React form. Why, and what is the complete fix?

Short answer: `setBusy(true)` is asynchronous, so two clicks in the same tick both read `busy === false`; guard with a ref and make the request idempotent. **Reasoning and implementation:** the handler checks `inFlight.current` (a ref updates synchronously) as well as the `busy` state, and sets the ref before awaiting. Independently, the request carries an `Idempotency-Key` that is reused for an unchanged payload, so even a retry after a timeout cannot create a second transaction; the server returns the original result with `Idempotent-Replayed: true`. A test double-clicks while the response is

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

held back and asserts that exactly one POST was made. **Alternatives and trade-offs:** disabling only (insufficient), debouncing the click (hides the symptom), a request deduplication layer in the HTTP wrapper (reasonable, but the server is the real authority). **Risks and pitfalls:** resetting the key on an unknown failure (breaks the guarantee); resetting the ref in the wrong place. **Likely follow-ups:** "what if the user edits the amount between retries?" (new payload, new key). **In this project:** `features/TransactionNew.tsx`, `TransactionNew.test.tsx` ("disables the button while a request is in flight").

Q140. How are the React components tested, and what is faked?

Short answer: Vitest and Testing Library render the real application and drive it as a user would; only the network is faked, by a scripted `fetch`. **Reasoning and implementation:** `FakeApi` records every request and answers from handlers keyed by `METHOD /path`, so tests assert on what the app sent (headers, query, body) and on what the user sees. Queries are by role and label (`getByRole('button', …)`, `getByLabelText`), which also proves accessibility labels exist. Sign-in goes through the real login call, so the auth provider, guards and routing are exercised together. 90 tests cover the HTTP client, auth, guards, every page and the validation rules. **Alternatives and trade-offs:** mocking modules (faster, tests the mock), MSW (a service worker intercepts real `fetch`; more realistic, more setup), Cypress or Playwright component tests (real browser, slower). **Risks and pitfalls:** asserting on implementation details, `act` warnings from unawaited updates, fake timers that never advance. **Likely follow-ups:** "what does the end-to-end suite add?" (the real backend, the real bundle and CORS; it found the 403 caused by a proxy that rewrote the `Host` header). **In this project:** `src/testing/helpers.tsx`, `core/core.test.tsx`, `features/*.test.tsx`.

Q141. [Scenario] You add the React UI on port 4281 and every sign-in returns 403 although the credentials are right. What is the cause and the safe fix?

Short answer: the API treated the request as cross-origin from an origin that is not on its CORS allow-list; make the request genuinely same-origin by preserving `Host` in the proxy, rather than loosening the allow-list. **Reasoning and implementation:** the 403 was a plain-text rejection from the CORS layer, not the JSON error the API returns for a bad login (a wrong password gives a JSON 401). Browsers send `Origin` on every POST; the API compares it with the request's `Host`. The development proxy had rewritten `Host` to the API's host, so `Origin: http://localhost:4281` no longer matched and 4281 was not allow-listed (default: only 4280). Fix: pass `Host` through, as nginx does (`proxy_set_header Host $host`). Verified: both UIs sign in and pass the same 19 browser checks with the default CORS settings, and a wrong password still shows the generic 401 message. **Alternatives and trade-offs:** add `http://localhost:4281` to `CORS_ALLOWED_ORIGINS` (explicit and safe, but widens a security control for a same-origin setup); a wildcard origin (rejected: removes the control, and cannot be combined with credentials). **Risks and pitfalls:** allow-listing `null` or a wildcard to "make it work"; `changeOrigin: true` in a dev proxy silently changing `Host`; trusting forwarded headers without a trusted-proxy list. **Likely follow-ups:** "why does a same-origin POST send an Origin header at all?" (browsers add it to every non-GET request; the server decides cross-origin by comparing it with its own view of the host). **In this project:** `docs/architecture/12-react-frontend.md`, `scripts/serve-frontend.mjs`, `SecurityIT.cors`, threat T13.

Q142. How do you keep a React bundle small, and what does this one weigh?

Short answer: few dependencies, code-split routes, no UI framework you do not need, and measure with the build output. **Reasoning and implementation:** three runtime dependencies (React, React DOM, React Router); each page is a `lazy()` chunk of 3 to 9 kB; the main chunk is about 279 kB (89 kB gzip) including React and the router; native form controls and plain CSS avoid a component library. The Angular edition's initial bundle is about 341 kB. Memoisation was not added because nothing measured slow. **Alternatives and trade-offs:** preact-compat (smaller,

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

compatibility risk), server-side rendering (faster first paint, needs a Node tier), a component library (productivity, weight). **Risks and pitfalls:** barrel imports that defeat tree shaking, importing a whole date or charting library for one function, optimising without measuring. **Likely follow-ups:** "how do you find what is in the bundle?" (a bundle visualiser, `vite build` size report). **In this project:** `frontend-react/vite.config.ts`, `App.tsx` (lazy routes).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.